

# Contents

|                    |           |
|--------------------|-----------|
| <b>Base64 函数参考</b> | <b>5</b>  |
| 函数列表               | 5         |
| 使用示例               | 5         |
| 错误处理               | 6         |
| <b>字符串操作函数参考</b>   | <b>6</b>  |
| 函数列表               | 6         |
| contains           | 8         |
| starts_with        | 8         |
| ends_with          | 8         |
| find               | 8         |
| substring          | 8         |
| char_at            | 8         |
| replace            | 9         |
| to_upper           | 9         |
| to_lower           | 9         |
| trim               | 9         |
| trim_start         | 9         |
| trim_end           | 10        |
| repeat             | 10        |
| reverse            | 10        |
| byte_len           | 10        |
| split              | 10        |
| join               | 11        |
| to_int             | 11        |
| to_float           | 11        |
| to_str             | 11        |
| <b>内置函数参考</b>      | <b>12</b> |
| 函数一览               | 12        |
| print              | 14        |
| Some               | 14        |
| length             | 14        |
| has_key            | 15        |
| add                | 15        |
| remove             | 15        |
| range              | 15        |
| exit               | 16        |
| args               | 16        |
| sleep              | 16        |
| env                | 16        |
| append             | 17        |
| pop                | 17        |
| reverse (列表)       | 18        |
| slice              | 18        |
| take               | 18        |
| tap                | 18        |
| filter             | 18        |
| map                | 19        |
| sort               | 19        |
| sort!              | 19        |
| reverse!           | 19        |
| appended           | 20        |

|                                   |           |
|-----------------------------------|-----------|
| append! . . . . .                 | 20        |
| first . . . . .                   | 20        |
| last . . . . .                    | 20        |
| get (映射) . . . . .                | 20        |
| iter . . . . .                    | 20        |
| next . . . . .                    | 21        |
| to_list . . . . .                 | 21        |
| <b>集合参考 (元组、列表、映射、集合)</b>         | <b>21</b> |
| 元组 . . . . .                      | 21        |
| 列表 . . . . .                      | 22        |
| 映射 . . . . .                      | 27        |
| 集合 . . . . .                      | 29        |
| 迭代器 . . . . .                     | 31        |
| <b>契约式设计 (Design by Contract)</b> | <b>32</b> |
| 前置条件 (require) . . . . .          | 32        |
| 后置条件 (ensure) . . . . .           | 33        |
| 组合示例 . . . . .                    | 33        |
| 结构体不变量 (invariant) . . . . .      | 33        |
| 规则 . . . . .                      | 34        |
| <b>控制流参考</b>                      | <b>34</b> |
| if / elif / else . . . . .        | 34        |
| while . . . . .                   | 35        |
| for . . . . .                     | 35        |
| async / await . . . . .           | 37        |
| @parallel for . . . . .           | 37        |
| break . . . . .                   | 38        |
| continue . . . . .                | 38        |
| ... (Ellipsis) . . . . .          | 38        |
| match . . . . .                   | 38        |
| 作用域规则 . . . . .                   | 41        |
| <b>指令</b>                         | <b>41</b> |
| 语法 . . . . .                      | 41        |
| 支持的目标 . . . . .                   | 42        |
| 内建指令 . . . . .                    | 42        |
| 注意事项 . . . . .                    | 46        |
| <b>錯誤参考</b>                       | <b>46</b> |
| 錯誤格式 . . . . .                    | 46        |
| 編譯錯誤一覽 . . . . .                  | 46        |
| 執行時錯誤一覽 . . . . .                 | 48        |
| <b>函数参考</b>                       | <b>48</b> |
| 函数定义语法 . . . . .                  | 48        |
| 参数与返回值的类型 . . . . .               | 49        |
| 递归 . . . . .                      | 49        |
| 重载 . . . . .                      | 50        |
| 默认参数 . . . . .                    | 51        |
| Unit 类型函数 . . . . .               | 51        |
| Task 与异步函数 . . . . .              | 52        |
| Lambda 函数 . . . . .               | 52        |
| 闭包 . . . . .                      | 53        |

|                           |           |
|---------------------------|-----------|
| 函数类型                      | 53        |
| 高阶函数                      | 53        |
| 泛型函数                      | 54        |
| UFCS（统一函数调用语法）            | 55        |
| 运算符重载                     | 55        |
| 检查/饱和算术                   | 57        |
| <b>HTTP 参考手册</b>          | <b>57</b> |
| 类型                        | 57        |
| 函数（来自 <code>http</code> ） | 57        |
| 使用示例                      | 59        |
| 行为                        | 61        |
| 支持的状态码                    | 61        |
| HTTP 客户端                  | 62        |
| 错误处理                      | 64        |
| <b>I/O 函数参考手册</b>         | <b>64</b> |
| 函数列表                      | 64        |
| 使用示例                      | 65        |
| 错误处理                      | 66        |
| 备注                        | 66        |
| <b>JSON 函数参考</b>          | <b>66</b> |
| 概述                        | 66        |
| 函数列表                      | 66        |
| 使用示例                      | 67        |
| 注意事项                      | 68        |
| <b>數學函式 (math)</b>        | <b>69</b> |
| 概述                        | 69        |
| 常數                        | 69        |
| 絕對值                       | 69        |
| 捨入                        | 69        |
| 冪次與平方根                    | 70        |
| 對數                        | 70        |
| 指數                        | 70        |
| 三角函式                      | 70        |
| 反三角函式                     | 70        |
| 雙參數函式                     | 71        |
| 判定函式                      | 71        |
| <b>网络 (TCP) 参考手册</b>      | <b>71</b> |
| 类型                        | 71        |
| 函数（来自 <code>net</code> ）  | 71        |
| 内建重载函数                    | 72        |
| 使用示例                      | 72        |
| 超时配置                      | 74        |
| 错误处理                      | 74        |
| 字节转换                      | 74        |
| <b>运算符参考</b>              | <b>75</b> |
| 优先级表                      | 75        |
| 算术运算符                     | 75        |
| 比较运算符                     | 75        |
| 逻辑运算符                     | 76        |

|                       |           |
|-----------------------|-----------|
| 位运算符                  | 76        |
| 错误传播运算符 (? / !!)      | 77        |
| 三元条件运算符               | 77        |
| 范围运算符                 | 77        |
| 空值合并运算符 (??)          | 78        |
| 复合赋值运算符               | 78        |
| 递增 / 递减运算符            | 78        |
| 运算的类型规则               | 79        |
| 运算符重载                 | 79        |
| <b>包参考</b>            | <b>82</b> |
| 概述                    | 82        |
| 导入语法                  | 82        |
| 包解析                   | 83        |
| 标准库 (std)             | 83        |
| 搜索路径优先级               | 85        |
| RY_PATH 环境变量          | 85        |
| 约束                    | 85        |
| 创建包文件                 | 85        |
| <b>项目管理</b>           | <b>86</b> |
| CLI 概述                | 86        |
| ry init - 项目初始化       | 86        |
| ry new - 创建新项目        | 87        |
| ry fmt - 代码格式化工具      | 87        |
| ry test - 运行测试        | 88        |
| ry self-update - 自我更新 | 88        |
| package.toml 配置文件     | 89        |
| <b>正規表達式函式參考手冊</b>    | <b>89</b> |
| 函式一覽                  | 89        |
| 支援的模式語法               | 90        |
| 使用範例                  | 90        |
| <b>结构体参考</b>          | <b>92</b> |
| 概述                    | 92        |
| 定义语法                  | 92        |
| 构造函数                  | 92        |
| 字段访问                  | 92        |
| 字段赋值                  | 93        |
| 作为函数参数与返回值使用          | 93        |
| 嵌套结构体                 | 93        |
| 比较 (== / !=)          | 93        |
| 记录子类型 (继承)            | 94        |
| 约束与错误                 | 95        |
| 枚举类型 (enum)           | 95        |
| <b>测试功能</b>           | <b>97</b> |
| 运行测试                  | 97        |
| 语法                    | 98        |
| 输出格式                  | 99        |
| 示例                    | 99        |
| 模拟 (Mock)             | 100       |
| 参数化测试 (@each)         | 100       |
| 基于属性的测试 (@property)   | 101       |

|                       |            |
|-----------------------|------------|
| 测试覆盖率                 | 101        |
| 限制                    | 101        |
| <b>类型参考</b>           | <b>101</b> |
| 类型一览                  | 101        |
| 类型标注语法                | 103        |
| 可用类型名称一览              | 103        |
| 类型别名                  | 104        |
| 字面量类型                 | 104        |
| 范围类型                  | 104        |
| none 关键字与 Option 类型简写 | 105        |
| F-String (字符串插值)      | 105        |
| 类型转换 (as)             | 106        |
| 带关联数据的 enum (ADT)     | 106        |
| 泛型 enum               | 107        |
| Error 类型              | 108        |
| union 类型              | 108        |
| any 类型                | 109        |
| 类型规则 (运算时的类型转换)       | 111        |
| 类型安全约束                | 112        |

[English](#) | [日本語](#) | [繁體中文](#)

## Base64 函数参考

Base64 编码与解码。所有函数需要从 base64 明确导入。

```
from base64 import encode, decode, encode_url_safe, decode_url_safe
```

### 函数列表

| 函数              | 签名                          | 说明                            |
|-----------------|-----------------------------|-------------------------------|
| encode          | (str) -> str                | 将字符串编码为标准 base64              |
| decode          | (str) -> Result<str, Error> | 解码标准 base64 字符串               |
| encode_url_safe | (str) -> str                | 将字符串编码为 URL-safe base64 (无填充) |
| decode_url_safe | (str) -> Result<str, Error> | 解码 URL-safe base64 字符串        |

### 使用示例

#### 基本编码与解码

```
from base64 import encode, decode

encoded = encode("Hello, World!")
print(encoded)  # SGVsbG8sIFdvcmaXkIQ==

match decode(encoded):
    case Ok(s):
        print(s)  # Hello, World!
    case Err(e):
        print(e.message)
```

## URL-safe Base64

URL-safe base64 使用 - 和 \_ 取代 + 和 /，并省略填充 (=)。适用于 URL、文件名和令牌。

```
from base64 import encode_url_safe, decode_url_safe

encoded = encode_url_safe("data with special chars: ?&=")
# No + / or = in the output

match decode_url_safe(encoded):
    case Ok(s):
        print(s)
    case Err(e):
        print(e.message)
```

## 处理字节数据

要编码/解码字节数据，请结合 io 中的 str\_to\_bytes / bytes\_to\_str 使用。

```
from base64 import encode, decode
from io import str_to_bytes, bytes_to_str

bytes = str_to_bytes("binary data")
encoded = encode(bytes_to_str(bytes))
```

## 错误处理

decode 和 decode\_url\_safe 返回 Result<str, Error>。当输入包含无效的 base64 字符时，解码会失败。

```
match decode("!!!not-valid!!!"):
    case Ok(s):
        print(s)
    case Err(e):
        print(e.message) # "invalid base64 character at position 0"
```

使用 ? 运算符:

```
fn process(input: str) -> Result<str, Error>:
    decoded = decode(input)?
    return Ok(decoded)
```

[English](#) | [日本語](#) | [繁體中文](#)

## 字符串操作函数参考

针对字符串 (str) 的操作函数一览。所有函数均可使用 UFCS 记法。

注意：所有字符串操作均支持 UTF-8。length()、char\_at()、substring()、find() 和 reverse() 以 Unicode 码位为单位操作，而非字节。如需获取字节长度，请使用 byte\_len()。

## 函数列表

### 搜索与判定

| 函数          | 签名                 | 说明         |
|-------------|--------------------|------------|
| contains    | (str, str) -> bool | 返回是否包含子字符串 |
| starts_with | (str, str) -> bool | 返回是否以前缀开头  |

| 函数        | 签名                        | 说明                         |
|-----------|---------------------------|----------------------------|
| ends_with | (str, str) -> bool        | 返回是否以后缀结尾                  |
| find      | (str, str) -> Option<int> | 返回子字符串的字符位置<br>(未找到为 None) |

## 提取与转换

| 函数        | 签名                     | 说明               |
|-----------|------------------------|------------------|
| substring | (str, int, int) -> str | 提取子字符串 (字符索引)    |
| char_at   | (str, int) -> str      | 获取指定位置的 UTF-8 字符 |
| replace   | (str, str, str) -> str | 替换所有匹配的子字符串      |

## 大小写转换

| 函数       | 签名         | 说明           |
|----------|------------|--------------|
| to_upper | str -> str | 转换为 ASCII 大写 |
| to_lower | str -> str | 转换为 ASCII 小写 |

## 去除空白

| 函数         | 签名         | 说明      |
|------------|------------|---------|
| trim       | str -> str | 去除前后的空白 |
| trim_start | str -> str | 去除开头的空白 |
| trim_end   | str -> str | 去除结尾的空白 |

## 生成与加工

| 函数       | 签名                | 说明               |
|----------|-------------------|------------------|
| repeat   | (str, int) -> str | 将字符串重复 n 次       |
| reverse  | str -> str        | 反转字符串 (UTF-8 感知) |
| byte_len | str -> int        | 返回字符串的字节长度       |

## 分割与连接

| 函数    | 签名                      | 说明     |
|-------|-------------------------|--------|
| split | (str, str) -> List<str> | 以分隔符分割 |
| join  | (List<str>, str) -> str | 以分隔符连接 |

## 类型转换

| 函数       | 签名                               | 说明         |
|----------|----------------------------------|------------|
| to_int   | str -> int                       | 将字符串转换为整数  |
| to_float | str -> float                     | 将字符串转换为浮点数 |
| to_str   | int/float/bool/str/record -> str | 将值转换为字符串   |

## contains

签名: `contains(string: str, substring: str) -> bool`

返回字符串 `string` 中是否包含子字符串 `substring`。

```
print(contains("hello", "ell"))    # true
print("hello".contains("xyz"))     # false (UFCS)
```

---

## starts\_with

签名: `starts_with(string: str, prefix: str) -> bool`

返回字符串 `string` 是否以 `prefix` 开头。

```
print(starts_with("hello", "hel")) # true
print("hello".starts_with("world")) # false (UFCS)
```

---

## ends\_with

签名: `ends_with(string: str, suffix: str) -> bool`

返回字符串 `string` 是否以 `suffix` 结尾。

```
print(ends_with("hello", "llo"))  # true
print("hello".ends_with("world")) # false (UFCS)
```

---

## find

签名: `find(string: str, substring: str) -> Option<int>`

返回字符串 `string` 中子字符串 `substring` 首次出现的字符位置。未找到时返回 `None`。

```
print(find("hello world", "world")) # Some(6)
print(find("hello", "xyz"))         # None
print("abcdef".find("cd"))          # Some(2) (UFCS)
```

---

## substring

签名: `substring(string: str, start: int, end: int) -> str`

返回字符串 `string` 从 `start` 到 `end` (不含) 的子字符串。索引为字符位置 (UTF-8 感知)。

```
print(substring("hello world", 0, 5))  # hello
print(substring("hello world", 6, 11)) # world
print("abcdef".substring(1, 4))        # bcd (UFCS)
```

---

## char\_at

签名: `char_at(string: str, i: int) -> str`

返回字符串 `string` 第 `i` 个位置的 UTF-8 字符作为字符串。



```
print(char_at("hello", 0))    # h
print("abc".char_at(2))      # c (UFCS)
```

---

## replace

签名: `replace(string: str, old: str, new: str) -> str`

返回将字符串 `string` 中所有 `old` 替换为 `new` 后的新字符串。

```
print(replace("hello world", "world", "ry"))    # hello ry
print(replace("aaa", "a", "bb"))                # bbbbbb
print("foo bar foo".replace("foo", "baz"))      # baz bar baz (UFCS)
```

---

## to\_upper

签名: `to_upper(string: str) -> str`

返回将 ASCII 小写字母 (a-z) 转换为大写后的新字符串。

```
print(to_upper("hello"))          # HELLO
print("Hello World".to_upper())   # HELLO WORLD (UFCS)
```

---

## to\_lower

签名: `to_lower(string: str) -> str`

返回将 ASCII 大写字母 (A-Z) 转换为小写后的新字符串。

```
print(to_lower("HELLO"))          # hello
print("Hello World".to_lower())   # hello world (UFCS)
```

---

## trim

签名: `trim(string: str) -> str`

返回去除字符串前后空白字符 (空格、制表符、换行、回车) 后的新字符串。

```
print(trim(" hello "))           # hello
print(" hi ".trim())             # hi (UFCS)
```

---

## trim\_start

签名: `trim_start(string: str) -> str`

返回去除字符串开头空白字符后的新字符串。

```
print(trim_start(" hello "))     # hello
print(" hi".trim_start())        # hi (UFCS)
```

---

## trim\_end

签名: `trim_end(string: str) -> str`

返回去除字符串结尾空白字符后的新字符串。

```
print(trim_end(" hello "))    # hello
print("hi ".trim_end())       # hi (UFCS)
```

---

## repeat

签名: `repeat(string: str, count: int) -> str`

返回将字符串 `string` 重复 `count` 次后的新字符串。

```
print(repeat("ab", 3))        # ababab
print("ha".repeat(3))         # hahaha (UFCS)
```

---

## reverse

签名: `reverse(string: str) -> str`

返回字符顺序反转后的新字符串 (UTF-8 感知)。

```
print(reverse("hello"))       # olleh
print("abc".reverse())        # cba (UFCS)
```

---

## byte\_len

签名: `byte_len(string: str) -> int`

返回字符串 `string` 的字节长度。与返回 UTF-8 字符数的 `length()` 不同, `byte_len()` 返回的是字节数。

```
print(byte_len("hello"))      # 5
print(byte_len("あいう"))     # 9
print(length("あいう"))       # 3 (characters)
```

---

## split

签名: `split(string: str, delimiter: str) -> List<str>`

以分隔符 `delimiter` 分割字符串 `string`, 返回 `List<str>`。

```
parts = split("a,b,c", ",")
print(parts[0])    # a
print(parts[1])    # b
print(parts[2])    # c

for word in "hello world".split(" "):
    print(word)
# hello
# world
```

---

## join

签名: `join(values: List<str>, sep: str) -> str`

以分隔符 `sep` 连接字符串列表的元素，返回合并后的字符串。

```
parts = ["a", "b", "c"]
print(join(parts, ","))      # a,b,c
print(parts.join("-"))      # a-b-c (UFCS)
```

---

## to\_int

签名: `to_int(string: str) -> int`

将字符串转换为整数。

```
print(to_int("42"))         # 42
print(to_int("-7"))         # -7
print("123".to_int())       # 123 (UFCS)
```

---

## to\_float

签名: `to_float(string: str) -> float`

将字符串转换为浮点数。

```
print(to_float("3.14"))     # 3.14
print("2.5".to_float())     # 2.5 (UFCS)
```

---

## to\_str

签名: `to_str(v: int | float | bool | str | record) -> str`

将值转换为字符串。

| 类型     | 转换格式                                 |
|--------|--------------------------------------|
| int    | %ld                                  |
| float  | %g                                   |
| bool   | "true" / "false"                     |
| str    | 直接返回                                 |
| record | TypeName(field1: val1, field2: val2) |

Record 类型自动生成 `to_str` 表示。如果提供了用户定义的 `fn to_str(v: MyRecord) -> str`，则优先使用用户定义的版本。这也适用于 `print()` 和 f-string 插值。

```
print(to_str(42))           # 42
print(to_str(3.14))         # 3.14
print(to_str(true))         # true
print(99.to_str())          # 99 (UFCS)
```

```
record Point:
  x: int
  y: int
```

```

p = Point(10, 20)
print(to_str(p))      # Point(x: 10, y: 20)
print(p)              # Point(x: 10, y: 20)
print(f"pos={p}")     # pos=Point(x: 10, y: 20)

```

[English](#) | [日本語](#) | [简体中文](#)

## 内置函数参考

### 函数一览

#### 核心

| 函数                                                                  | 说明                                                                               |
|---------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <code>print(expr)</code>                                            | 将值输出到标准输出                                                                        |
| <code>length(value)</code>                                          | 返回列表、映射、集合的元素数量，或字符串的 UTF-8 字符数                                                  |
| <code>range(n) / range(start, end) / range(start, end, step)</code> | 生成整数列表                                                                           |
| <code>exit(code)</code>                                             | 以指定的退出码终止进程                                                                      |
| <code>args()</code>                                                 | 以 <code>List&lt;str&gt;</code> 返回命令行参数                                           |
| <code>available_parallelism()</code>                                | 返回运行时工作线程数 ( <code>int</code> )                                                  |
| <code>sleep(duration_ms)</code>                                     | 暂停执行指定的毫秒数                                                                       |
| <code>env(key)</code>                                               | 返回环境变量为 <code>Option&lt;str&gt;</code>                                           |
| <code>env(key, default)</code>                                      | 返回环境变量，若未设置则返回 <code>default</code>                                              |
| <code>send(stream, data)</code>                                     | 通过 <code>TcpStream</code> 发送 <code>List&lt;u8&gt;</code> ，返回发送的字节数               |
| <code>recv(stream, max)</code>                                      | 从 <code>TcpStream</code> 接收最多 <code>max</code> 字节，返回 <code>List&lt;u8&gt;</code> |
| <code>close(handle)</code>                                          | 关闭 <code>TcpStream</code> 或 <code>TcpListener</code>                             |
| <code>block_on(task)</code>                                         | 阻塞当前线程直到 <code>Task&lt;T&gt;</code> 完成并返回其结果                                     |

### Option

| 函数                      | 说明                             |
|-------------------------|--------------------------------|
| <code>Some(expr)</code> | 构造 <code>Option</code> 类型的有值变体 |

### 集合操作

| 函数                                                      | 说明                                              |
|---------------------------------------------------------|-------------------------------------------------|
| <code>has_key(map, key)</code>                          | 返回映射中是否存在该键                                     |
| <code>add(set, value)</code>                            | 向集合添加元素（重复则忽略）                                  |
| <code>remove(set, value)</code>                         | 从集合删除元素                                         |
| <code>append(list, value) / append!(list, value)</code> | 向列表末尾添加元素（就地修改）                                 |
| <code>appended(list, value)</code>                      | 返回添加元素后的新列表（非破坏性）                               |
| <code>pop(list)</code>                                  | 移除并返回列表的最后一个元素 ( <code>Option&lt;T&gt;</code> ) |
| <code>reverse(list)</code>                              | 返回反转后的新列表（也适用于字符串）                              |
| <code>reverse!(list)</code>                             | 就地反转列表（破坏性）                                     |
| <code>slice(list, start, end)</code>                    | 返回从 <code>start</code> 到 <code>end</code> 的新子列表 |
| <code>take(list, n)</code>                              | 返回包含前 <code>n</code> 个元素的新列表                    |
| <code>tap(list, fn)</code>                              | 对每个元素调用 <code>fn</code> 以执行副作用，返回原始列表           |

| 函数                                           | 说明                                     |
|----------------------------------------------|----------------------------------------|
| <code>filter(list, pred)</code>              | 返回仅包含满足谓词的元素的新列表                       |
| <code>map(list, fn)</code>                   | 返回将每个元素转换后的新列表                         |
| <code>sort(list) / sort(list, comp)</code>   | 返回排序后的新列表（默认升序）                        |
| <code>sort!(list) / sort!(list, comp)</code> | 就地排序列表（破坏性）                            |
| <code>insert(list, i, val)</code>            | 在索引 <code>i</code> 处插入元素               |
| <code>remove_at(list, i)</code>              | 移除并返回索引 <code>i</code> 处的元素            |
| <code>items(map)</code>                      | 返回 (键, 值) 元组的列表                        |
| <code>remove(map, key)</code>                | 删除指定键的条目                               |
| <code>get(map, key)</code>                   | 返回键的值 ( <code>Option&lt;V&gt;</code> ) |
| <code>get(map, key, default)</code>          | 返回键的值，若不存在则返回默认值                       |
| <code>union(set, set)</code>                 | 返回两个集合的并集                              |
| <code>intersection(set, set)</code>          | 返回两个集合的交集                              |
| <code>difference(set, set)</code>            | 返回两个集合的差集                              |
| <code>symmetric_difference(set, set)</code>  | 返回两个集合的对称差                             |
| <code>is_subset(set, set)</code>             | 返回第一个集合是否为第二个的子集                       |
| <code>is_superset(set, set)</code>           | 返回第一个集合是否为第二个的超集                       |

## 迭代器

| 函数                              | 说明                                                               |
|---------------------------------|------------------------------------------------------------------|
| <code>iter(collection)</code>   | 从 List、Set 或 Map 创建惰性迭代器                                         |
| <code>next(iter)</code>         | 返回下一个元素 ( <code>Option&lt;T&gt;</code> )，耗尽时返回 <code>None</code> |
| <code>to_list(iter)</code>      | 将迭代器剩余的所有元素收集到 <code>List&lt;T&gt;</code>                        |
| <code>filter(iter, pred)</code> | 返回只产出满足谓词的元素的惰性迭代器                                               |
| <code>map(iter, fn)</code>      | 返回转换每个元素的惰性迭代器                                                   |
| <code>take(iter, n)</code>      | 返回最多产出 <code>n</code> 个元素的惰性迭代器                                  |

## 字符串操作

| 函数                                                                | 说明                                           |
|-------------------------------------------------------------------|----------------------------------------------|
| <code>contains(string, substring)</code>                          | 是否包含子字符串                                     |
| <code>starts_with(string, prefix)</code>                          | 是否以前缀开头                                      |
| <code>ends_with(string, suffix)</code>                            | 是否以后缀结尾                                      |
| <code>find(string, substring)</code>                              | 子字符串的字符位置 ( <code>Option&lt;int&gt;</code> ) |
| <code>byte_len(string)</code>                                     | 返回字符串的字节长度                                   |
| <code>substring(string, start, end)</code>                        | 提取子字符串                                       |
| <code>char_at(string, i)</code>                                   | 获取指定位置的字符                                    |
| <code>replace(string, old, new)</code>                            | 替换所有出现的子字符串                                  |
| <code>to_upper(string) / to_lower(string)</code>                  | 大小写转换                                        |
| <code>trim(string) / trim_start(string) / trim_end(string)</code> | 去除空白                                         |
| <code>repeat(string, count)</code>                                | 将字符串重复 <code>n</code> 次                      |
| <code>reverse(string)</code>                                      | 反转字符串                                        |
| <code>split(string, delimiter)</code>                             | 分割字符串并返回列表                                   |
| <code>join(list, sep)</code>                                      | 以分隔符连接列表中的字符串                                |
| <code>to_int(s) / to_float(s) / to_str(v)</code>                  | 类型转换                                         |

-> 详细请参阅[字符串操作函数参考](#)

## print

签名：`print(expr)`

将值输出到标准输出。末尾会追加换行。

| 类型                         | 输出格式                                       |
|----------------------------|--------------------------------------------|
| <code>int</code>           | <code>%ld</code>                           |
| <code>float</code>         | <code>%g</code>                            |
| <code>bool</code>          | <code>true / false</code>                  |
| <code>str</code>           | <code>%s</code>                            |
| <code>Option (Some)</code> | <code>Some(value)</code>                   |
| <code>Option (None)</code> | <code>None</code>                          |
| <code>list</code>          | <code>[elem1, elem2, ...]</code>           |
| <code>map</code>           | <code>{key1: val1, key2: val2, ...}</code> |
| <code>set</code>           | <code>{elem1, elem2, ...}</code>           |
| <code>enum</code>          | 变体名称（例如： <code>Red</code> ）                |

```
print(42)           # 42
print(3.14)         # 3.14
print(true)         # true
print("hello")      # hello
print(Some(1))      # Some(1)
print(None)         # None
print([1, 2, 3])    # [1, 2, 3]
print({"a": 1})     # {a: 1}
print({1, 2, 3})    # {1, 2, 3}
```

错误条件：直接传入结构体或元组会产生编译错误。

---

## Some

签名：`Some(expr) -> Option<T>`

构造 `Option` 类型的有值变体。

```
x: Option<int> = Some(42)
print(x)       # Some(42)
```

---

## length

签名：`length(x: List<T> | Map<K, V> | Set<T> | str) -> int`

返回列表、映射、集合的元素数量，或字符串的 UTF-8 字符数。如需获取字节长度，请使用 `byte_len()`。

```
print(length([1, 2, 3]))      # 3
print(length({"a": 1, "b": 2})) # 2
print(length({1, 2, 3}))      # 3
print(length("hello"))        # 5
print(length("あいいう"))     # 3 (UTF-8 字符数)
```

---

## has\_key

签名: `has_key(m: Map<K, V>, key: K) -> bool`

返回映射中是否存在指定的键。也可使用 UFCS 记法。

```
m = {"a": 1, "b": 2}
print(has_key(m, "a"))    # true
print(m.has_key("z"))     # false (UFCS)
```

---

## add

签名: `add(s: Set<T>, value: T)`

向集合添加元素。若元素已存在则不做任何操作。也可使用 UFCS 记法。

```
s = {1, 2, 3}
s.add(4)          # UFCS
add(s, 5)         # 普通调用
s.add(1)          # 已存在，因此忽略
print(length(s))  # 5
```

---

## remove

签名: `remove(s: Set<T>, value: T)`

从集合删除元素。也可使用 UFCS 记法。

```
s = {1, 2, 3}
s.remove(2)       # UFCS
print(2 in s)     # false
```

---

## range

签名: `range(n: int) -> List<int>` / `range(start: int, end: int) -> List<int>` / `range(start: int, end: int, step: int) -> List<int>`

生成整数列表。

| 形式                                   | 生成的值                                                          |
|--------------------------------------|---------------------------------------------------------------|
| <code>range(n)</code>                | <code>[0, 1, ..., n-1]</code>                                 |
| <code>range(start, end)</code>       | <code>[start, start+1, ..., end-1]</code>                     |
| <code>range(start, end, step)</code> | <code>[start, start+step, start+2*step, ...]</code> (不包含 end) |

- `step > 0` 时，从 `start` 向 `end` 递增生成。
- `step < 0` 时，从 `start` 向 `end` 递减生成。
- `step == 0` 时，会产生运行时错误。
- 如果范围为空（例如 `range(0, 10, -1)`），返回空列表。

```
print(range(3))          # [0, 1, 2]
print(range(2, 5))       # [2, 3, 4]
print(range(0, 10, 2))   # [0, 2, 4, 6, 8]
print(range(10, 0, -3))  # [10, 7, 4, 1]
```

```
for i in range(3):
    print(i)
# 0
# 1
# 2
```

---

## exit

签名：`exit(code: int)`

以指定的退出码立即终止进程。`exit()` 之后的代码将不会被执行。

```
exit(0)      # 正常终止
exit(1)      # 错误终止
```

---

## args

签名：`args() -> List<str>`

以字符串列表的形式返回传递给脚本的命令行参数。不包含解释器名称或脚本文件名——仅包含脚本路径之后的参数。

```
# 运行：ry script.ry hello world
a = args()
print(length(a))    # 2
print(a[0])         # hello
print(a[1])         # world

for x in args():
    print(x)
```

---

## sleep

签名：`sleep(duration_ms: int) -> Unit`

暂停当前线程的执行指定的毫秒数。若 `duration_ms` 为 0 或负数，则函数立即返回。

```
sleep(1000)    # 等待 1 秒
sleep(0)       # 立即返回
```

---

## env

签名：`env(key: str) -> Option<str>` / `env(key: str, default: str) -> str`

返回环境变量的值。单参数形式返回 `Option<str>`（若已设置则为 `Some(value)`，未设置则为 `None`）。双参数形式在变量未设置时返回 `default`。

若项目根目录（包含 `package.toml` 的目录）中存在 `.env` 文件，启动时会自动载入到进程环境中。现有的环境变量不会被 `.env` 的值覆盖。

安全提示：`.env` 文件通常包含密钥（API 密钥、数据库密码、令牌等）。请不要将 `.env` 提交到版本控制系统（添加到 `.gitignore` 或类似文件中），并将其内容视为敏感配置。



```

# 单参数形式：返回 Option<str>
path = env("PATH")
match path:
    case Some(v):
        print(v)
    case None:
        print("PATH not set")

# 双参数形式：带默认值返回 str
port = env("PORT", "8080")
print(port)    # 若 PORT 未设置则为 "8080"

```

### .env 文件格式

```

# 注释以 # 开头
DATABASE_URL=postgres://localhost/mydb
API_KEY="secret-key-123"
EMPTY_VALUE=
QUOTED='single quoted'

```

### 环境专属 .env 文件

设置 `RY_ENV` 时，Ry 会按照以下优先顺序载入环境专属的 .env 文件：

- 先载入 `.env.<env>`（例如 `RY_ENV=dev` 时载入 `.env.dev`）
- 再载入 `.env`（已由 `.env.<env>` 设置的值不会被覆盖）
- `RY_ENV=prod` 时不载入任何 `.env` 文件（安全考虑）
- `RY_ENV` 未设置时仅载入 `.env`（向后兼容）

环境模式的详细信息请参阅 [RY\\_ENV](#)。

## append

签名：`append(list: List<T>, value: T)`

向列表末尾添加元素。此为就地修改操作——列表会被直接修改。也可使用 UFCS 记法。

```

xs = [1, 2]
xs.append(3)
print(xs)    # [1, 2, 3]

```

## pop

签名：`pop(list: List<T>) -> Option<T>`

移除并返回列表的最后一个元素（`Option<T>`）。列表为空时返回 `None`。也可使用 UFCS 记法。

```

xs = [1, 2, 3]
v = xs.pop()
print(v)    # Some(3)
print(xs)   # [1, 2]

```

## reverse (列表)

签名: `reverse(list: List<T>) -> List<T>`

返回元素顺序反转的新列表。原始列表不会被修改。也适用于字符串（请参阅[字符串操作](#)）。也可使用 UFCS 记法。

```
xs = [1, 2, 3]
ys = reverse(xs)
print(ys)    # [3, 2, 1]
print(xs)    # [1, 2, 3] (未修改)
```

---

## slice

签名: `slice(list: List<T>, start: int, end: int) -> List<T>`

返回从 `start`（含）到 `end`（不含）的新子列表。索引会被钳制在有效范围内（0 到 `length(list)`）。也可使用 UFCS 记法。

```
xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))    # [2, 3]
print(slice(xs, 0, 100))  # [1, 2, 3, 4, 5] (钳制)
```

---

## take

签名: `take(list: List<T>, n: int) -> List<T>`

返回包含前 `n` 个元素的新列表。若 `n` 超过列表长度，返回整个列表的副本。若 `n <= 0`，返回空列表。原始列表不会被修改。也可使用 UFCS 记法。

```
xs = [1, 2, 3, 4, 5]
ys = xs.take(3)
print(ys)    # [1, 2, 3]
print(xs.take(10))  # [1, 2, 3, 4, 5] (钳制)
print(xs.take(0))   # []
```

---

## tap

签名: `tap(list: List<T>, fn: fn(T) -> R) -> List<T>`

对每个元素调用给定函数（忽略返回值），然后返回原始列表。适用于方法链中的调试或插入副作用。也可使用 UFCS 记法。

```
xs = [1, 2, 3]
ys = xs.tap(fn(x: int) => print(x)).map(fn(x: int) => x * 2)
# 输出 1, 2, 3, 然后 ys = [2, 4, 6]
```

---

## filter

签名: `filter(list: List<T>, pred: fn(T) -> bool) -> List<T>`

返回仅包含谓词返回 `true` 的元素的新列表。原始列表不会被修改。也可使用 UFCS 记法。

```
xs = [1, 2, 3, 4, 5]
ys = xs.filter(fn(x: int) => x > 3)
print(ys)    # [4, 5]
print(xs)    # [1, 2, 3, 4, 5] (未修改)
```

---

## map

签名: `map(list: List<T>, fn: fn(T) -> U) -> List<U>`

返回将每个元素以给定函数转换后的新列表。输出元素类型可以与输入不同。原始列表不会被修改。也可使用 UFCS 记法。

```
xs = [1, 2, 3]
ys = xs.map(fn(x: int) => x * 2)
print(ys)    # [2, 4, 6]
```

---

## sort

签名: `sort(list: List<T>) -> List<T>` / `sort(list: List<T>, comp: fn(T, T) -> bool) -> List<T>`

返回排序后的新列表。默认为升序。可提供自定义比较函数（第一参数应排在第二参数之前时返回 `true`）。原始列表不会被修改。排序是稳定的（相等元素保持原始顺序）。也可使用 UFCS 记法。

```
xs = [3, 1, 2]
print(xs.sort())    # [1, 2, 3]

# 降序排序
desc = xs.sort(fn(a: int, b: int) => a > b)
print(desc)    # [3, 2, 1]
```

---

## sort!

签名: `sort!(list: List<T>) / sort!(list: List<T>, comp: fn(T, T) -> bool)`

就地排序列表。排序算法与 `sort()` 相同，但修改原始列表而非创建新列表。也可使用 UFCS 记法。

```
xs = [3, 1, 2]
xs.sort!()
print(xs)    # [1, 2, 3]
```

---

## reverse!

签名: `reverse!(list: List<T>)`

就地反转列表。也可使用 UFCS 记法。

```
xs = [1, 2, 3]
xs.reverse!()
print(xs)    # [3, 2, 1]
```

---

## appended

签名: `appended(list: List<T>, value: T) -> List<T>`

返回添加元素后的新列表。原始列表不会被修改。也可使用 UFCS 记法。

```
xs = [1, 2]
ys = xs.appended(3)
print(xs)    # [1, 2] (未修改)
print(ys)    # [1, 2, 3]
```

---

## append!

签名: `append!(list: List<T>, value: T)`

`append()` 的别名。就地向列表末尾添加元素。为配合 ! 命名约定而提供。

---

## first

签名: `first(list: List<T>) -> Option<T>`

返回列表的第一个元素 (`Option<T>`)。列表为空时返回 `None`。

```
print(first([10, 20, 30])) # Some(10)
```

---

## last

签名: `last(list: List<T>) -> Option<T>`

返回列表的最后一个元素 (`Option<T>`)。列表为空时返回 `None`。

```
print(last([10, 20, 30])) # Some(30)
```

---

## get (映射)

签名: `get(map: Map<K, V>, key: K) -> Option<V>` / `get(map: Map<K, V>, key: K, default: V) -> V`

双参数形式返回键的值 (`Option<V>`)。三参数形式返回键的值，若不存在则返回默认值。

```
m = {"a": 1, "b": 2}
print(get(m, "a"))      # Some(1)
print(get(m, "z"))      # None
print(get(m, "z", 0))   # 0
```

---

## iter

签名: `iter(collection: List<T> | Set<T>) -> Iterator<T>` / `iter(collection: Map<K, V>) -> Iterator<(K, V)>`

从集合创建惰性迭代器。迭代器不复制数据；它引用原始集合。也可使用 UFCS 记法。

- 对于 `List<T>` 和 `Set<T>`，元素类型为 `T`。

- 对于 `Map<K, V>`，元素类型为元组 `(K, V)`。

```
xs = [1, 2, 3]
it = xs.iter()           # Iterator<int>
ys = it.to_list()        # [1, 2, 3]

m = {"a": 1, "b": 2}
for k, v in m.iter():     # Iterator<(str, int)>
    print(k)
```

---

## next

签名：`next(iter: Iterator<T>) -> Option<T>`

返回迭代器的下一个元素 (`Option<T>`)。当迭代器耗尽时返回 `None`。每次调用时迭代器会推进其内部状态。也可使用 UFCS 记法。

```
it = [10, 20].iter()
print(it.next())    # Some(10)
print(it.next())    # Some(20)
print(it.next())    # None
```

---

## to\_list

签名：`to_list(iter: Iterator<T>) -> List<T>`

将迭代器剩余的所有元素收集到新列表中。也可使用 UFCS 记法。

```
xs = [1, 2, 3, 4, 5]
ys = xs.iter().filter(fn(x: int) => x > 2).to_list()
print(ys)    # [3, 4, 5]
```

[English](#) | [日本語](#) | [繁體中文](#)

# 集合参考（元组、列表、映射、集合）

## 元组

### 概述

固定长度、异质类型的值组合。以 LLVM literal `StructType` 实现，是栈上的值类型。

### 语法

```
t = (1, 3.14)
t: (int, float) = (1, 3.14)
```

### 类型注解

```
pair: (int, str) = (42, "hello")
triple: (int, float, bool) = (1, 2.0, true)
```

## 元素访问

使用 `.0`、`.1`、`...` 的数值索引访问。

```
t = (10, 3.14)
print(t.0)    # 10
print(t.1)    # 3.14
```

## 函数返回值

```
fn swap(a: int, b: int) -> (int, int):
    return (b, a)
```

```
result = swap(1, 2)
print(result.0)    # 2
print(result.1)    # 1
```

## 约束与错误

| 约束                         | 详细                             |
|----------------------------|--------------------------------|
| 超出范围的索引                    | 编译错误                           |
| 直接将元组传给 <code>print</code> | 编译错误 ( <code>print</code> 不支持) |

## 列表

### 概述

相同类型的可变长度序列。分配在堆上。

### 语法

```
xs = [1, 2, 3]
xs: List<int> = [1, 2, 3]
```

### 支持的元素类型

`int`, `float`, `bool`, `str`

### 索引访问

```
xs = [1, 2, 3]
print(xs[0])    # 1
print(xs[2])    # 3
```

### 索引赋值

```
xs = [1, 2, 3]
xs[0] = 99
print(xs[0])    # 99
```

### length

```
xs = [1, 2, 3]
print(length(xs))    # 3
```

## print

```
xs = [1, 2, 3]
print(xs)    # [1, 2, 3]
```

## for 遍历

```
xs = [10, 20, 30]
for x in xs:
    print(x)
# 10
# 20
# 30
```

## append

向列表末尾添加元素。此为就地修改操作。

```
xs = [1, 2]
xs.append(3)
print(xs)    # [1, 2, 3]
```

## pop

移除并返回列表的最后一个元素。对空列表调用会产生运行时错误。

```
xs = [1, 2, 3]
v = xs.pop()
print(v)      # 3
print(xs)     # [1, 2]
```

## reverse

返回元素顺序反转的新列表。原始列表不会被修改。也适用于字符串。

```
xs = [1, 2, 3]
print(reverse(xs))    # [3, 2, 1]
print(xs)             # [1, 2, 3] (unchanged)
```

## slice

返回从 start（含）到 end（不含）的新子列表。索引会被钳制在有效范围内。

```
xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))    # [2, 3]
print(slice(xs, 0, 100))  # [1, 2, 3, 4, 5] (clamped)
```

## take

返回包含前 n 个元素的新列表。若 n 超过列表长度，返回整个列表的副本。若 n <= 0，返回空列表。原始列表不会被修改。

```
xs = [1, 2, 3, 4, 5]
ys = xs.take(3)
print(ys)    # [1, 2, 3]
print(xs.take(10))    # [1, 2, 3, 4, 5] (clamped)
print(xs.take(0))     # []
```

## tap

对每个元素调用给定函数（忽略返回值），然后返回原始列表。适用于方法链中的调试或插入副作用。

```
xs = [1, 2, 3]
ys = xs.tap(fn(x: int) => print(x)).map(fn(x: int) => x * 2)
# prints 1, 2, 3, then ys = [2, 4, 6]
```

## filter

返回仅包含满足谓词的元素的新列表。原始列表不会被修改。

```
xs = [1, 2, 3, 4, 5]
ys = xs.filter(fn(x: int) => x > 3)
print(ys)    # [4, 5]
```

## map

返回将每个元素以给定函数转换后的新列表。输出元素类型可以与输入不同。原始列表不会被修改。

```
xs = [1, 2, 3]
ys = xs.map(fn(x: int) => x * 2)
print(ys)    # [2, 4, 6]
```

## sort

返回排序后的新列表。默认为升序。可提供自定义比较函数。原始列表不会被修改。排序是稳定的（相等元素保持原始顺序）。内部使用 TimSort。

```
xs = [3, 1, 2]
print(xs.sort())    # [1, 2, 3]

# Descending order with comparator
desc = xs.sort(fn(a: int, b: int) => a > b)
print(desc)    # [3, 2, 1]
```

## filter、map、sort 的链式调用

这些函数返回新列表，因此可通过 UFCS 进行链式调用。

```
xs = [5, 3, 1, 4, 2]
result = xs.filter(fn(x: int) => x > 1).map(fn(x: int) => x * 10).sort()
print(result)    # [20, 30, 40, 50]
```

## reduce

使用累加函数将列表归约为单个值，以第一个元素作为初始值。

```
xs = [1, 2, 3, 4, 5]
total = reduce(xs, fn(a: int, b: int) => a + b)
print(total)    # 15
```

## fold

使用明确的初始值和累加函数将列表折叠为单个值。

```
xs = [1, 2, 3, 4, 5]
total = fold(xs, 0, fn(a: int, b: int) => a + b)
print(total)    # 15
```



## any

如果至少有一个元素满足谓词，则返回 `true`。

```
xs = [1, 2, 3, 4, 5]
print(any(xs, fn(x: int) => x > 4))    # true
print(any(xs, fn(x: int) => x > 9))    # false
```

## all

如果所有元素都满足谓词，则返回 `true`。

```
xs = [2, 4, 6]
print(all(xs, fn(x: int) => x > 0))    # true
print(all(xs, fn(x: int) => x > 3))    # false
```

## sum

返回所有元素的总和。

```
xs = [1, 2, 3, 4, 5]
print(sum(xs))    # 15
```

## min

返回最小的元素。

```
xs = [3, 1, 4, 1, 5]
print(min(xs))    # 1
```

## max

返回最大的元素。

```
xs = [3, 1, 4, 1, 5]
print(max(xs))    # 5
```

## first

返回第一个元素。对空列表调用会产生运行时错误。

```
xs = [10, 20, 30]
print(first(xs))    # 10
```

## last

返回最后一个元素。对空列表调用会产生运行时错误。

```
xs = [10, 20, 30]
print(last(xs))    # 30
```

## is\_empty

如果列表没有元素则返回 `true`。

```
xs = [1, 2, 3]
print(is_empty(xs))    # false
print(is_empty([]))    # true (requires type annotation in practice)
```

## enumerate

返回 (index, element) 元组的列表。

```
xs = [10, 20, 30]
pairs = enumerate(xs)
# pairs = [(0, 10), (1, 20), (2, 30)]

# for 循环中的元组解构
for i, x in enumerate(xs):
    print(f"{i}: {x}")    # 0: 10, 1: 20, 2: 30
```

## zip

将两个列表合并为 (elem1, elem2) 元组的列表。结果长度等于较短的列表。

```
xs = [1, 2, 3]
ys = ["a", "b", "c"]
pairs = zip(xs, ys)
# pairs = [(1, "a"), (2, "b"), (3, "c")]

# for 循环中的元组解构
for a, b in zip(xs, ys):
    print(f"{a}: {b}")    # 1: a, 2: b, 3: c
```

## insert

在指定索引处插入元素。该索引及之后的元素向右移动。

```
xs = [1, 2, 3]
insert(xs, 1, 99)
print(xs)    # [1, 99, 2, 3]
```

## remove\_at

移除并返回指定索引处的元素。该索引之后的元素向左移动。

```
xs = [1, 2, 3, 4]
v = remove_at(xs, 1)
print(v)    # 2
print(xs)    # [1, 3, 4]
```

## remove

从列表中移除第一个匹配的指定值。若找不到该值，则不做任何操作。这是一个可变操作。

```
xs = [1, 2, 3, 2, 4]
remove(xs, 2)
print(xs)    # [1, 3, 2, 4]
```

## distinct

返回一个移除重复元素的新列表。保持原始顺序（保留第一次出现）。原始列表不会被修改。

```
xs = [1, 2, 3, 2, 1, 4]
print(distinct(xs))    # [1, 2, 3, 4]
print(xs)              # [1, 2, 3, 2, 1, 4] (unchanged)
```

flatten

将嵌套列表（列表的列表）展开一层。返回新列表。原始列表不会被修改。

```
xs = [[1, 2], [3, 4]]
print(flatten(xs))    # [1, 2, 3, 4]
print(xs)             # [[1, 2], [3, 4]] (unchanged)
```

操作复杂度

| 操作                        | 复杂度        |
|---------------------------|------------|
| xs[i] 索引访问                | O(1)       |
| append / append!          | 均摊 O(1)    |
| pop                       | O(1)       |
| first, last               | O(1)       |
| insert, remove_at         | O(n)       |
| sort / sort!              | O(n log n) |
| take                      | O(n)       |
| tap                       | O(n)       |
| filter, map, reduce, fold | O(n)       |
| reverse / reverse!        | O(n)       |
| distinct                  | O(n)       |
| length                    | O(1)       |

约束与错误

| 约束          | 详细               |
|-------------|------------------|
| 所有元素必须为相同类型 | 混合不同类型会产生编译错误    |
| 空列表 []      | 无法进行类型推断，会产生编译错误 |
| 超出范围的访问     | 运行时错误 (exit(1))  |

映射

概述

键与值的对应表。分配在堆上。

语法

```
m = {"a": 1, "b": 2}
m: Map<str, int> = {"a": 1, "b": 2}
```

键访问

```
m = {"a": 1, "b": 2}
print(m["a"])    # 1
```

插入与更新

```
m = {"a": 1}
m["b"] = 2      # Insert new entry
m["a"] = 99     # Update existing entry
```

## length

```
m = {"a": 1, "b": 2, "c": 3}
print(length(m))    # 3
```

## print

```
m = {"a": 1, "b": 2}
print(m)            # {a: 1, b: 2}
```

## has\_key

```
m = {"a": 1, "b": 2}
print(m.has_key("a"))    # true
print(m.has_key("z"))    # false
```

## keys

返回映射中所有键的列表。

```
m = {"a": 1, "b": 2, "c": 3}
print(keys(m))          # ["a", "b", "c"]
```

## values

返回映射中所有值的列表。

```
m = {"a": 1, "b": 2, "c": 3}
print(values(m))        # [1, 2, 3]
```

## items

返回映射中所有条目的 (key, value) 元组列表。

```
m = {"a": 1, "b": 2}
pairs = items(m)
# pairs = [("a", 1), ("b", 2)]
```

## remove (Map)

从映射中删除指定键的条目。若键不存在则不做任何操作。

```
m = {"a": 1, "b": 2}
remove(m, "a")
print(m)            # {b: 2}
```

## get

返回指定键的值，若键不存在则返回默认值。

```
m = {"a": 1, "b": 2}
print(get(m, "a", 0))    # 1
print(get(m, "z", 0))    # 0
```

## merge

返回一个合并两个映射的新映射。当键重复时，第二个映射的值优先。原始映射不会被修改。

```

m1 = {"a": 1, "b": 2}
m2 = {"b": 99, "c": 3}
m3 = merge(m1, m2)
print(m3["a"])    # 1
print(m3["b"])    # 99
print(m3["c"])    # 3

```

## 约束与错误

| 约束         | 详细                                                        |
|------------|-----------------------------------------------------------|
| 所有键必须为相同类型 | 混合不同类型的键会产生编译错误                                           |
| 所有值必须为相同类型 | 混合不同类型的值会产生编译错误                                           |
| 空映射        | 需要类型注解（如 <code>m: Map&lt;str, int&gt; = {"a": 1}</code> ） |
| 访问不存在的键    | 运行时错误（ <code>exit(1)</code> ）                             |
| 键查找        | 哈希表（平均 $O(1)$ ）                                           |
| 容量溢出       | 自动扩展为 2 倍                                                 |

## 集合

### 概述

持有相同类型的元素且不重复的集合。分配在堆上。

### 语法

```

s = {1, 2, 3}
s: Set<int> = {1, 2, 3}

```

### 支持的元素类型

`int`, `float`, `bool`, `str`

### `in` 运算符（成员检查）

```

s = {1, 2, 3}
print(2 in s)    # true
print(5 in s)    # false

```

### `length`

```

s = {1, 2, 3}
print(length(s)) # 3

```

### `print`

```

s = {1, 2, 3}
print(s)    # {1, 2, 3}

```

### `add`（添加元素）

添加重复的元素时会被忽略。

```
s = {1, 2, 3}
s.add(4)           # Add
s.add(1)           # Ignored because it already exists
print(length(s))   # 4
```

remove (删除元素)

```
s = {1, 2, 3}
s.remove(2)
print(2 in s)      # false
```

for 遍历

```
s = {10, 20, 30}
for x in s:
    print(x)
```

空集合

空集合需要类型注解。

```
s: Set<int> = {}
```

函数参数

```
fn has_value(s: Set<int>, v: int) -> bool:
    return v in s
```

union

返回包含两个集合所有元素的新集合。

```
a = {1, 2, 3}
b = {3, 4, 5}
print(union(a, b))    # {1, 2, 3, 4, 5}
```

intersection

返回仅包含两个集合中都存在的元素的新集合。

```
a = {1, 2, 3}
b = {2, 3, 4}
print(intersection(a, b)) # {2, 3}
```

difference

返回包含在第一个集合中但不在第二个集合中的元素的新集合。

```
a = {1, 2, 3}
b = {2, 3, 4}
print(difference(a, b))  # {1}
```

symmetric\_difference

返回包含在任一集合中但不同时在两个集合中的元素的新集合。

```
a = {1, 2, 3}
b = {2, 3, 4}
print(symmetric_difference(a, b)) # {1, 4}
```

## is\_subset

如果第一个集合的所有元素都包含在第二个集合中，则返回 `true`。

```
a = {1, 2}
b = {1, 2, 3}
print(is_subset(a, b))    # true
print(is_subset(b, a))    # false
```

## is\_superset

如果第一个集合包含第二个集合的所有元素，则返回 `true`。

```
a = {1, 2, 3}
b = {1, 2}
print(is_superset(a, b))  # true
print(is_superset(b, a))  # false
```

## 约束与错误

| 约束                  | 详细              |
|---------------------|-----------------|
| 所有元素必须为相同类型         | 混合不同类型会产生编译错误   |
| 空集合 <code>{}</code> | 需要类型注解          |
| 元素查找                | 哈希表（平均 $O(1)$ ） |
| 容量溢出                | 自动扩展为 2 倍       |

## 迭代器

### 概述

惰性迭代器抽象，支持高效的数据转换管道。迭代器不会复制或具体化中间结果——每个元素按需处理。

### 创建迭代器

使用 `iter()` 从任何集合创建迭代器：

```
xs = [1, 2, 3, 4, 5]
it = xs.iter()           # Iterator<int>

s = {10, 20, 30}
sit = s.iter()           # Iterator<int>

m = {"a": 1, "b": 2}
mit = m.iter()           # Iterator<(str, int)>
```

### 惰性方法链

迭代器方法返回新的迭代器，形成一个只在消费时才执行的管道：

| 方法                       | 说明                            |
|--------------------------|-------------------------------|
| <code>.filter(fn)</code> | 仅产出谓词返回 <code>true</code> 的元素 |
| <code>.map(fn)</code>    | 使用给定函数转换每个元素                  |
| <code>.take(n)</code>    | 最多产出 <code>n</code> 个元素       |

```
result = [1, 2, 3, 4, 5]
  .iter()
  .filter(fn(x: int) => x > 2)
  .map(fn(x: int) => x * 2)
  .take(2)
  .to_list()    # [6, 8]
```

消费迭代器

| 方法                      | 说明                                       |
|-------------------------|------------------------------------------|
| <code>.to_list()</code> | 将所有元素收集到 <code>List&lt;T&gt;</code>      |
| <code>.next()</code>    | 返回下一个元素，类型为 <code>Option&lt;T&gt;</code> |

```
it = [10, 20].iter()
print(it.next())    # Some(10)
print(it.next())    # Some(20)
print(it.next())    # None
```

For 循环支持

迭代器可以直接在 for 循环中使用:

```
for x in [1, 2, 3].iter().filter(fn(x: int) => x > 1):
  print(x)
# 2
# 3
```

```
for k, v in {"a": 1, "b": 2}.iter():
  print(k)
```

[English](#) | [日本語](#) | [繁體中文](#)

## 契约式设计 (Design by Contract)

Ry 支持 Eiffel 风格的契约式设计，包含前置条件 (require)、后置条件 (ensure) 和结构体不变量 (invariant)。契约违反时，程序将以 `exit(1)` 终止。

### 前置条件 (require)

前置条件在函数入口处检查。它们指定函数被正确调用所需满足的条件。

```
fn deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  new_balance: int = balance + amount
  return new_balance
```

若前置条件未满足，程序将输出以下消息并终止：

```
Contract violation: require failed in deposit()
```



## 后置条件 (ensure)

后置条件在每个 `return` 之前检查。它们指定函数对其返回值的保证。

### 变量绑定

`ensure` 需要一个变量名来绑定返回值。此变量可在后置条件表达式中使用。

```
fn abs(x: int) -> int:
  ensure v:
    v >= 0
  if x < 0:
    return -x
  return x
```

由于 Ry 的函数参数是不可变的，可以在 `ensure` 块中直接引用参数来与入口值比较：

```
fn increment(x: int) -> int:
  ensure v:
    v == x + 1
  return x + 1
```

### 元组解构

对于返回元组的函数，可以用逗号分隔指定多个变量名：

```
fn divide(a: int, b: int) -> (int, int):
  ensure q, r:
    q >= 0
    r >= 0
  return (a // b, a % b)
```

绑定变量的数量必须与元组元素数量一致。

---

## 组合示例

```
fn deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  ensure v:
    v >= 0
    v == balance + amount
  new_balance: int = balance + amount
  return new_balance
```

---

## 结构体不变量 (invariant)

不变量是结构体实例必须始终满足的条件。在以下时机检查：- 构造时 - 每次字段赋值后

```
record BankAccount:
  balance: int
  min_balance: int
  invariant:
    balance >= min_balance
```

```
a = BankAccount(100, 0)    # OK: 100 >= 0
a.balance = -1             # Contract violation: invariant failed
```

---

## 规则

- `require` 和 `ensure` 块为可选，写在函数体之前。
- 同时使用时，`require` 必须在 `ensure` 之前。
- `ensure` 需要变量绑定来命名返回值（例：`ensure v:`）。
- 对于元组返回值，可指定多个绑定变量（例：`ensure q, r:`）。
- `invariant` 写在 `record` 定义的末尾，所有字段声明之后。
- 所有契约违反以 `exit(1)` 终止程序并输出诊断消息。

[English](#) | [日本語](#) | [简体中文](#)

## 控制流参考

### if / elif / else

#### 语法

```
if condition:
    # then 代码块
elif condition:
    # elif 代码块（可多个）
else:
    # else 代码块（可省略）
```

#### 条件式的类型

| 类型   | 为 false 的值 | 为 true 的值 |
|------|------------|-----------|
| bool | false      | true      |
| int  | 0          | 非 0       |

float 和 str 无法直接用于条件式。

#### 示例

```
x = 10

if x > 5:
    print("big")
elif x == 5:
    print("five")
else:
    print("small")
```

#### 作用域规则

- `if` / `elif` / `else` 的各个代码块分别拥有独立的块作用域。
- 在代码块内声明的变量无法从代码块外访问。

```
if true:
    y = 42
# y 在此处无法访问
```

---

## while

### 语法

```
while condition:
    # 循环体
```

当条件式为 `true` 时，重复执行循环体。

### 示例

```
i = 0
while i < 5:
    print(i)
    i += 1
```

### 搭配 `break` / `continue`

```
i = 0
while true:
    if i >= 3:
        break
    i += 1
```

---

## for

### 语法

```
# 列表 / 集合遍历
for x in iterable_expr:
    # 各元素依次赋值给 x

# range (从 0 开始)
for i in range(n):
    # i = 0, 1, ..., n-1

# range (指定起始与结束)
for i in range(start, end):
    # i = start, start+1, ..., end-1

# range (指定步长)
for i in range(start, end, step):
    # i = start, start+step, start+2*step, ...
```

### 映射键值遍历

```
for k, v in map_expr:
    # k 为键, v 为各条目的值
```

### 元组解构

遍历元组列表时，可以将元组解构为 `N` 个变量（需与元组元素数量匹配）。使用 `_` 丢弃值。

```

xs = [10, 20, 30]

for i, x in enumerate(xs):
    print(f"{i}: {x}")    # 0: 10, 1: 20, 2: 30

for a, b in zip([1, 2], [10, 20]):
    print(a + b)          # 11, 22

for _, x in enumerate(xs):
    print(x)              # 丢弃索引

# N 元素解构 (3 个以上的变量)
triples = [(1, 2, 3), (4, 5, 6)]
for a, b, c in triples:
    print(a + b + c)      # 6, 15

for a, _, c in triples:
    print(a + c)          # 4, 10 (丢弃中间元素)

```

范围运算符 (..)

.. 运算符创建包含两端的整数范围。1 .. 5 产生 [1, 2, 3, 4, 5]。

```

for i in 1 .. 5:
    print(i)              # 1 2 3 4 5

```

示例

```

xs = [10, 20, 30]
for x in xs:
    print(x)

s = {1, 2, 3}
for x in s:
    print(x)

for i in range(5):
    print(i)              # 0 1 2 3 4

for i in range(2, 6):
    print(i)              # 2 3 4 5

for i in range(0, 10, 2):
    print(i)              # 0 2 4 6 8

for i in range(10, 0, -3):
    print(i)              # 10 7 4 1

# 映射遍历
m = {"a": 1, "b": 2}
for k, v in m:
    print(k)
    print(v)

```

# 范围运算符

```
for i in 1 .. 3:
    print(i)      # 1 2 3
```

---

## async / await

`async fn` 声明一个并发运行的函数。调用 `async fn` 返回 `Task<T>`。在另一个 `async fn` 内部使用 `await`，或从同步上下文中使用 `block_on()` 等待结果。

```
async fn add(a: int, b: int) -> int:
    return a + b
```

```
# 从同步上下文中，使用 block_on()
t: Task<int> = add(20, 22)
print(block_on(t))           # 42
print(block_on(add(1, 2)))   # 3
```

```
# 在 async fn 内部，使用 await
async fn double_add(a: int, b: int) -> int:
    result = await add(a, b)
    return result * 2
```

### 规则

- `async fn name(...) -> T`: 使用等待结果类型 `T` 声明。
  - 调用 `async fn` 会立即返回 `Task<T>`。
  - `await expr` 要求 `expr` 为 `Task<T>` 并产生 `T`。
  - `await` 只能在 `async fn` 内部使用。从同步上下文中使用 `block_on(task)`。
  - `block_on(task)` 阻塞当前线程直到任务完成并返回结果。
  - 支持 `async fn ... -> Unit`；当不产生值时，`block_on(task)` 是等待的主要方式。
  - 任务在运行时工作线程池上运行；不是每个任务一个操作系统线程。
  - `v1` 不支持 `async lambda` 和 `async @native fn`。
- 

## @parallel for

`@parallel` 只能附加到使用 `range(...)` 或整数 `..` 范围的计数 `for` 循环上。循环体在运行时工作线程池上以并行块运行。

```
@parallel
for i in range(8):
    print(i)
```

### 约束

- 仅支持 `range(...)` 和整数 `..` 循环。
- 不支持解构迭代。
- 拒绝对外层可变绑定的赋值。
- 拒绝 `break` 和 `continue`。
- `v1` 中拒绝循环体内的索引赋值和字段赋值。

使用 `available_parallelism()` 查看运行时工作线程数。

---

## break

- 立即跳出最内层的循环（while 或 for）。
- 在循环外使用会产生编译错误。

```
for i in range(10):
    if i == 5:
        break      # 在 i == 5 时跳出
    print(i)       # 0 1 2 3 4
```

### 错误示例

```
# 在循环外使用 break 会产生编译错误
break      # Error: break outside loop
```

---

## continue

- 结束最内层循环的当前迭代，跳至下一次迭代。
- 在循环外使用会产生编译错误。

```
for i in range(5):
    if i == 2:
        continue   # 跳过 i == 2
    print(i)       # 0 1 3 4
```

---

## ... (Ellipsis)

- 不执行任何操作的语句（no-op）。用作空代码块的占位符。
- 可在任何代码块中使用：函数体、if/elif/else、while、for、match case 等。

```
fn not_yet():
    ...

if true:
    ...
else:
    ...
```

---

## match

### 语法

```
match expression:
    case pattern:
        # 主体
    case pattern if guard_condition:
        # 带守卫的主体
    case _:
        # 通配符（匹配任何值）
```

### 模式的种类

| 模式                   | 示例                            | 说明                     |
|----------------------|-------------------------------|------------------------|
| 通配符                  | <code>-</code>                | 匹配任何值                  |
| 字面值                  | <code>0, "hello", true</code> | 值的相等比较                 |
| 变量绑定                 | <code>n</code>                | 匹配任何值并绑定到变量            |
| enum 变体              | <code>Color::Red</code>       | enum 标签的比较（简单 enum）    |
| ADT enum 变体          | <code>Shape::Circle(r)</code> | 匹配带有关联数据的 enum 变体并绑定   |
| <code>Some(x)</code> | <code>Some(v)</code>          | 当 Option 有值时，绑定其内容     |
| <code>None</code>    | <code>None</code>             | 当 Option 无值时           |
| <code>Ok(x)</code>   | <code>Ok(v)</code>            | 当 Result 为 Ok 时，绑定其内容  |
| <code>Err(x)</code>  | <code>Err(e)</code>           | 当 Result 为 Err 时，绑定错误值 |
| OR 模式                | <code>1 \   2 \   3</code>    | 任一替代方案匹配时即匹配           |

## guard 子句

可以使用 `case pattern if condition:` 的形式指定守卫条件。只有当模式匹配且守卫条件为真时，该分支才会被执行。

## OR 模式

可以使用 `|` 组合多个模式，任一模式匹配时即匹配。OR 模式中不允许使用变量绑定（`n`、`Some(x)`、`Ok(v)`、`Err(e)`）。

```
match x:
    case 1 | 2 | 3:
        print("small")
    case _:
        print("other")

# enum OR 模式
match color:
    case Color::Red | Color::Blue:
        print("warm or cool")
    case Color::Green:
        print("green")
```

## 穷举性检查

- enum 类型：必须覆盖所有变体或包含 `_`。OR 模式中的各替代方案会分别计算。
- Option 类型：必须覆盖 `Some` 和 `None` 或包含 `_`。
- bool 类型：必须覆盖 `true` 和 `false` 或包含 `_`。
- int / float / str 字面值：`_` 为必需。
- 带守卫的分支不计入穷举性。

## 示例

```
# enum 匹配
enum Color:
    Red
    Green
```

Blue

```
match color:
    case Color::Red:
        print("red")
    case Color::Green:
        print("green")
    case Color::Blue:
        print("blue")

# Option 匹配
x: Option<int> = Some(42)
match x:
    case Some(v):
        print(v)
    case None:
        print("nothing")

# Result 匹配
fn divide(a: int, b: int) -> Result<int, Error>:
    if b == 0:
        return Err(Error("division by zero"))
    return Ok(a // b)

match divide(10, 2):
    case Ok(v):
        print(v)          # 5
    case Err(e):
        print(e.message)

# 字面值匹配
match x:
    case 0:
        print("zero")
    case 1:
        print("one")
    case _:
        print("other")

# guard 子句
match x:
    case n if n > 0:
        print("positive")
    case n if n < 0:
        print("negative")
    case _:
        print("zero")
```

## ADT enum 匹配

当 enum 变体携带关联数据时，使用绑定模式来提取值。

```
enum Shape:
    Circle(float)
    Rectangle(float, float)
```



```

    Point

s = Shape::Circle(3.14)
match s:
    case Shape::Circle(r):
        print(r)          # 3.14
    case Shape::Rectangle(w, h):
        print(w)
        print(h)
    case Shape::Point:
        print("point")

```

多字段变体会按声明顺序将各字段绑定到不同的名称。

## 作用域规则

- 各 case 分支拥有块作用域。
- 通过变量绑定模式 (n)、Some(x)、Ok(v) 或 Err(e) 绑定的变量仅在该分支内有效。

## 作用域规则

### 块作用域

- if / elif / else / while / for / match 的各代码块拥有块作用域。
- 在代码块内声明的变量会在代码块结束时离开作用域。

```

for i in range(3):
    tmp = i * 2
# tmp 在此处无法访问

if true:
    a = 1
# a 在此处无法访问

```

### 内层作用域的重新赋值

- 在内层作用域中对变量赋值会修改外层的变量 (Python 风格的作用域)。
- 不会产生遮蔽——内层的赋值会修改同一个变量。

```

x = 10
if true:
    x = 99 # 修改外层的 x
    print(x) # 99
print(x) # 99

```

[English](#) | [日本語](#) | [繁體中文](#)

## 指令

指令是可以附加到声明上的编译时元数据注解。使用 @name 语法，类似于 Java 注解。

## 语法

```

@name
@name(key=value, ...)

```

指令放置在目标声明之前。可以堆叠多个指令。

## 支持的目标

指令可以应用到以下声明:

- `fn` - 函数定义
- `record` - 结构体定义
- 变量声明 (使用 `@const` 或普通赋值)
- `record` 定义内的字段
- `for` - 仅限计数循环, 用于 `@parallel`
- `it` - 测试用例定义 (仅限 `@each` 和 `@property`)

## 内建指令

### `@deprecated`

将声明标记为已弃用。当已弃用的实体被使用 (调用、引用或访问) 时, 会发出编译时警告。

应用于函数:

```
@deprecated
fn old_function() -> int:
    return 42

print(old_function())    # warning: 'old_function' is deprecated
```

应用于类型:

```
@deprecated
record OldPoint:
    x: int
    y: int

@const
p = OldPoint(1, 2)    # warning: 'OldPoint' is deprecated
```

应用于变量:

```
@deprecated
@const
old_value = 99

print(old_value)      # warning: 'old_value' is deprecated
```

应用于字段:

```
record Config:
    @deprecated
    old_setting: int
    new_setting: int

@const
c = Config(1, 2)
print(c.old_setting)    # warning: 'Config.old_setting' is deprecated
print(c.new_setting)    # no warning
```

### `@const`

将变量标记为不可变。使用 `@const` 声明的变量在初始化后无法重新赋值。未使用 `@const` 时, 变量默认为可变。

```
@const
x = 42
# x = 10    # Error: cannot reassign @const variable
```

搭配类型注解:

```
@const
name: str = "hello"
```

元组解构:

```
@const
a, b = (1, 2)
```

**@native**

声明由运行时（内建）提供实现的函数。该函数不能有函数体。

基本语法:

```
@native
fn contains(string: str, substring: str) -> bool
```

```
print(contains("hello world", "world")) # true
```

运算符重载:

```
@native
fn operator+(a: str, b: str) -> str
```

```
print("hello" + " world") # hello world
```

与 UFCS 搭配使用:

```
@native
fn to_upper(string: str) -> str
```

```
print("hello".to_upper()) # HELLO
```

参数数量验证:

当 @native 声明包含类型签名时，编译器会在调用处验证参数数量。支持重载函数（例如：1、2、3 个参数的 range），只要任一重载匹配即通过验证。

```
@native
fn range(n: int) -> List<int>
@native
fn range(start: int, end: int) -> List<int>
```

```
print(length(range(5)))      # OK: matches 1-arg overload
print(length(range(1, 10)))  # OK: matches 2-arg overload
print(length(range()))       # Error: expects 1 or 2 argument(s), but got 0
```

标准库声明 (core/):

core/ 目录包含所有内建函数的 @native 声明，按类别组织：

| 文件               | 内容                                                                             |
|------------------|--------------------------------------------------------------------------------|
| core/builtins.ry | print, length, range, enumerate, zip, exit, args, available_parallelism, sleep |

| 文件                   | 内容                                                                                                                                                |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| core/str.ry          | contains, starts_with, ends_with, find, substring, char_at, replace, to_upper, to_lower, trim, trim_start, trim_end, repeat, reverse, split, join |
| core/convert.ry      | to_int, to_float, to_str                                                                                                                          |
| core/list.ry         | append, pop, insert, remove_at, slice, distinct, flatten, sort, first, last, is_empty                                                             |
| core/map.ry          | keys, values, items, has_key, get, merge                                                                                                          |
| core/set.ry          | add, remove, union, intersection, difference, symmetric_difference, is_subset, is_superset                                                        |
| core/higher_order.ry | filter, map, reduce, fold, any, all, sum, min, max                                                                                                |

当 ry 可执行文件附近存在 core/ 目录时，这些文件会作为前置自动加载。前置机制使得内建函数调用时的参数数量验证生效。

约束: - @native 函数不能有函数体（签名后不能加:）- 提供函数体会导致解析错误: @native function must not have a body - 声明的函数必须对应到现有的内建函数，否则在编译时会发生错误。

未来扩展方向: - @native("libfoo.so") — 绑定外部共享库的 FFI。

#### @parallel

将计数 for 循环标记为并行执行。

```
@parallel
for i in range(8):
    work(i)
```

支持的目标:

- 仅限 for 语句

约束:

- 一个 for 语句上只允许使用一个 @parallel 指令。
- 可迭代对象必须是 range(...) 或整数 .. 范围。
- 不支持解构迭代。
- 禁止对外部可变量赋值。
- v1 中禁止在循环体内使用 break、continue、索引赋值和字段赋值。

#### @each

启用参数化测试，以不同参数多次运行 it 块。

语法:

```
@each([(arg1, arg2, ...), ...])
it("description with {0} and {1}", fn(param1: type, param2: type):
    # test body
)
```

支持的目标: 仅限 it 调用

约束: - 参数必须是元组列表 - 元组的元素数量必须与 lambda 参数数量匹配 - 描述字符串中的 {0}, {1}, ... 会被替换为参数的字符串表示

## @property

启用基于属性的测试，为 `it` 块生成随机输入。

语法：

```
@property(count=100)
it("property name", fn(a: int, b: int):
    # test body with random values
)
```

支持的目标：仅限 `it` 调用

参数：

| 参数                 | 类型               | 默认值 | 说明     |
|--------------------|------------------|-----|--------|
| <code>count</code> | <code>int</code> | 100 | 随机试验次数 |

支持的参数类型：

| 类型                 | 范围               |
|--------------------|------------------|
| <code>int</code>   | -1000 到 1000     |
| <code>float</code> | -1000.0 到 1000.0 |
| <code>bool</code>  | true 或 false     |
| <code>str</code>   | 随机 ASCII、0-20 字符 |

失败时会显示反例（导致失败的参数值）。

## @inline

为 LLVM 优化器提供内联提示。默认情况下，标记函数进行积极内联。

基本用法（始终内联）：

```
@inline
fn add(a: int, b: int) -> int:
    return a + b
```

带 `mode` 参数：

```
@inline(mode="always")
fn hot_path(x: int) -> int:
    return x * 2 + 1
```

```
@inline(mode="hint")
fn medium_path(x: int) -> int:
    return x + 1
```

```
@inline(mode="never")
fn cold_error_handler(msg: str):
    print("ERROR: " + msg)
```

模式：

| 模式          | LLVM 属性      | 说明       |
|-------------|--------------|----------|
| always (默认) | AlwaysInline | 始终内联此函数  |
| hint        | InlineHint   | 向优化器建议内联 |
| never       | NoInline     | 禁止内联此函数  |

约束：- @inline 不能与 @native 一起使用（native 函数没有可内联的函数体）。- 未知的 mode 值会导致编译错误。

### 参数（未来扩展）

指令支持可选的参数语法，为未来扩展做准备：

```
@deprecated(reason="use new_api instead")
fn old_api() -> int:
    return 0
```

目前，参数会被解析但不会被 @deprecated 指令使用。

### 注意事项

- 已弃用的实体仍然正常运作，仅会发出警告。
- 警告在使用点发出，不在定义点发出。
- 定义已弃用的实体但不使用它，不会产生警告。
- 未知的指令名称会导致解析错误。
- 在不支持的目标（如 if、while）上使用指令会导致解析错误。@parallel 是唯一支持在 for 上使用的指令。

[English](#) | [日本語](#) | [繁體中文](#)

## 錯誤參考

### 錯誤格式

ry 以 Rust 風格的豐富格式顯示編譯錯誤，顯示錯誤的確切位置及原始碼上下文：

```
error: cannot reassign @const variable: x
--> main.ry:5:1
|
5 | x = 10
|   ^ cannot reassign @const variable: x
```

每條錯誤訊息包含：- 錯誤等級和訊息 (error: ...) - 檔案位置 (--> 檔案: 行: 列) - 原始碼行 (相關程式碼) - 插入符號指示器 (^) 指向確切的列位置

### 編譯錯誤一覽

| 錯誤內容            | 原因                  | 範例                   |
|-----------------|---------------------|----------------------|
| 使用未宣告的變數        | 參照了未宣告的變數           | print(x) (x 未宣告)     |
| 對 @const 變數重新賦值 | 對以 @const 宣告的變數重新賦值 | @const x = 1 → x = 2 |
| 重新宣告同名變數        | 在同一作用域重新宣告同名變數      | x = 1 → 同名再宣告        |
| 變數型別變更賦值        | 對變數賦值不同型別的值         | x = 1 → x = 3.14     |
| 型別標註不一致         | 宣告時的型別與賦值的型別不同      | x: int = 3.14        |

| 錯誤內容                  | 原因                                 | 範例                                       |
|-----------------------|------------------------------------|------------------------------------------|
| 多載回傳型別重複              | 定義了引數型別相同但僅回傳型別不同的多載               | 引數 (int, int) 但回傳型別分別為 int 和 float 的兩個函式 |
| 多載解析失敗                | 不存在與引數型別匹配的多載                      | fn add(a: int, b: int) 卻呼叫 add(1.0, 2.0) |
| 對位元運算傳入 float         | 對 &、\ 、^、~、<<、>> 傳入 float 型別       | 3.14 & 1                                 |
| 空串列                   | 無法從空串列字面值 [] 推論型別                  | xs = []                                  |
| 空映射                   | 無法從空映射字面值 {} 推論型別                  | m = {}                                   |
| 元組超出範圍的索引             | 存取元組中不存在的索引                        | t = (1, 2) → t.2                         |
| 在迴圈外使用 break/continue | 在 for/while 迴圈外使用 break 或 continue | 在函式頂層使用 break                            |
| 在區塊內匯入模組              | 在函式或條件區塊內使用 from 陳述式               | 在函式內使用 from math                         |
| 循環匯入                  | 模組之間互相匯入                           | a.ry 匯入 b.ry, b.ry 匯入 a.ry               |
| 相同欄位名重複               | 在結構體內定義了兩次相同的欄位名                   | 在 type T: x: int 中定義了兩個 x                |
| match 窮舉性不足           | match 未覆蓋所有模式                      | enum 的部分變體未覆蓋、Option 缺少 None、字面值缺少 _     |
| ? 用於非 Result 型別       | 對非 Result 型別的表達式使用?                | x = 42 → x?                              |
| ? 用於非 Result 函式       | 在不回傳 Result 的函式中                   | fn foo() -> int: → bar()?                |

## 編譯錯誤範例

```

# 對 @const 變數重新賦值
@const
x = 10
x = 20    # 錯誤

# 型別變更賦值
n = 1
n = "hello"    # 錯誤：對 int 型別賦值 str 型別

# 空串列
xs = []    # 錯誤：無法推論型別

# 在迴圈外使用 break
break    # 錯誤：在迴圈外

# 在區塊內匯入
fn foo():
    from math    # 錯誤：僅能在頂層匯入

# 相同欄位名重複
record Bad:

```

```
x: int
x: float # 錯誤：x 重複
```

## 執行時錯誤一覽

| 錯誤內容      | 原因                                                                                           | 範例                                  |
|-----------|----------------------------------------------------------------------------------------------|-------------------------------------|
| 串列超出範圍的存取 | 串列的索引超出範圍                                                                                    | <code>xs = [1, 2, 3] → xs[5]</code> |
| 映射不存在的鍵存取 | 參照映射中不存在的鍵                                                                                   | <code>m = {"a": 1} → m["b"]</code>  |
| 契約違反      | <code>require</code> 、 <code>ensure</code> 或 <code>invariant</code> 條件評估為 <code>false</code> | 參閱 <a href="#">契約式設計</a>            |

所有執行時錯誤都會以 `exit(1)` 結束程序。

### 執行時錯誤範例

```
# 串列超出範圍的存取
xs = [1, 2, 3]
print(xs[10]) # 執行時錯誤：exit(1)
```

```
# 映射不存在的鍵存取
m = {"a": 1}
print(m["z"]) # 執行時錯誤：exit(1)
```

[English](#) | [日本語](#) | [简体中文](#)

## 函数参考

### 函数定义语法

```
fn function_name(param_name: type, ...) -> return_type:
    # body
    return value
```

- 参数类型可省略。省略时视为 `any` 类型。
- 返回类型可省略。省略时会从函数体推断（命名函数和 `lambda` 均如此）。若无 `return` 语句则推断为 `Unit`。若要显式允许任意返回类型，请指定 `-> any`。
- 函数体为缩进的代码块。
- 具有显式返回类型（`Unit` 和 `any` 除外）的函数，必须在所有控制流路径中包含 `return` 语句。若缺少则会产生编译错误。
- 函数可以定义 `require`（前置条件）和 `ensure`（后置条件）子句。参阅[契约式设计](#)。

命名约定：函数名称和参数名称必须使用 `snake_case`（如 `add`、`get_value`、`map_list`）。编译器会强制执行此约定。

```
fn add(a: int, b: int) -> int:
    return a + b
```

```
fn greet(name: str) -> Unit:
    print("Hello, " + name) # 返回类型为 Unit（显式）
```



## 参数与返回值的类型

| 项目   | 说明                                 |
|------|------------------------------------|
| 参数类型 | 可省略。省略：type 时默认为 any               |
| 返回类型 | 可省略。省略时从函数体推断（若无 return 语句则为 Unit） |
| Unit | 不返回值的函数的返回类型                       |

注意：函数参数是不可变的。不能在函数体内对参数重新赋值。这确保了入口时的参数值始终可用于后置条件检查（参阅[契约式设计](#)）。

```
fn no_return(x: int) -> Unit: # 返回类型 Unit（显式）
    print(x)
```

```
fn get_value() -> int:      # 返回类型 int
    return 42
```

```
fn identity(x) -> any:     # 参数类型 any（省略）
    return x
```

### 类型省略与 any

当参数的类型标注被省略时，该参数被视为 any —— 一种在运行时接受任意基本值的动态类型。这类似于 Python 的无类型参数。

```
# 所有参数默认为 any
fn add(a, b):
    return a + b

add(1, 2)           # 3 (int + int)
add("hello", " world") # "hello world" (str + str)
add(1, 2.0)         # 3.0 (int + float)
```

也可以在类型标注中显式使用 any：

```
fn identity(x: any) -> any:
    return x
```

### 返回类型推断

当返回类型省略时，会从函数体中的 return 语句推断：

```
fn double(x: int):      # 返回类型推断为 int
    return x * 2

fn greet(name: str):    # 返回类型推断为 Unit（无 return）
    print("Hello, " + name)
```

若要显式允许任意返回类型，请使用 -> any：

```
fn flexible(x: any) -> any:
    return x           # 可以返回 int、float、str 等
```

## 递归

函数可以调用自身。

```
fn factorial(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

## 尾调用优化

编译器会自动检测自递归尾调用——即函数的最后一个操作是调用自身——并应用 LLVM 的 `musttail` 优化。这保证了尾递归函数使用常量栈空间，防止深度递归时的栈溢出。

```
fn sum_to(n: int, acc: int) -> int:
    if n <= 0:
        return acc
    return sum_to(n - 1, acc + n)    # 尾调用 → 被优化
```

```
sum_to(1000000, 0)    # 不会栈溢出
```

尾调用优化的条件：

- 函数在 `return` 语句中直接调用自身 (`return f(args)`)
- 调用结果不经过任何进一步计算直接返回 (`return n * f(n-1)` 不是尾调用)
- 函数没有 `ensure` (后置条件) 子句

互递归 (A 调用 B, B 调用 A) 目前不会被优化。

## 重载

可以定义参数数量或类型不同的同名函数。

### 规则

- 参数的数量或类型不同即可定义同名函数。
- 调用时会根据参数的类型和数量选择适当的函数。
- 仅返回类型不同的重载是不允许的。

```
fn area(side: int) -> int:
    return side * side
```

```
fn area(w: int, h: int) -> int:
    return w * h
```

```
a = area(5)        # 25
b = area(3, 4)     # 12
```

### 解析优先级

当多个重载匹配一个调用时，编译器使用以下优先级（从高到低）选择最具体的重载：

1. 精确类型匹配 — 参数类型与形参类型完全匹配
2. 隐式拓宽 — 安全的拓宽转换 (`u8 → int`、`u8 → float`、`int → float`)
3. **union** 类型匹配 — 参数类型是 `union` 形参类型的成员
4. **any** 类型匹配 — 形参类型是 `any` (接受任何值)

精确匹配数量最多的重载胜出。如果两个或更多重载具有相同的具体程度，编译器会报告歧义错误。

低级数值类型 (`i8`、`i16`、`i32`、`i64`、`u8`–`u64`、`f32`) 不参与隐式拓宽——需要显式 `as` 转换。

```

fn process(x: int) -> str:
    return "int"

fn process(x) -> str:           # x: any
    return "any"

process(42)           # "int" — 精确匹配 (int) 优于 any
process("hello")      # "any" — str 没有精确匹配, 回退到 any

fn double(x: float) -> float:
    return x * 2.0

double(5)             # OK — int 隐式拓宽为 float, 返回 10.0

```

---

## 默认参数

参数可以有默认值，允许调用者省略尾部的参数。

### 语法

```

fn connect(host: str, port: int = 8080, timeout: int = 30):
    # ...

connect("localhost")           # port=8080, timeout=30
connect("localhost", 3000)      # port=3000, timeout=30
connect("localhost", 3000, 10000) # port=3000, timeout=10000

```

### 规则

- 默认参数必须放在所有非默认参数之后。
- 有默认值的参数必须有显式的类型标注（例如：`x: int = 10`；`x = 10` 是编译错误）。
- 默认值必须是编译时常量表达式（字面量和 `@const` 变量）。
- 如果默认参数导致模糊的重载（参数数量范围重叠且类型匹配），编译器会报告错误。

```

# 错误：模糊的重载
fn calc(x: int, y: int = 0) -> int:
    return x + y
fn calc(x: int) -> int:           # 与上面的 calc(int) 冲突
    return x * 2

```

### 限制

- 泛型函数和 `lambda` 表达式不支持默认参数。
- 

## Unit 类型函数

不返回值的函数会返回 `Unit`。返回类型可以省略（推断为 `Unit`）或用 `-> Unit` 显式指定。

```

fn log(msg: str) -> Unit:
    print(msg)

```

---

## Task 与异步函数

`Task<T>` 是用于并发工作的内置句柄类型。`async fn` 返回 `Task<T>`，`await` 在另一个 `async fn` 内部提取 `T`，`block_on(task)` 从同步上下文中阻塞直到任务完成。

```
async fn add(a: int, b: int) -> int:
    return a + b

# 从同步上下文中，使用 block_on()
t: Task<int> = add(20, 22)
print(block_on(t))                # 42
block_on(add(1, 2))               # 等待并丢弃结果

# 在 async fn 内部，使用 await
async fn double_add(a: int, b: int) -> int:
    return (await add(a, b)) * 2
```

### 规则

- `async fn name(...) -> T`: 使用等待结果类型 `T` 声明。
  - 调用 `async fn` 会立即返回 `Task<T>`。
  - `await expr` 要求 `expr` 为 `Task<T>` 并产生 `T`。
  - `await` 只能在 `async fn` 内部使用。从同步上下文中使用 `block_on(task)`。
  - `block_on(task)` 阻塞当前线程直到任务完成并返回结果。
  - 支持 `async fn ... -> Unit`；当不产生值时，`block_on(task)` 是等待的主要方式。
  - 任务在运行时工作线程池上运行；不是每个任务一个操作系统线程。
  - `v1` 不支持 `async lambda` 和 `async @native fn`。
- 

## Lambda 函数

可以内联定义匿名函数。

### 语法

```
# 单一表达式（返回类型从表达式推断）
fn(param_name: type, ...) => expression

# 参数类型可省略（默认为 any）
fn(param_name, ...) => expression

# 多行代码块
fn(param_name: type, ...):
    # 多个语句
    return value

# 带显式返回类型（可选）
fn(param_name: type, ...) -> return_type => expression
```

### 示例

```
double = fn(x: int) => x * 2
result = double(5)    # 10

add = fn(a: int, b: int) => a + b
sum = add(3, 4)       # 7
```

```
# 多行 lambda
abs = fn(x: int):
    if x < 0:
        return -x
    return x
```

## 闭包

Lambda 函数会以值捕获定义时外层作用域的变量。

```
base = 10
add_base = fn(x: int) => x + base    # 以值捕获 base

base = 99                          # 不影响已捕获的值
r = add_base(5)                    # 15 (使用捕获时的 base = 10)
```

### 捕获规则

| 项目        | 内容         |
|-----------|------------|
| 捕获方式      | 值捕获（复制）    |
| 捕获时机      | Lambda 定义时 |
| 外层变量修改的影响 | 无（因为是复制）   |

## 函数类型

用于将函数作为值处理的类型。

### 语法

```
fn(param_type1, param_type2, ...) -> return_type
```

### 示例

```
f: fn(int) -> int = fn(x: int) => x * 2
g: fn(int, int) -> int = fn(a: int, b: int) => a + b

fn apply(func: fn(int) -> int, x: int) -> int:
    return func(x)

result = apply(f, 5)    # 10
```

## 高阶函数

可以接收函数作为参数，或将函数作为返回值返回。

```
fn map_list(xs: List<int>, f: fn(int) -> int) -> List<int>:
    result: List<int> = []
    for x in xs:
        result += [f(x)]
```

```

    return result

doubled = map_list([1, 2, 3], fn(x: int) => x * 2)
# [2, 4, 6]

```

---

## 泛型函数

函数可以有类型参数，实现类型安全的复用而无需代码重复。

语法

```

fn name<T, U>(param1: T, param2: U) -> T:
    # 使用 T、U 作为类型的函数体

```

示例

```

fn identity<T>(x: T) -> T:
    return x

# 显式类型参数
result = identity[int](42)      # 42
result = identity[str]("hello") # "hello"

# 类型推断（从实际参数推导类型参数）
result = identity(42)           # T = int, result = 42
result = identity("hello")      # T = str, result = "hello"

```

多个类型参数

```

fn pick_first<T, U>(a: T, b: U) -> T:
    return a

result = pick_first(1, "x")      # T = int, U = str, result = 1
result = pick_first("hello", 42) # T = str, U = int, result = "hello"

```

类型约束（边界）

类型参数可以使用 `: RecordName` 语法以 `record` 类型进行约束。具体类型必须是绑定类型本身或其子类型。

```

record Animal:
    name: str
    legs: int

record Dog < Animal:
    breed: str

fn get_name<T: Animal>(a: T) -> str:
    return a.name

get_name(Dog("Rex", 4, "Lab")) # OK — Dog 是 Animal 的子类型
get_name(Animal("Cat", 4))     # OK — 精确类型匹配

```

有约束和无约束的类型参数可以混合使用：

```

fn pair_name<T: Animal, U>(a: T, x: U) -> str:
    return a.name

```

## 工作原理

泛型函数使用单态化：为每个唯一的类型参数组合生成函数的特化版本。相同的实例化会被缓存并在多次调用间复用。当存在类型约束时，会在实例化时进行验证。

---

## UFCS（统一函数调用语法）

可以使用 `a.f(b)` 的形式调用 `f(a, b)`。方便用于方法链。

### 语法

```
# 普通调用
f(a, b)

# UFCS 调用（等价）
a.f(b)
```

### 链接

```
fn double(x: int) -> int:
    return x * 2

fn add_one(x: int) -> int:
    return x + 1

result = 5.double().add_one()    # double(5) -> 10, add_one(10) -> 11
```

### 与字段访问混用

字段访问 (`.field`) 和 UFCS (`.method()`) 使用相同的点号记法，但通过是否有参数来区分。

```
p = Point(3, 4)
length = p.x.to_float()    # 字段访问 + UFCS
```

---

## 运算符重载

可以为用户定义类型定义运算符的行为。

### 语法

```
# 二元运算符（2 个参数）
fn operator<op>(a: type, b: type) -> return_type:
    ...

# 一元运算符（1 个参数）
fn operator<op>(a: type) -> return_type:
    ...
```

### 可重载的运算符

---

| 类别     | 运算符                                      |
|--------|------------------------------------------|
| 算术（二元） | <code>+ - * / % ** //</code>             |
| 比较（二元） | <code>== != &lt; &lt;= &gt; &gt;=</code> |

| 类别      | 运算符            |
|---------|----------------|
| 位运算（二元） | & \   ^ << >>  |
| 逻辑（二元）  | and or         |
| 成员测试    | in             |
| 下标      | []（读取）、[]=（写入） |
| 调用      | ()             |
| 转换      | as             |
| 一元      | - ~ not        |

## 返回类型约束

比较、逻辑和成员测试运算符必须返回 bool：

| 类别   | 运算符             | 必需返回类型   |
|------|-----------------|----------|
| 比较   | == != < <= > >= | bool     |
| 逻辑   | and or not      | bool     |
| 成员测试 | in              | bool     |
| 转换   | as              | 必需（目标类型） |

*# OK*

```
fn operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y
```

*# Error: comparison operator '==' must return 'bool', but returns 'int'*

```
fn operator==(a: Vec2, b: Vec2) -> int:
    return 42
```

算术运算符和位运算符没有返回类型约束。

## 二元 / 一元的区别

依参数个数区分。

```
record Vec2:
    x: float
    y: float
```

*# 二元 +*

```
fn operator+(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)
```

*# 一元 -*

```
fn operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)
```

*# 比较*

```
fn operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y
```

```
v1 = Vec2(1.0, 2.0)
```

```
v2 = Vec2(3.0, 4.0)
```

```
v3 = v1 + v2      # Vec2(4.0, 6.0)
```

```
v4 = -v1          # Vec2(-1.0, -2.0)
```



## 检查/饱和算术

用于低级整数类型（i8、i16、i32、i64、u8、u16、u32、u64）的显式溢出控制内置函数。两个参数必须是相同类型。

| 函数                   | 返回值              | 行为             |
|----------------------|------------------|----------------|
| checked_add(a, b)    | Result<T, Error> | 溢出时返回 Err      |
| checked_sub(a, b)    | Result<T, Error> | 下溢时返回 Err      |
| checked_mul(a, b)    | Result<T, Error> | 溢出时返回 Err      |
| saturating_add(a, b) | T                | 溢出时钳制到类型最小/最大值 |
| saturating_sub(a, b) | T                | 下溢时钳制到类型最小/最大值 |
| saturating_mul(a, b) | T                | 溢出时钳制到类型最小/最大值 |
| wrapping_add(a, b)   | T                | 显式回绕（与 + 相同）   |
| wrapping_sub(a, b)   | T                | 显式回绕（与 - 相同）   |
| wrapping_mul(a, b)   | T                | 显式回绕（与 * 相同）   |

```
# Checked: 返回 Result, 使用 match 或 ? 处理
r = checked_add(2147483647i32, 1i32)
match r:
    case Ok(v):
        print(v)
    case Err(e):
        print("overflow!")    # 输出 "overflow!"

# Saturating: 钳制到边界
v = saturating_add(2147483647i32, 100i32)
print(v as int)    # 2147483647

# Wrapping: 自文档化的回绕行为
v = wrapping_add(2147483647i32, 1i32)
print(v as int)    # -2147483648
```

注意：这些函数不支持浮点类型（f32）或高级 int 类型。低级整数的默认 +、-、\* 运算符使用回绕行为（有符号为补码，无符号为模运算）。

[English](#)

## HTTP 参考手册

### 类型

| 类型                 | 说明               |
|--------------------|------------------|
| HttpRequest        | 传入 HTTP 请求的不透明句柄 |
| HttpResponse       | 传出 HTTP 响应的不透明句柄 |
| HttpClientResponse | HTTP 客户端响应的不透明句柄 |

HttpRequest 由服务器框架提供。HttpResponse 通过 http\_response() 创建。HttpClientResponse 由客户端函数（http\_get、http\_post、http\_request）返回。

### 函数（来自 http）

这些函数需要显式导入：

```
from http import http_listen, http_method, http_path, http_header, http_body, http_query, http_query_all
```

## 服务器

| 函数          | 签名                                                                                                                          | 说明                                                                               |
|-------------|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| http_listen | (host: str, port: int, handler: fn(HttpRequest) -> HttpResponse) -> Unit                                                    | 在指定地址启动 HTTP 服务器。在接受循环中阻塞，对每个请求调用 handler。                                       |
| http_listen | (host: str, port: int, handler: fn(HttpRequest) -> HttpResponse, max_requests: int) -> Unit                                 | 启动一个在处理 max_requests 个请求后停止的 HTTP 服务器。支持 async fn + block_on() 的生命周期管理。          |
| http_listen | (host: str, port: int, handler: fn(HttpRequest) -> HttpResponse, max_requests: int, port_callback: fn(int) -> Unit) -> Unit | 与上述相同，但在 bind + listen 成功后调用 port_callback 并传入实际绑定的端口。可与端口 0 配合使用以获取操作系统分配的临时端口。 |

## 请求访问器

| 函数               | 签名                                                     | 说明                                                                               |
|------------------|--------------------------------------------------------|----------------------------------------------------------------------------------|
| http_method      | (req: HttpRequest) -> str                              | 返回 HTTP 方法（例如 "GET"、"POST"）。                                                     |
| http_path        | (req: HttpRequest) -> str                              | 返回不含查询字符串的请求路径（例如对于 "/search?q=hello" 返回 "/search"）。                             |
| http_header      | (req: HttpRequest, key: str) -> Option<str>            | 返回请求头的值（不区分大小写查找）。未找到时返回 None。                                                   |
| http_body        | (req: HttpRequest) -> str                              | 以字符串形式返回请求体。                                                                     |
| http_query       | (req: HttpRequest, key: str) -> Option<str>            | 返回查询参数的值。未找到时返回 None。值会自动进行 URL 解码。                                              |
| http_query_all   | (req: HttpRequest) -> Map<str, str>                    | 以映射形式返回所有查询参数。键和值会自动进行 URL 解码。                                                   |
| http_cookie      | (req: HttpRequest, name: str) -> Option<str>           | 按名称返回 cookie 的值。未找到时返回 None。                                                     |
| http_cookies     | (req: HttpRequest) -> Map<str, str>                    | 以映射形式返回所有 cookie。                                                                |
| http_form_field  | (req: HttpRequest, name: str) -> Option<str>           | 返回 multipart 表单文本字段的值。未找到时返回 None。                                               |
| http_form_file   | (req: HttpRequest, name: str) -> Option<Map<str, str>> | 以 Option 形式返回文件上传信息。Some(map) 包含 "filename"、"content_type"、"data" 键。未找到时返回 None。 |
| http_form_fields | (req: HttpRequest) -> Map<str, str>                    | 以映射形式返回所有 multipart 表单文本字段。                                                      |

## 响应构建器

| 函数            | 签名                                                               | 说明                          |
|---------------|------------------------------------------------------------------|-----------------------------|
| http_response | (status: int, headers: Map<str, str>, body: str) -> HttpResponse | 使用给定的状态码、响应头和响应体创建 HTTP 响应。 |

## 使用示例

### 基本 HTTP 服务器

```
from http import http_listen, http_method, http_path, http_header, http_body, http_response

http_listen("127.0.0.1", 8080, fn(req: HttpRequest) -> HttpResponse:
    method = http_method(req)
    path = http_path(req)
    if path == "/hello":
        return http_response(200, {"Content-Type": "text/plain"}, "Hello, World!")
    if path == "/echo":
        body = http_body(req)
        return http_response(200, {"Content-Type": "text/plain"}, body)
    return http_response(404, {"Content-Type": "text/plain"}, "Not Found")
)
```

### 使用 async fn 的非阻塞服务器

```
from http import http_listen, http_path, http_response

async fn start_server(port: int) -> str:
    http_listen("127.0.0.1", port, fn(req: HttpRequest) -> HttpResponse:
        path = http_path(req)
        if path == "/api/health":
            return http_response(200, {"Content-Type": "application/json"}, "{\"status\": \"ok\"}")
        return http_response(404, {"Content-Type": "text/plain"}, "Not Found")
    )
    return "done"

t = start_server(8080)
# 服务器在后台任务中运行
```

### 带请求限制的服务器 (max\_requests)

```
from http import http_listen, http_path, http_response, http_get, http_client_status, http_client_body

port_holder = [0]
fn on_port(p: int) -> Unit:
    port_holder[0] = p

async fn start_server() -> str:
    http_listen("127.0.0.1", 0, fn(req: HttpRequest) -> HttpResponse:
        return http_response(200, {"Content-Type": "text/plain"}, "Hello!")
    , 1, on_port) # 处理 1 个请求后停止；调用 on_port 传入绑定的端口
    return "done"

t = start_server()
sleep(100) # 等待服务器启动
port = port_holder[0]

match http_get("http://127.0.0.1:" + to_str(port) + "/"):
    case Ok(resp):
        print(http_client_body(resp)) # "Hello!"
    case Err(e):
        print("error")
```

```
result = block_on(t) # 服务器在处理 1 个请求后退出; block_on 完成
```

读取查询参数

```
from http import http_listen, http_path, http_query, http_query_all, http_response

http_listen("127.0.0.1", 8080, fn(req: HttpRequest) -> HttpResponse:
    path = http_path(req)
    if path == "/search":
        match http_query(req, "q"):
            case Some(query):
                return http_response(200, {"Content-Type": "text/plain"}, "Search: " + query)
            case None:
                return http_response(400, {"Content-Type": "text/plain"}, "Missing query parameter: q")
    return http_response(404, {"Content-Type": "text/plain"}, "Not Found")
)
```

读取请求头

```
from http import http_listen, http_header, http_response

http_listen("127.0.0.1", 8080, fn(req: HttpRequest) -> HttpResponse:
    match http_header(req, "Authorization"):
        case Some(token):
            return http_response(200, {"Content-Type": "text/plain"}, "Authenticated: " + token)
        case None:
            return http_response(401, {"Content-Type": "text/plain"}, "Unauthorized")
)
```

处理表单提交

```
from http import http_listen, http_form_field, http_form_file, http_response

http_listen("127.0.0.1", 8080, fn(req: HttpRequest) -> HttpResponse:
    match http_form_field(req, "username"):
        case Some(name):
            match http_form_file(req, "avatar"):
                case Some(file_info):
                    filename = file_info["filename"]
                    return http_response(200, {"Content-Type": "text/plain"}, "Hello " + name + ", file: " + filename)
                case None:
                    return http_response(400, {"Content-Type": "text/plain"}, "No file uploaded")
        case None:
            return http_response(400, {"Content-Type": "text/plain"}, "Missing username")
)
```

读取 Cookie

```
from http import http_listen, http_cookie, http_cookies, http_response

http_listen("127.0.0.1", 8080, fn(req: HttpRequest) -> HttpResponse:
    match http_cookie(req, "session_id"):
        case Some(sid):
            return http_response(200, {"Content-Type": "text/plain"}, "Session: " + sid)
)
```

```

    case None:
        return http_response(401, {"Content-Type": "text/plain"}, "No session")
)

```

## 行为

- `http_listen()` 绑定到地址，开始监听，并进入接受循环。
- 使用 3 个参数调用时，接受循环无限运行。
- 使用 4 个参数调用时 (`max_requests`)，服务器在处理指定数量的请求后停止。`max_requests` 必须是正整数。这支持 `async fn + block_on()` 的生命周期管理。格式错误的请求（静默跳过）不计入限制。
- 使用 5 个参数调用时 (`max_requests port_callback`) 在 `bind + listen` 成功后同步调用 `port_callback` 并传入实际绑定的端口。这允许安全使用端口 0（操作系统分配的临时端口）以避免并行测试中的端口冲突。
- 服务器默认支持 HTTP/1.1 keep-alive。单个连接上可以处理多个请求。服务器在每个请求上检查 `Connection` 头：如果发送了 `Connection: close`，则在响应后关闭连接；否则连接保持打开以处理后续请求。空闲连接在 5 秒超时后关闭。
- 如果响应头映射中未提供 `Content-Length`，则会自动添加。
- 服务器支持基于 `Content-Length` 的请求体读取和 `Transfer-Encoding: chunked` 解码的 HTTP/1.1。
- 当请求中存在 `Transfer-Encoding: chunked` 时，请求体会自动解码并拼接 - Ry 代码透明地接收完整的请求体。
- 如果同时存在 `Transfer-Encoding: chunked` 和 `Content-Length`，请求将被视为格式错误而拒绝（依据 RFC 9112）。
- 要发送分块响应，请在传给 `http_response()` 的响应头映射中包含 `"Transfer-Encoding": "chunked"`。响应体将自动以分块格式编码。
- 通过 `http_header()` 的头部查找不区分大小写。
- `http_path()` 返回不含查询字符串的路径。查询参数通过 `http_query()` 或 `http_query_all()` 单独访问。
- 查询参数值会自动进行 URL 解码（`%20` -> 空格，`+` -> 空格）。
- 对于重复的查询参数键，返回第一个值。
- `http_cookie()` 和 `http_cookies()` 通过按 `;` 分割来解析 `Cookie` 头，然后按第一个 `=` 分割每个键值对。名称和值的前后空白会被去除。
- 对于重复的 `cookie` 名称，返回第一个值。
- `Cookie` 值可以包含 `=` 字符（只有第一个 `=` 用于分隔名称和值）。
- `http_form_field()`、`http_form_file()` 和 `http_form_fields()` 解析 `multipart/form-data` 请求体。解析是惰性的 - 在第一次调用时解析请求体并缓存。
- `boundary` 参数从 `Content-Type` 头中提取，支持带引号和不带引号的值。
- `Content-Disposition` 中带有 `filename` 的部分被视为文件上传；没有的则被视为文本字段。
- 对于重复的字段/文件名称，返回第一个值。
- `http_form_file()` 返回 `Some(map)`，包含 `"filename"`、`"content_type"` 和 `"data"` 键；如果未找到字段则返回 `None`。如果部分未指定 `Content-Type`，默认为 `"application/octet-stream"`。
- 对于非 `multipart` 请求，表单函数返回 `None`（对于 `http_form_field`、`http_form_file`）或空映射（对于 `http_form_fields`）。

## 支持的状态码

以下状态码具有标准原因短语（RFC 9110）：

| 代码  | 原因                            |
|-----|-------------------------------|
| 100 | Continue                      |
| 101 | Switching Protocols           |
| 200 | OK                            |
| 201 | Created                       |
| 202 | Accepted                      |
| 203 | Non-Authoritative Information |
| 204 | No Content                    |
| 205 | Reset Content                 |

| 代码  | 原因                         |
|-----|----------------------------|
| 206 | Partial Content            |
| 300 | Multiple Choices           |
| 301 | Moved Permanently          |
| 302 | Found                      |
| 303 | See Other                  |
| 304 | Not Modified               |
| 307 | Temporary Redirect         |
| 308 | Permanent Redirect         |
| 400 | Bad Request                |
| 401 | Unauthorized               |
| 403 | Forbidden                  |
| 404 | Not Found                  |
| 405 | Method Not Allowed         |
| 406 | Not Acceptable             |
| 408 | Request Timeout            |
| 409 | Conflict                   |
| 410 | Gone                       |
| 411 | Length Required            |
| 413 | Content Too Large          |
| 414 | URI Too Long               |
| 415 | Unsupported Media Type     |
| 416 | Range Not Satisfiable      |
| 417 | Expectation Failed         |
| 422 | Unprocessable Content      |
| 426 | Upgrade Required           |
| 429 | Too Many Requests          |
| 500 | Internal Server Error      |
| 501 | Not Implemented            |
| 502 | Bad Gateway                |
| 503 | Service Unavailable        |
| 504 | Gateway Timeout            |
| 505 | HTTP Version Not Supported |

其他状态码使用 "Unknown" 作为原因短语。

## HTTP 客户端

### 客户端函数

| 函数                     | 签名                                                                                                             | 说明                         |
|------------------------|----------------------------------------------------------------------------------------------------------------|----------------------------|
| <code>http_get</code>  | <code>(url: str) -&gt; Result&lt;HttpClientResponse, Error&gt;</code>                                          | 向给定 URL 发送 HTTP GET 请求。    |
| <code>http_post</code> | <code>(url: str, body: str, headers: Map&lt;str, str&gt;) -&gt; Result&lt;HttpClientResponse, Error&gt;</code> | 发送带有请求体和请求头的 HTTP POST 请求。 |

| 函数                        | 签名                                                                                                                          | 说明                 |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------|--------------------|
| <code>http_request</code> | <code>(method: str, url: str, headers: Map&lt;str, str&gt;, body: str) -&gt; Result&lt;HttpClientResponse, Error&gt;</code> | 使用自定义方法发送 HTTP 请求。 |

## 响应访问器

| 函数                                     | 签名                                                                        | 说明                                         |
|----------------------------------------|---------------------------------------------------------------------------|--------------------------------------------|
| <code>http_client_status</code>        | <code>(resp: HttpClientResponse) -&gt; int</code>                         | 返回 HTTP 状态码。                               |
| <code>http_client_body</code>          | <code>(resp: HttpClientResponse) -&gt; str</code>                         | 以字符串形式返回响应体。                               |
| <code>http_client_header</code>        | <code>(resp: HttpClientResponse, key: str) -&gt; Option&lt;str&gt;</code> | 返回响应头的值（不区分大小写）。未找到时返回 <code>None</code> 。 |
| <code>http_client_response_free</code> | <code>(resp: HttpClientResponse) -&gt; Unit</code>                        | 释放响应及其关联的内存。使用完响应后调用。                      |

## 客户端使用示例

```
from http import http_get, http_post, http_client_status, http_client_body, http_client_header
```

# 简单的 GET 请求

```
match http_get("http://example.com/api/data"):
    case Ok(resp):
        status = http_client_status(resp)
        body = http_client_body(resp)
        print(to_str(status) + ": " + body)
    case Err(e):
        print("Request failed")
```

# 带请求体和请求头的 POST 请求

```
headers: Map<str, str> = {"Content-Type": "application/json"}
match http_post("http://example.com/api/data", "{\"key\": \"value\"}", headers):
    case Ok(resp):
        print(http_client_body(resp))
    case Err(e):
        print("Request failed")
```

## 客户端行为

- 支持 `http://` 和 `https://` URL。HTTPS 使用 TLS 和系统 CA 证书包进行证书验证。
- Host 头会根据 URL 自动添加。
- 始终发送 `Connection: close`；每个请求使用单独的 TCP 连接。
- Content-Length 始终自动添加（包括空请求体的 0）。用户提供的 Content-Length 头会被正确的值覆盖。
- 响应体读取支持 Content-Length、Transfer-Encoding: chunked 或读取到连接关闭。
- 响应头查找不区分大小写。
- 连接超时为 5 秒；接收超时为 30 秒。
- 连接失败、无效 URL 或格式错误的响应时返回 `Err`。
- `HttpClientResponse` 拥有已分配的内存（头部、响应体）。使用完毕后调用 `http_client_response_free()` 以避免内存泄漏。

## 重定向行为

HTTP 客户端函数会自动跟随重定向响应（带有 Location 头的 3xx 响应）。

- 支持的状态码：301、302、303、307、308
- 最大重定向次数：10（超过时返回 Err）
- 方法转换（依据 RFC 9110）：
  - 301、302：POST 会被更改为 GET（请求体被丢弃）；其他方法保持不变
  - 303：方法始终更改为 GET（请求体被丢弃）
  - 307、308：方法和请求体保持不变
- **URL 解析**：支持 Location 头中的绝对 URL、协议相对 URL (//...)、绝对路径 (/...) 和相对路径
- 用户提供的头部在每次重定向跳转时重新发送，但敏感头部（Authorization、Proxy-Authorization、Cookie）在跨源重定向（不同主机或端口）时会被移除
- 如果 Location 头缺失或为空，则按原样返回重定向响应
- 调用者仅接收所有重定向完成后的最终响应

## 错误处理

- 如果 bind() 失败（例如端口已被占用），http\_listen() 会引发运行时错误。
- 格式错误的请求或 keep-alive 连接上的空闲超时会导致连接关闭。服务器随后继续接受新连接。
- 处理函数必须始终返回 HttpResponse - 没有默认响应。
- 客户端函数返回 Result<HttpClientResponse, Error> - 使用 match 处理成功和失败情况。

[English](#) | [日本語](#) | [繁體中文](#)

## I/O 函数参考手册

标准输入输出与文件操作。所有函数均需从 io 明确导入。

```
from io import read_text, write_text, file_exists
```

### 函数列表

#### 标准输入

| 函数        | 签名         | 说明                   |
|-----------|------------|----------------------|
| read_line | ( ) -> str | 从 stdin 读取一行（移除末尾换行） |
| read_all  | ( ) -> str | 读取 stdin 直到 EOF      |

#### 文件 I/O

| 函数          | 签名                                     | 说明           |
|-------------|----------------------------------------|--------------|
| read_text   | (str) -> Result<str, Error>            | 将整个文件作为字符串读取 |
| write_text  | (str, str) -> Result<Unit, Error>      | 将字符串写入文件（覆盖） |
| append_text | (str, str) -> Result<Unit, Error>      | 在文件末尾追加字符串   |
| file_exists | (str) -> bool                          | 检查文件是否存在     |
| delete_file | (str) -> Result<Unit, Error>           | 删除文件         |
| read_bytes  | (str) -> Result<List<u8>, Error>       | 将文件作为字节列表读取  |
| write_bytes | (str, List<u8>) -> Result<Unit, Error> | 将字节列表写入文件    |



## 字节转换

| 函数                        | 签名                                                           | 说明               |
|---------------------------|--------------------------------------------------------------|------------------|
| <code>str_to_bytes</code> | <code>(str) -&gt; List&lt;u8&gt;</code>                      | 将字符串转换为 UTF-8 字节 |
| <code>bytes_to_str</code> | <code>(List&lt;u8&gt;) -&gt; Result&lt;str, Error&gt;</code> | 将字节列表转换为字符串      |

## 使用示例

### 读写文件

```
from io import read_text, write_text, append_text, file_exists, delete_file

match write_text("hello.txt", "Hello, World!"):
    case Ok(_):
        match read_text("hello.txt"):
            case Ok(content):
                print(content)    # Hello, World!
            case Err(e):
                print(e.message)
    case Err(e):
        print(e.message)

print(file_exists("hello.txt"))    # true

match delete_file("hello.txt"):
    case Ok(_):
        print(file_exists("hello.txt"))    # false
    case Err(e):
        print(e.message)
```

### 字节操作

```
from io import str_to_bytes, bytes_to_str, write_bytes, read_bytes

bs = str_to_bytes("ABC")
print(length(bs))    # 3

match write_bytes("data.bin", bs):
    case Ok(_):
        match read_bytes("data.bin"):
            case Ok(rb):
                match bytes_to_str(rb):
                    case Ok(s):
                        print(s)    # ABC
                    case Err(e):
                        print(e.message)
            case Err(e):
                print(e.message)
    case Err(e):
        print(e.message)
```

从标准输入读取

```
from io import read_line

name = read_line()
print(f"Hello, {name}!")
```

## 错误处理

文件操作返回 `Result<T, Error>` 而不是在失败时终止程序。使用 `match` 配合 `Ok/Err` 模式来处理错误:

```
match read_text("missing.txt"):
    case Ok(content):
        print(content)
    case Err(e):
        print(e.message)    # cannot open file 'missing.txt' for reading
```

| 操作                                                                            | 错误条件        |
|-------------------------------------------------------------------------------|-------------|
| <code>read_text</code> / <code>read_bytes</code>                              | 文件不存在或无法打开  |
| <code>write_text</code> / <code>write_bytes</code> / <code>append_text</code> | 无法打开文件进行写入  |
| <code>delete_file</code>                                                      | 无法删除文件      |
| <code>bytes_to_str</code>                                                     | 输入包含 NUL 字节 |

## 备注

- 使用 `List<u8>` 作为缓冲区类型。标准列表操作 (`length()`、`append()`、`slice()`、索引访问) 均可用于字节列表。
- 文件路径若未指定绝对路径, 则为相对于当前工作目录的相对路径。
- `write_text` 与 `write_bytes` 会覆盖现有文件。若要追加内容, 请使用 `append_text`。

[English](#) | [日本語](#) | [繁體中文](#)

## JSON 函数参考

JSON 解析与序列化。所有函数需要从 `json` 明确导入。

```
from json import parse, stringify, json_type, json_get, json_at, json_str, json_int, json_float, json_b
```

### 概述

`json` 包提供将 JSON 文本解析为不透明的 `JsonValue` 类型、通过访问函数读取其内容、以及重新序列化为文本的功能。由于 JSON 值可以是异构的 (对象可以包含字符串、数字、布尔值、数组和嵌套对象), 本包使用不透明指针类型和类型化的访问函数。

### 函数列表

解析 / 序列化

| 函数                     | 签名                                                      | 说明                                       |
|------------------------|---------------------------------------------------------|------------------------------------------|
| <code>parse</code>     | <code>(str) -&gt; Result&lt;JsonValue, Error&gt;</code> | 将 JSON 字符串解析为 <code>JsonValue</code>     |
| <code>stringify</code> | <code>(JsonValue) -&gt; str</code>                      | 将 <code>JsonValue</code> 序列化为紧凑的 JSON 文本 |
| <code>stringify</code> | <code>(JsonValue, int) -&gt; str</code>                 | 以缩进格式输出 (参数为空格数)                         |

## 类型查询

| 函数        | 签名                 | 说明                                                                   |
|-----------|--------------------|----------------------------------------------------------------------|
| json_type | (JsonValue) -> str | 返回 JSON 类型: "object", "array", "string", "number", "boolean", "null" |

## 对象 / 数组访问

| 函数       | 签名                                           | 说明         |
|----------|----------------------------------------------|------------|
| json_get | (JsonValue, str) -> Result<JsonValue, Error> | 以键从对象获取字段  |
| json_at  | (JsonValue, int) -> Result<JsonValue, Error> | 以索引从数组获取元素 |

## 值提取

| 函数         | 签名                                  | 说明     |
|------------|-------------------------------------|--------|
| json_str   | (JsonValue) -> Result<str, Error>   | 提取字符串值 |
| json_int   | (JsonValue) -> Result<int, Error>   | 提取整数值  |
| json_float | (JsonValue) -> Result<float, Error> | 提取浮点数值 |
| json_bool  | (JsonValue) -> Result<bool, Error>  | 提取布尔值  |

## 集合信息

| 函数        | 签名                       | 说明          |
|-----------|--------------------------|-------------|
| json_len  | (JsonValue) -> int       | 返回数组长度或对象键数 |
| json_keys | (JsonValue) -> List<str> | 返回对象的键列表    |

## 内存管理

| 函数        | 签名                  | 说明                   |
|-----------|---------------------|----------------------|
| json_free | (JsonValue) -> Unit | 释放 JsonValue 及其所有子元素 |

## 使用示例

### 解析与访问字段

```
from json import parse, json_get, json_str, json_int, json_free

match parse("{\"name\": \"Alice\", \"age\": 30}"):
    case Ok(data):
        match json_get(data, "name"):
            case Ok(val):
                match json_str(val):
                    case Ok(name):
                        print(name)      # "Alice"
```

```

        case Err(e):
            print("error")
    case Err(e):
        print("error")
    json_free(data)
case Err(e):
    print("parse error: " + e.message)

```

## 处理数组

```

from json import parse, json_at, json_int, json_len, json_free

match parse("[10, 20, 30]"):
    case Ok(data):
        print(to_str(json_len(data)))    # 3
        match json_at(data, 0):
            case Ok(elem):
                match json_int(elem):
                    case Ok(n):
                        print(to_str(n))    # 10
                    case Err(e):
                        print("error")
            case Err(e):
                print("error")
        json_free(data)
    case Err(e):
        print("parse error")

```

## 带缩进的序列化

```

from json import parse, stringify, json_free

match parse("{\"key\":\"value\",\"count\":42}"):
    case Ok(data):
        print(stringify(data, 2))
        # {
        #   "key": "value",
        #   "count": 42
        # }
        json_free(data)
    case Err(e):
        print("error")

```

## 注意事项

- `json_int` 接受整数和整数值的浮点数（例如 `42.0 → 42`）
- `json_float` 接受浮点数和整数（例如 `42 → 42.0`）
- `json_get` 和 `json_at` 返回解析树中子元素的引用——不要对子元素调用 `json_free`，只对 `parse` 返回的根值调用
- `json_type` 对整数和浮点数都返回 `"number"`

[English](#) | [日本語](#) | [繁體中文](#)

## 數學函式 (math)

### 概述

math 套件提供數學常數與函式。與 std 套件不同，它不會自動匯入。需要使用明確匯入來存取函式。

```
from math import sqrt, PI, sin
```

---

### 常數

| 常數  | 型別    | 說明                      |
|-----|-------|-------------------------|
| PI  | float | 圓周率 (3.141592653589793) |
| E   | float | 尤拉數 (2.718281828459045) |
| Inf | float | 正無窮大                    |
| NaN | float | 非數值 (Not a Number)      |

```
from math import PI, E, Inf, NaN
```

```
circumference = 2.0 * PI * radius
```

---

### 絕對值

| 函式     | 簽章               | 說明      |
|--------|------------------|---------|
| abs(x) | (int) -> int     | 整數的絕對值  |
| abs(x) | (float) -> float | 浮點數的絕對值 |

```
from math import abs
```

```
abs(-5)      # 5
abs(-3.14)   # 3.14
```

---

### 捨入

| 函式       | 簽章             | 說明   |
|----------|----------------|------|
| floor(x) | (float) -> int | 向下取整 |
| ceil(x)  | (float) -> int | 向上取整 |
| round(x) | (float) -> int | 四捨五入 |

```
from math import floor, ceil, round
```

```
floor(3.7)   # 3
ceil(3.2)    # 4
round(3.5)   # 4
```

---

## 幕次與平方根

| 函式                     | 簽章                                      | 說明       |
|------------------------|-----------------------------------------|----------|
| <code>sqrt(x)</code>   | <code>(float) -&gt; float</code>        | 平方根      |
| <code>pow(x, y)</code> | <code>(float, float) -&gt; float</code> | x 的 y 次方 |

```
from math import sqrt, pow
```

```
sqrt(9.0)      # 3.0  
pow(2.0, 3.0)  # 8.0
```

## 對數

| 函式                    | 簽章                               | 說明           |
|-----------------------|----------------------------------|--------------|
| <code>log(x)</code>   | <code>(float) -&gt; float</code> | 自然對數 (底數 e)  |
| <code>log2(x)</code>  | <code>(float) -&gt; float</code> | 底數 2 的對數     |
| <code>log10(x)</code> | <code>(float) -&gt; float</code> | 常用對數 (底數 10) |

## 指數

| 函式                  | 簽章                               | 說明       |
|---------------------|----------------------------------|----------|
| <code>exp(x)</code> | <code>(float) -&gt; float</code> | e 的 x 次方 |

## 三角函式

| 函式                  | 簽章                               | 說明         |
|---------------------|----------------------------------|------------|
| <code>sin(x)</code> | <code>(float) -&gt; float</code> | 正弦 (x 為弧度) |
| <code>cos(x)</code> | <code>(float) -&gt; float</code> | 餘弦 (x 為弧度) |
| <code>tan(x)</code> | <code>(float) -&gt; float</code> | 正切 (x 為弧度) |

## 反三角函式

| 函式                   | 簽章                               | 說明          |
|----------------------|----------------------------------|-------------|
| <code>asin(x)</code> | <code>(float) -&gt; float</code> | 反正弦 (結果為弧度) |
| <code>acos(x)</code> | <code>(float) -&gt; float</code> | 反餘弦 (結果為弧度) |
| <code>atan(x)</code> | <code>(float) -&gt; float</code> | 反正切 (結果為弧度) |

## 雙參數函式

| 函式                       | 簽章                                      | 說明                       |
|--------------------------|-----------------------------------------|--------------------------|
| <code>atan2(y, x)</code> | <code>(float, float) -&gt; float</code> | y/x 的反正切 (結果為弧度)         |
| <code>hypot(x, y)</code> | <code>(float, float) -&gt; float</code> | 斜邊長度: $\sqrt{x^2 + y^2}$ |

## 判定函式

| 函式                     | 簽章                              | 說明                  |
|------------------------|---------------------------------|---------------------|
| <code>is_nan(x)</code> | <code>(float) -&gt; bool</code> | 若 x 為 NaN 則回傳 true  |
| <code>is_inf(x)</code> | <code>(float) -&gt; bool</code> | 若 x 為正或負無窮大則回傳 true |

```
from math import is_nan, is_inf, NaN, Inf
```

```
is_nan(NaN)    # true
is_inf(Inf)    # true
is_nan(1.0)    # false
```

[English](#)

## 网络 (TCP) 参考手册

### 类型

| 类型                       | 说明                  |
|--------------------------|---------------------|
| <code>TcpListener</code> | TCP 服务器套接字的不透明句柄    |
| <code>TcpStream</code>   | TCP 连接的不透明句柄        |
| <code>TlsStream</code>   | TLS 加密 TCP 连接的不透明句柄 |

这两种类型都是不透明指针。不能直接构造；使用 `bind()` 或 `connect()` 获取 `TcpListener`/`TcpStream`，使用 `tls_connect()` 获取 `TlsStream`。

### 函数 (来自 net)

这些函数需要显式导入：

```
from net import bind, listen, accept, connect, listener_port, shutdown, set_timeout, set_recv_timeout, s
```

| 函数                  | 签名                                                                                 | 说明                                                           |
|---------------------|------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <code>bind</code>   | <code>(host: str, port: int) -&gt; Result&lt;TcpListener, Error&gt;</code>         | 创建绑定到指定地址的 TCP 服务器套接字。失败时返回 <code>Err</code> 。使用端口 0 进行动态分配。 |
| <code>listen</code> | <code>(listener: TcpListener, backlog: int) -&gt; Result&lt;Unit, Error&gt;</code> | 开始监听传入连接。失败时返回 <code>Err</code> 。                            |
| <code>accept</code> | <code>(listener: TcpListener) -&gt; Result&lt;TcpStream, Error&gt;</code>          | 接受新连接。等待客户端连接最多 1 秒。超时或失败时返回 <code>Err</code> 。              |

| 函数               | 签名                                                 | 说明                                                   |
|------------------|----------------------------------------------------|------------------------------------------------------|
| connect          | (host: str, port: int) -> Result<TcpStream, Error> | 连接到远程 TCP 服务器。5 秒后超时。超时或失败时返回 Err。                   |
| listener_port    | (listener: TcpListener) -> int                     | 返回监听器绑定的实际端口号。在绑定到端口 0（操作系统分配的端口）时很有用。               |
| shutdown         | (listener: TcpListener) -> Unit                    | 通知监听器停止接受连接。使任何挂起的 accept() 在最多 1 秒内返回。              |
| tls_connect      | (host: str, port: int) -> Result<TlsStream, Error> | 使用 TLS 加密连接到远程服务器。根据系统 CA 证书包验证服务器证书。连接或握手失败时返回 Err。 |
| set_timeout      | (stream: TcpStream\ TlsStream, ms: int) -> Unit    | 设置接收和发送超时（毫秒）。                                       |
| set_recv_timeout | (stream: TcpStream\ TlsStream, ms: int) -> Unit    | 设置接收超时（毫秒）。如果在此时间内没有数据到达，recv() 返回 Err。              |
| set_send_timeout | (stream: TcpStream\ TlsStream, ms: int) -> Unit    | 设置发送超时（毫秒）。                                          |

## 内建重载函数

这些函数是内建的，可与 TCP 套接字类型配合使用。无需导入。

| 函数    | 签名                                                                   | 说明                                               |
|-------|----------------------------------------------------------------------|--------------------------------------------------|
| send  | (stream: TcpStream\ TlsStream, data: List<u8>) -> Result<int, Error> | 通过 TCP 或 TLS 连接发送字节。成功时返回 Ok 和已发送的字节数，失败时返回 Err。 |
| recv  | (stream: TcpStream\ TlsStream, max: int) -> Result<List<u8>, Error>  | 接收最多 max 个字节。连接关闭时返回 Ok 和空列表，出错时返回 Err。          |
| close | (handle: TcpStream\ TlsStream) -> Unit                               | 关闭 TCP 或 TLS 流。                                  |
| close | (handle: TcpListener) -> Unit                                        | 关闭 TCP 监听器。                                      |

## 使用示例

### 回显服务器

```

from net import bind, listen, accept, connect
from io import str_to_bytes, bytes_to_str

# 服务器
match bind("127.0.0.1", 8080):
    case Ok(server):
        match listen(server, 128):

```



```

        case Ok(_):
            match accept(server):
                case Ok(conn):
                    match recv(conn, 4096):
                        case Ok(data):
                            match send(conn, data):
                                case Ok(_):
                                    ...
                                case Err(e):
                                    print(e.message)
                            case Err(e):
                                print(e.message)
                        close(conn)
                    case Err(e):
                        ...
            case Err(e):
                print("listen failed")
        close(server)
    case Err(e):
        print("bind failed")

```

客户端

```

match connect("127.0.0.1", 8080):
    case Ok(conn):
        match send(conn, str_to_bytes("hello")):
            case Ok(_):
                ...
            case Err(e):
                print(e.message)
        match recv(conn, 4096):
            case Ok(resp):
                match bytes_to_str(resp):
                    case Ok(s):
                        print(s)
                    case Err(e):
                        print(e.message)
            case Err(e):
                print(e.message)
        close(conn)
    case Err(e):
        print("connect failed")

```

使用 `async fn` 的并发回显服务器

```

from net import bind, listen, accept, connect, listener_port
from io import str_to_bytes, bytes_to_str

```

```

async fn echo_server(server: TcpListener) -> str:
    match accept(server):
        case Ok(conn):
            match recv(conn, 4096):
                case Ok(data):
                    match send(conn, data):
                        case Ok(_):

```

```

        ...
        case Err(e):
            ...
            case Err(e):
                ...
                close(conn)
            case Err(e):
                ...
            close(server)
        return "done"

match bind("127.0.0.1", 0):
    case Ok(server):
        match listen(server, 1):
            case Ok(_):
                port = listener_port(server)
                t = echo_server(server)
                # ... 使用 port 的客户端代码 ...
                block_on(t)
            case Err(e):
                ...
        case Err(e):
            ...

```

## 超时配置

默认情况下，如果未设置自定义超时，`recv()` 使用 30 秒超时。使用 `set_timeout()`、`set_recv_timeout()` 或 `set_send_timeout()` 来覆盖默认值：

```

from net import connect, set_recv_timeout

match connect("127.0.0.1", 8080):
    case Ok(conn):
        set_recv_timeout(conn, 5000) # 5 秒超时
        match recv(conn, 4096):
            case Ok(data):
                ...
            case Err(e):
                print("timeout or error")
        close(conn)
    case Err(e):
        print("connect failed")

```

传入 0 可禁用超时（无限等待）。

## 错误处理

- 除 `close()` 外，所有 TCP 函数都返回 `Result<T, Error>` - 使用 `match` 配合 `Ok/Err` 来处理失败情况。
- 当对端关闭连接时，`recv()` 返回 `Ok` 和空的 `List<u8>`；实际错误（超时、套接字错误）时返回 `Err`。
- `close()` 关闭套接字并释放句柄。在关闭后使用句柄是未定义行为。

## 字节转换

TCP 操作使用 `List<u8>`。使用 `io` 中的 `str_to_bytes()` 和 `bytes_to_str()` 在字符串和字节列表之间进行转换。

## 运算符参考

### 优先级表

优先级数字越小越高（越先被求值）。

| 优先级  | 运算符                       | 说明            | 结合性 |
|------|---------------------------|---------------|-----|
| 0    | ? !!                      | 错误传播（后缀）      | 左   |
| 1    | ()                        | 分组            | —   |
| 2    | +x -x ~x                  | 一元正号、负号、位 NOT | 右   |
| 3    | **                        | 幂运算           | 右   |
| 3.5  | as                        | 类型转换          | 左   |
| 4    | * / % //                  | 乘法、除法、取余、整数除法 | 左   |
| 5    | + -                       | 加法、减法         | 左   |
| 6    | << >> >>>                 | 位移            | 左   |
| 7    | &                         | 位 AND         | 左   |
| 8    | ^                         | 位 XOR         | 左   |
| 9    | \                         | 位 OR          | 左   |
| 10   | == != < <= > >= in not in | 比较、成员测试       | 左   |
| 11   | not                       | 逻辑 NOT        | 右   |
| 12   | and                       | 逻辑 AND        | 左   |
| 13   | or                        | 逻辑 OR         | 左   |
| 13.5 | ??                        | 空值合并          | 左   |
| 14   | ?:                        | 三元条件          | 右   |

### 算术运算符

| 运算符 | 说明                   | 示例                               |
|-----|----------------------|----------------------------------|
| +   | 加法 / 字符串拼接           | 1 + 2 -> 3、"a" + "b" -> "ab"     |
| -   | 减法                   | 5 - 3 -> 2                       |
| *   | 乘法 / 字符串重复           | 4 * 3 -> 12、"ab" * 3 -> "ababab" |
| /   | 除法（始终为 float）        | 7 / 2 -> 3.5                     |
| //  | 向下取整除法（向 $-\infty$ ） | 7 // 2 -> 3、-7 // 2 -> -4        |
| %   | 取余                   | 7 % 3 -> 1                       |
| **  | 幂运算（始终为 float）       | 2 ** 10 -> 1024.0                |
| -x  | 一元负号                 | -5、-3.14                         |
| +x  | 一元正号                 | +5（不改变正负号）                       |

```
a = 10 // 3    # 3 (int)
b = 10 / 3     # 3.3333... (float)
c = 2 ** 8     # 256.0 (float)
s = "foo" + "bar" # "foobar"
```

### 比较运算符

全部返回 bool。

| 运算符 | 说明   |
|-----|------|
| ==  | 等于   |
| !=  | 不等于  |
| <   | 小于   |
| <=  | 小于等于 |
| >   | 大于   |
| >=  | 大于等于 |

- 可用于数值类型 (int / float) 和 bool。
- str 之间以字典序 (字节顺序) 比较。
- 记录类型支持 == 和 !=，自动生成逐字段比较 (参见[结构体参考](#))。
- in 运算符用于集合、列表、映射的成员测试 (x in s)。
- not in 运算符为 in 的否定 (x not in s)。
- 对于映射，in 检查键是否存在。

```
x = 3 < 5          # true
y = "abc" < "abd"  # true (字典序)
s = {1, 2, 3}
z = 2 in s         # true
w = 4 not in s     # true
xs = [1, 2, 3]
a = 2 in xs        # true (列表线性搜索)
m = {"a": 1}
b = "a" in m       # true (映射键搜索)
```

## 逻辑运算符

| 运算符 | 说明     | 类型                  |
|-----|--------|---------------------|
| and | 逻辑 AND | bool x bool -> bool |
| or  | 逻辑 OR  | bool x bool -> bool |
| not | 逻辑 NOT | bool -> bool        |

```
a = true and false  # false
b = true or false   # true
c = not true        # false
```

## 位运算符

仅可用于 int 类型。对 float 或 bool 使用会产生编译错误。

| 运算符 | 说明         | 示例                              |
|-----|------------|---------------------------------|
| &   | 位 AND      | 0b1100 & 0b1010 -> 0b1000       |
| \   | 位 OR       | 0b1100 \  0b1010 -> 0b1110      |
| ^   | 位 XOR      | 0b1100 ^ 0b1010 -> 0b0110       |
| ~   | 位 NOT (一元) | ~0 -> -1                        |
| <<  | 左移         | 1 << 4 -> 16                    |
| >>  | 算术右移       | 16 >> 2 -> 4                    |
| >>> | 逻辑右移       | -1 >>> 1 -> 9223372036854775807 |

```
flags = 0b0001 | 0b0010  # 3
masked = flags & 0b0011   # 3
shifted = 1 << 8          # 256
```

## 错误传播运算符 (? / !!)

后缀 ? 运算符用于解包 Result 值。如果值为 Ok(v)，则求值为 v。如果值为 Err(e)，则外层函数立即返回 Err(e)。

!! 运算符是 ? 的别名，语义完全相同。两者可以互换使用。

外层函数必须具有 Result 返回类型。

```
fn safe_divide(a: int, b: int) -> Result<int, Error>:
    if b == 0:
        return Err(Error("division by zero"))
    return Ok(a // b)

fn compute(a: int, b: int, c: int) -> Result<int, Error>:
    x = safe_divide(a, b)?      # 若 b == 0 则提前返回 Err
    y = safe_divide(x, c)!!
    return Ok(y + 1)
```

这等同于以下 match 模式，但更加简洁：

```
fn compute(a: int, b: int, c: int) -> Result<int, Error>:
    match safe_divide(a, b):
        case Ok(x):
            match safe_divide(x, c):
                case Ok(y):
                    return Ok(y + 1)
                case Err(e):
                    return Err(e)
        case Err(e):
            return Err(e)
```

---

## 三元条件运算符

```
x = condition ? true_value : false_value
```

对 condition 进行求值。若为真，返回 true\_value；否则返回 false\_value。两个分支必须具有相同的类型。右结合，因此嵌套三元运算符从右向左结合。

```
x = 3 > 2 ? 10 : 20      # 10
s = false ? "yes" : "no" # "no"

# 嵌套（右结合）
y = true ? (false ? 1 : 2) : 3  # 2
```

---

## 范围运算符

.. 运算符创建包含两端的整数范围。

```
xs = 1 .. 5      # [1, 2, 3, 4, 5]

for i in 1 .. 3:
    print(i)      # 1 2 3
```

结果是包含从左操作数到右操作数（两端皆含）的所有整数的 List<int>。

---

## 空值合并运算符 (??)

```
x = option_val ?? default_val
```

如果 `option_val` 为 `Some(v)`，则返回 `v`。否则返回 `default_val`。右操作数必须与 `Option` 的内部类型相同。

```
a: int? = Some(10)
b: int? = none

print(a ?? 0)    # 10
print(b ?? 0)    # 0
```

---

## 复合赋值运算符

更新变量的简写形式。`x op= y` 等价于 `x = x op y`。

| 运算符                        | 等价表达式                         |
|----------------------------|-------------------------------|
| <code>x += y</code>        | <code>x = x + y</code>        |
| <code>x -= y</code>        | <code>x = x - y</code>        |
| <code>x *= y</code>        | <code>x = x * y</code>        |
| <code>x /= y</code>        | <code>x = x / y</code>        |
| <code>x %= y</code>        | <code>x = x % y</code>        |
| <code>x //= y</code>       | <code>x = x // y</code>       |
| <code>x **= y</code>       | <code>x = x ** y</code>       |
| <code>x &amp;= y</code>    | <code>x = x &amp; y</code>    |
| <code>x \ = y</code>       | <code>x = x \  y</code>       |
| <code>x ^= y</code>        | <code>x = x ^ y</code>        |
| <code>x &lt;&lt;= y</code> | <code>x = x &lt;&lt; y</code> |
| <code>x &gt;&gt;= y</code> | <code>x = x &gt;&gt; y</code> |

```
x = 10
x += 5    # x = 15
x -= 3    # x = 12
x *= 2    # x = 24
x /= 3    # x = 8
x //= 3   # x = 2
x &= 6    # x = 0
```

## 递增 / 递减运算符

仅后缀、仅语句级别的运算符，用于将变量增减 1。内部分别被转换为 `x = x + 1` 和 `x = x - 1`。

| 运算符              | 等价表达式                  |
|------------------|------------------------|
| <code>x++</code> | <code>x = x + 1</code> |
| <code>x--</code> | <code>x = x - 1</code> |

```
count = 0
count++    # count = 1
count++    # count = 2
count--    # count = 1

f = 1.5
f++        # f = 2.5 (int 1 会提升为 float)
```

注意：++ / -- 只能作为语句使用，不能在表达式中使用。@const 变量不能使用递增/递减（不可变性会被强制执行）。

运算的类型规则

| 运算                  | 左操作数类型                   | 右操作数类型                 | 结果类型  |
|---------------------|--------------------------|------------------------|-------|
| + - *               | int                      | int                    | int   |
| + - *               | float                    | int / float            | float |
| + - *               | int                      | float                  | float |
| /                   | 任意数值                     | 任意数值                   | float |
| //                  | int                      | int                    | int   |
| //                  | float 或 int（其中一方为 float） | -                      | float |
| **                  | 任意数值                     | 任意数值                   | float |
| %                   | int                      | int                    | int   |
| %                   | float 或 int（其中一方为 float） | -                      | float |
| +                   | str                      | str                    | str   |
| == != < <= > >=     | 数值 / bool / str          | 同类型                    | bool  |
| *                   | str                      | int                    | str   |
| in                  | 任意                       | Set / List / Map<K, V> | bool  |
| not in              | 任意                       | Set / List / Map<K, V> | bool  |
| & \   ^ ~ << >> >>> | int                      | int                    | int   |
| and or not          | bool                     | bool                   | bool  |

运算符重载

可以为用户定义类型定义运算符的行为。

语法

```
# 二元运算符（2 个参数）
fn operator+(a: MyType, b: MyType) -> MyType:
    ...

# 一元运算符（1 个参数）
fn operator-(a: MyType) -> MyType:
    ...
```

可重载的运算符一览

| 类别      | 运算符                                       |
|---------|-------------------------------------------|
| 算术（二元）  | + - * / % ** //                           |
| 比较（二元）  | == != < <= > >=                           |
| 位运算（二元） | & \   ^ << >> >>>                         |
| 逻辑（二元）  | and or                                    |
| 成员测试    | in                                        |
| 下标      | []（读取）、[]=（写入）                            |
| 调用      | ()                                        |
| 转换      | as                                        |
| 一元      | - ~ not                                   |
| 复合赋值    | += -= *= /= %= //= **= &= \  = ^= <<= >>= |

## 返回类型约束

比较运算符和逻辑运算符必须返回 bool：

| 类别   | 运算符                                                                                                       | 必需返回类型   |
|------|-----------------------------------------------------------------------------------------------------------|----------|
| 比较   | <code>==</code> <code>!=</code> <code>&lt;</code> <code>&lt;=</code> <code>&gt;</code> <code>&gt;=</code> | bool     |
| 逻辑   | <code>and</code> <code>or</code> <code>not</code>                                                         | bool     |
| 成员测试 | <code>in</code>                                                                                           | bool     |
| 转换   | <code>as</code>                                                                                           | 必需（目标类型） |

*# OK*

```
fn operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y
```

*# Error: comparison operator '=' must return 'bool', but returns 'int'*

```
fn operator==(a: Vec2, b: Vec2) -> int:
    return 42
```

算术运算符和位运算符没有返回类型约束。

## 二元 / 一元的区别

依参数个数区分。

*# 二元 -*

```
fn operator-(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x - b.x, a.y - b.y)
```

*# 一元 -*

```
fn operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)
```

## 复合赋值运算符重载

复合赋值运算符（`+=`、`-=` 等）可以独立重载。这使得大数据结构可以进行就地优化。

```
record Matrix:
    data: List
    rows: int
    cols: int

fn operator+=(a: Matrix, b: Matrix) -> Matrix:
    for i in range(len(a.data)):
        a.data[i] = a.data[i] + b.data[i]
    return a
```

解析优先级 当 `x += y` 被求值时：

1. 如果定义了 `operator+=` → 直接调用
2. 如果未定义 `operator+=` 但定义了 `operator+` → 回退到 `x = x + y`
3. 如果都未定义（非内置类型）→ 编译错误

```
record Vec2:
    x: float
    y: float
```

```
fn operator+=(a: Vec2, b: Vec2) -> Vec2:
```



```
    return Vec2(a.x + b.x, a.y + b.y)
```

```
v = Vec2(1.0, 2.0)
v += Vec2(3.0, 4.0) # 直接调用 operator+=
# v.x == 4.0, v.y == 6.0
```

复合赋值运算符要求恰好 2 个参数，没有返回类型约束。

### 下标运算符重载

[] (读取) 和 []= (写入) 运算符可以为用户定义类型启用自定义下标行为。支持多索引访问 (例如 `m[row, col]`)。

```
record Grid:
    a: int
    b: int
    c: int
    d: int

# 读取: 需要 2+ 个参数 (对象 + 索引)
fn operator[](g: Grid, row: int, col: int) -> int:
    if row == 0 and col == 0:
        return g.a
    if row == 0 and col == 1:
        return g.b
    if row == 1 and col == 0:
        return g.c
    return g.d

# 写入: 需要 3+ 个参数 (对象 + 索引 + 值)
fn operator[]=(g: Grid, row: int, col: int, value: int):
    ...

g = Grid(1, 2, 3, 4)
print(g[0, 1])      # 2
g[1, 0] = 99
```

优先尝试用户定义的下标运算符；如果没有匹配，则回退到内置下标行为 (用于列表、映射和数组)。

### 成员测试运算符重载

`in` 运算符可以被重载以定义自定义成员测试。必须返回 `bool`。

```
record Range:
    lo: int
    hi: int

fn operator in(value: int, r: Range) -> bool:
    return value >= r.lo and value < r.hi

r = Range(1, 10)
print(5 in r)      # true
print(15 not in r) # true
```

优先尝试用户定义的 `in` 运算符；如果没有匹配，则回退到内置行为 (用于集合、映射和列表)。定义了 `in` 后，`not in` 自动支持。

## 调用运算符重载

( ) 运算符使记录可以作为可调用对象使用。至少需要 2 个参数 (对象+ 参数)。

```
record Adder:
  base: int

fn operator()(a: Adder, x: int) -> int:
  return a.base + x
```

```
add5 = Adder(5)
print(add5(10))    # 15
```

当持有记录值的变量被像函数一样调用时，编译器首先尝试 `operator()` 重载。如果没有匹配，其他调用解析策略 (函数、构造函数、`lambda`) 优先。

## 转换运算符重载

`as` 运算符可以被重载以定义自定义类型转换。接受恰好 1 个参数 (源值)，必须指定返回类型 (目标类型)。分派按源类型和返回类型匹配。

```
record Celsius:
  value: int

record Fahrenheit:
  value: int

fn operator as(c: Celsius) -> Fahrenheit:
  return Fahrenheit(c.value * 9 // 5 + 32)

c = Celsius(100)
f = c as Fahrenheit    # Fahrenheit(212)
```

优先尝试用户定义的 `as` 运算符；如果没有匹配，则回退到内置转换 (`int`、`float`、`bool`、`str` 等)。

[English](#) | [日本語](#) | [繁體中文](#)

## 包参考

### 概述

Ry 使用包系统来组织代码。包可以是单个 `.ry` 文件，或是包含多个 `.ry` 文件的目录。使用 `from` 语句导入包。`std` 包 (标准库) 会自动导入到每个程序中。

---

### 导入语法

#### 导入全部定义

```
from math
```

导入包内的所有函数与类型。

#### 选择性导入

```
from math import sqrt
```

仅导入指定的定义。

## 多重选择性导入

```
from math import sqrt, PI
```

以逗号分隔选择性导入多个定义。

## 包解析

以点号分隔的包名称按以下方式解析：

| 导入语句            | 解析结果                             |
|-----------------|----------------------------------|
| from math       | math/ 目录（包）或 math.py 文件          |
| from utils.math | utils/math/ 目录或 utils/math.py 文件 |
| from str        | str/ 目录或 str.py 文件               |

## 解析顺序

对于每个搜索路径：1. 目录 ({path}/) — 若存在，加载目录内的所有 .py 文件（包）2. 文件 ({path}.py) — 单个文件（向后兼容）

## 目录包

当包解析为目录时：- 目录内的所有 .py 文件会自动加载 - 以\_开头的文件会被排除 - 不需要特殊的入口文件（如 \_\_init\_\_.py）- 目录内文件中定义的所有函数与类型都会被导出

## 私有符号

名称以 \_（下划线）开头的定义为包内部的私有符号，无法被导入：

- 通配导入（from pkg）会自动排除 \_ 前缀的符号
- 具名导入（from pkg import \_helper）会产生编译错误

```
# mylib/internal.py
fn _helper() -> int:      # 私有 — 无法导入
    return 42
fn public_api() -> int:   # 公开 — 可导入
    return _helper()

mypackage/
  calc.py      # fn add(), fn sub()
  string.py    # fn concat()

from mypackage      # 导入 add, sub, concat
from mypackage import add  # 仅导入 add
```

## 标准库 (std)

std 包会自动导入到每个程序中。提供的功能：- 内建函数（print, length, range 等）- 字符串函数（contains, find, replace 等）- 类型转换函数（to\_int, to\_float, to\_str）- 集合函数（map, filter, sort 等）

## 子包

以下子包需要明确导入：

| 包                 | 说明               |
|-------------------|------------------|
| <code>math</code> | 数学常量与函数          |
| <code>io</code>   | 文件 I/O、标准输入、字节转换 |

```
from math import sqrt, PI, sin
```

也可以直接从标准库的包中明确导入特定定义：

```
from str import contains
```

## RY\_HOME

标准库安装于 `$RY_HOME/lib/std/`。RY\_HOME 的默认值为 `~/.ry`。

```
export RY_HOME="$HOME/.ry" # default
```

## RY\_ENV

RY\_ENV 环境变量控制运行时环境模式。也可以使用 `--env=<value>` CLI 标志。

| 值             | 别名          | .env 加载                                        | lib 搜索                                                                                       |
|---------------|-------------|------------------------------------------------|----------------------------------------------------------------------------------------------|
| prod          | production  | 禁用                                             | 仓库构建的项目覆盖 →<br><code>\$RY_HOME/lib</code> → <code>exe/./lib</code><br>→ <code>exe/lib</code> |
| dev           | development | <code>.env.dev</code> → <code>.env</code>      | 与 prod 相同                                                                                    |
| test          | —           | <code>.env.test</code> → <code>.env</code>     | 与 prod 相同                                                                                    |
| staging       | —           | <code>.env.staging</code> → <code>.env</code>  | 与 prod 相同                                                                                    |
| internal      | —           | <code>.env.internal</code> → <code>.env</code> | 仓库构建的项目覆盖 →<br><code>exe/./lib</code> → <code>exe/lib</code> (跳过<br><code>\$RY_HOME</code> ) |
| (未设置)<br>(默认) | —           | 仅 <code>.env</code>                            | 与 prod 相同                                                                                    |

别名会自动解析为规范形式。例如 `RY_ENV=production` 会被规范化为 `prod`。

在 `prod` 模式下，出于安全考虑不会加载 `.env` 文件——生产环境的机密信息应通过基础设施级别的环境变量管理（CI/CD、密钥管理等）。

其他模式下，先加载 `.env.<env>`（若存在），再加载 `.env`。由于不会覆盖已存在的环境变量，环境专属的值会优先生效。

# 简写形式（推荐）

```
RY_ENV=dev ./build/ry app.ry
```

# 完整名称（向后兼容）

```
RY_ENV=development ./build/ry app.ry
```

# CLI 标志

```
./build/ry --env=dev test
```

# prod 模式：不加载 `.env`

```
RY_ENV=prod ./build/ry app.ry
```

# 开发 Ry 自身时的额外隔离

```
RY_ENV=internal ./build/ry test
```

当在 Ry 源码树中构建 ry 可执行文件时，它可以使用项目 `package.toml` 中的仓库本地 `stdlib` 覆盖。这使得仓库构建与签出的 `lib/std` 保持一致，即使 `~/.ry/lib/std` 版本较旧。已安装的 ry 二进制文件会忽略该覆盖，继续使用 `$RY_HOME/lib/std`。

---

## 搜索路径优先级

1. 导入来源文件所在的目录
2. 使用仓库构建的 ry 时，来自当前 Ry 签出的仓库本地 `stdlib` 覆盖
3. `$RY_HOME/lib`（标准库位置）
4. 可执行文件相对的 `lib/` 目录
5. `RY_PATH` 环境变量中包含的路径（以冒号分隔）

---

## RY\_PATH 环境变量

在 `RY_PATH` 中以冒号分隔指定目录，即可添加到包搜索路径。

```
export RY_PATH="/usr/local/ry/lib:/home/user/ry-packages"
```

---

## 约束

| 约束     | 详细            |
|--------|---------------|
| 可使用的位罝 | 仅限顶层（函数或块内不可） |
| 重复导入   | 自动跳过（不会产生错误）  |
| 循环导入   | 编译错误          |

```
# 错误示例：在块内导入
fn main():
    from math    # Error: imports only allowed at top level
```

```
# OK: 多次导入相同包不会产生错误
from math
from math    # Skipped
```

---

## 创建包文件

### 单文件包

```
# calc.ry
fn add(a: int, b: int) -> int:
    return a + b

fn sub(a: int, b: int) -> int:
    return a - b

# main.ry
from calc import add, sub

print(add(1, 2))    # 3
print(sub(5, 3))    # 2
```

## 目录包

```
mylib/  
  calc.ry  
  string.ry  
  
# main.ry  
from mylib import add, concat
```

[English](#) | [日本語](#) | [简体中文](#)

## 项目管理

### CLI 概述

```
ry <file.ry> [args...]      # 运行 Ry 脚本  
echo '<code>' | ry          # 从标准输入运行代码  
ry test [options] [<file> | <dir>] # 运行测试  
ry init                    # 初始化项目  
ry new <project-name>      # 创建新项目  
ry fmt [options] [<file> | <dir>] # 格式化源文件  
ry self-update [options]   # 更新 ry 本身
```

### 全局选项

| 选项            | 说明                                                     |
|---------------|--------------------------------------------------------|
| -h, --help    | 显示帮助                                                   |
| -v, --version | 显示版本                                                   |
| --env=<env>   | 设置环境 (production development internal)。覆盖 RY_ENV 环境变量。 |

### 标准输入执行

当没有给出文件参数且标准输入不是终端时，ry 从标准输入读取源代码并执行：

```
echo 'print("hello")' | ry
```

### ry init - 项目初始化

将当前目录初始化为 Ry 项目。

```
ry init
```

### 生成的文件与目录

```
my-project/  
  package.toml      # 项目配置文件  
  src/  
    main.ry         # 入口点（示例代码）
```

### 行为

1. 若 package.toml 已存在则错误退出
2. 创建 src/ 目录（若不存在）

3. 生成 `package.toml` (`name` 为当前目录名称)
4. 生成 `src/main.ry` (若已存在则跳过)

---

## ry new - 创建新项目

创建新目录并将其初始化为 Ry 项目。

```
ry new my-project
```

生成的文件与目录

```
my-project/  
  package.toml      # 项目配置文件  
  src/  
    main.ry         # 入口点 (示例代码)
```

行为

1. 若未指定项目名称则错误退出
2. 若同名目录已存在则错误退出
3. 创建 `<project-name>/` 目录
4. 在其中创建 `src/` 目录
5. 生成 `package.toml` (`name` 为指定的项目名称)
6. 生成 `src/main.ry`

---

## ry fmt - 代码格式化工具

以一致的 2 空格缩进和规范风格格式化 `.ry` 源代码文件。

```
ry fmt          # 格式化项目中所有 .ry 文件 (需要 package.toml)  
ry fmt src/main.ry # 格式化单个文件  
ry fmt src/      # 递归格式化目录中所有 .ry 文件  
ry fmt --check   # 检查文件是否已格式化 (未格式化则 exit 1)  
ry fmt --check src/ # 检查指定目录
```

格式化规则

- 每个代码块层级使用 2 空格缩进
- 二元运算符前后加空格 (`a + b`，而非 `a+b`)
- 逗号后加空格 (`f(a, b)`，而非 `f(a,b)`)
- 顶层定义之间 (函数、记录、枚举) 加空行
- 注释会被保留

行为

1. 读取源代码文件，解析为 AST，并以规范格式重新输出
2. 将格式化结果写回文件 (就地修改)
3. 使用 `--check` 时仅报告未格式化的文件，若存在则以代码 1 退出 (适用于 CI)
4. 递归格式化时跳过 `.git/`、`build/`、`node_modules/` 目录

注意事项

- 不需要 LLVM 初始化 (快速启动)
- 复合赋值运算符 (`+=`、`-=` 等) 因解析器会进行脱糖，格式化后会变成展开形式 (`x = x + expr`)

- 十六进制 (0xFF) 和二进制 (0b1010) 数字字面量会被转换为十进制表示

## ry test - 运行测试

发现并运行测试文件 (\*.test.ry)。完整的测试语法文档请参阅[测试](#)。

```
ry test           # 自动发现并运行所有 *.test.ry 文件
ry test tests/spec # 运行目录下所有测试
ry test test_file.ry # 运行指定的测试文件
ry test -p        # 并行运行测试
ry test -w        # 监视模式：文件变更时重新运行
ry test --coverage # 收集行覆盖率信息
```

### 选项

| 选项                | 说明        |
|-------------------|-----------|
| -p, --parallel    | 并行运行测试    |
| -w, --watch       | 监视变更并重新运行 |
| --coverage, --cov | 收集覆盖率信息   |
| -h, --help        | 显示帮助      |

### 行为

1. 无参数时，搜索 package.toml 以找到项目根目录，并递归发现\*.test.ry 文件（跳过.git、build、node\_modules）
2. 所有测试通过时退出码为 0，有失败时为 1
3. --coverage 与 --parallel 一起使用时会回退到顺序执行

## ry self-update - 自我更新

将 ry 本身更新至最新版本。从 GitHub Releases 下载二进制文件并替换当前的可执行文件。

```
ry self-update      # 更新至最新稳定版
ry self-update --nightly # 更新至最新 nightly 预发布版
ry self-update v0.0.1 # 更新至指定版本
```

### 行为

1. 显示当前版本
2. 根据参数解析更新目标版本
  - 无参数：GitHub 的最新稳定发行版 (/releases/latest)
  - --nightly：最新的预发布版
  - 指定版本：指定标签的发行版
3. 若与当前版本相同，则以 "Already up to date." 退出
4. 下载二进制文件并替换当前的可执行文件

### 注意事项

- 执行需要 curl 和 tar 命令
- 若因权限不足导致二进制文件替换失败，会显示建议使用 sudo 的消息（不会自动执行 sudo）
- 下载会先在临时目录中进行；但若跨文件系统的 cp 回退操作被中断，目标二进制文件可能处于不完整状态



## package.toml 配置文件

以 TOML 格式描述项目的元数据与路径设置。

```
[project]
name = "my-project"
version = "0.1.0"
entry = "src/main.ry"

[paths]
src = "src"
```

### [project] 部分

| 键       | 说明              |
|---------|-----------------|
| name    | 项目名称（初始化时为目录名称） |
| version | 版本字符串           |
| entry   | 作为入口点的源代码文件     |

### [paths] 部分

| 键   | 说明    |
|-----|-------|
| src | 源代码目录 |

## TOML 子集规范

package.toml 支持以下 TOML 子集。

- 部分头：[section]
- 键值对：key = "value"（仅字符串值）
- 注释：从 # 到行尾
- 空行会被忽略

[English](#) | [日本語](#) | [繁體中文](#)

## 正規表達式函式參考手冊

正規表達式函式的一覽表。所有函式皆支援 UFCS 記法。模式字串使用標準的正規表達式語法。

注意：Phase 1 中模式以普通字串傳遞。專用的正規表達式字面量語法可能在未來版本中加入。

### 函式一覽

| 函式             | 簽名                      | 說明                      |
|----------------|-------------------------|-------------------------|
| regex_match    | (str, str) -> bool      | 文字全體是否匹配模式              |
| regex_search   | (str, str) -> int       | 傳回第一個匹配的起始位置（找不到時傳回 -1） |
| regex_replace  | (str, str, str) -> str  | 將匹配的部分替換為替換字串           |
| regex_split    | (str, str) -> List<str> | 以模式匹配進行分割               |
| regex_find_all | (str, str) -> List<str> | 傳回所有不重疊的匹配結果            |

## 支援的模式語法

| 語法     | 說明                | 範例                         |
|--------|-------------------|----------------------------|
| abc    | 字面字元              | "hello"                    |
| .      | 任意一個字元（換行除外）      | "a.c" 匹配 "abc"、"aXc"       |
|        | 選擇                | "cat dog" 匹配 "cat" 或 "dog" |
| *      | 零次或多次重複           | "a*" 匹配 ""、"a"、"aaa"       |
| +      | 一次或多次重複           | "a+" 匹配 "a"、"aaa"          |
| ?      | 零次或一次             | "a?" 匹配 "" 或 "a"           |
| {n}    | 恰好 n 次            | "a{3}" 匹配 "aaa"            |
| {n,m}  | n 到 m 次           | "a{2,4}" 匹配 "aa" 到 "aaaa"  |
| {n,}   | 至少 n 次            | "a{2,}" 匹配 "aa"、"aaa"、...  |
| *?     | 零次或多次（非貪婪）        | ".*?" 匹配最短                 |
| +?     | 一次或多次（非貪婪）        | ".+?" 匹配最短                 |
| ??     | 零次或一次（非貪婪）        | "a??" 優先零次                 |
| {n,m}? | 範圍（非貪婪）           | "a{2,4}?" 優先 n 次           |
| (...)  | 群組                | "(ab)+" 匹配 "abab"          |
| [abc]  | 字元類別              | "[aeiou]" 匹配母音             |
| [a-z]  | 字元範圍              | "[a-z]+" 匹配小寫單字            |
| [^...] | 否定字元類別            | "[^0-9]" 匹配非數字             |
| ^      | 字串開頭錨點            | "^hello"                   |
| \$     | 字串結尾錨點            | "world\$"                  |
| \d     | 數字 [0-9]          | "\d+" 匹配數字                 |
| \D     | 非數字 [^0-9]        |                            |
| \w     | 單字字元 [a-zA-Z0-9_] | "\w+" 匹配識別符                |
| \W     | 非單字字元             |                            |
| \s     | 空白字元              | "\s+" 匹配空格與 Tab            |
| \S     | 非空白字元             |                            |
| \b     | 單字邊界              | "\bword\b" 匹配完整單字          |
| \B     | 非單字邊界             | "\Bword" 匹配單字內部            |
| (?i)   | 忽略大小寫旗標           | "(?i)hello" 匹配 "HELLO"     |
| \.     | 跳脫特殊字元            | "\." 匹配字面的 .               |

## 使用範例

### regex\_match

```
print(regex_match("[a-z]+", "hello")) # true
print(regex_match("[0-9]+", "hello")) # false
print(regex_match("[a-zA-Z_]\w*", "my_var")) # true
```

### regex\_search

```
pos = regex_search("[0-9]+", "abc123def")
print(pos) # 3
```

### regex\_replace

```
s = regex_replace("[0-9]+", "a1b2c3", "X")
print(s) # aXbXcX
```

### regex\_split

```
parts = regex_split("\\s+", "hello world foo")
print(length(parts)) # 3
```

```
print(parts[0])    # hello
```

#### regex\_find\_all

```
matches = regex_find_all("[0-9]+", "a1b23c456")
print(length(matches)) # 3
print(matches[0])      # 1
print(matches[1])      # 23
```

#### 範圍量詞

```
print(regex_match("\\d{3}-\\d{4}", "123-4567")) # true
print(regex_match("a{2,4}", "aaa"))             # true
print(regex_match("(ab){2,}", "ababab"))         # true
```

#### 非貪婪（最短）匹配

```
# 貪婪：匹配最長
g = regex_replace("\\.*\\\"", "\\\"a\\\" and \\\"b\\\"", "X")
print(g) # X
```

```
# 非貪婪：匹配最短
l = regex_replace("\\.*?\\\"", "\\\"a\\\" and \\\"b\\\"", "X")
print(l) # X and X
```

#### 取得個別 HTML 標籤

```
tags = regex_find_all("<.*?>", "<a> <bb> <ccc>")
print(length(tags)) # 3
```

注意：非貪婪匹配控制整體匹配長度。沒有使用括號分組時，greedy/lazy 混合模式可能與 PCRE 引擎行為不同。

#### 單字邊界

```
# 匹配完整單字
pos = regex_search("\\bworld\\b", "hello world")
print(pos) # 6

# 取得所有單字
words = regex_find_all("\\b\\w+\\b", "hello world foo")
print(length(words)) # 3

# \B 匹配非邊界（單字內部）
pos2 = regex_search("\\Bworld", "helloworld")
print(pos2) # 5
```

#### 忽略大小寫匹配

```
# (?i) 放在模式開頭即可忽略大小寫
print(regex_match("(?i)hello", "HELLO")) # true
print(regex_match("(?i)hello", "Hello")) # true

# 字元類別也適用
print(regex_match("(?i)[a-z]+", "ABC")) # true
```

```
# replace 和 find_all 也可使用
```

```
s = regex_replace("(?i)hello", "Hello HELLO hello", "X")
print(s)  # X X X
```

注意：(?i) 必須出現在模式的開頭，並適用於整個模式。不支援部分忽略大小寫（例如 (?i:sub)pattern）。

## UFCS 記法

```
# pattern.function(text, ...)
m = "[a-z]+".regex_match("hello")
print(m)  # true
```

[English](#) | [日本語](#) | [简体中文](#)

## 结构体参考

### 概述

结构体是栈上的值类型。使用 `record` 关键字定义。结构体可以使用 `invariant` 子句实现契约式设计。参阅[契约式设计](#)。

命名约定：结构体名称必须使用 PascalCase（如 `Point`、`Rectangle`）。字段名称必须使用 `snake_case`。编译器会强制执行这些约定。

---

### 定义语法

```
record TypeName:
  field_name: type
  field_name: type
```

### 示例

```
record Point:
  x: int
  y: int

record Rectangle:
  width: float
  height: float
```

---

### 构造函数

按照字段定义顺序传递参数。不支持命名参数。

```
p = Point(10, 20)
r = Rectangle(3.0, 4.5)
```

---

### 字段访问

使用点号记法读取字段。

```
p = Point(10, 20)
print(p.x)    # 10
print(p.y)    # 20
```

## 字段赋值

| 变量声明          | 字段赋值 |
|---------------|------|
| 可变 (无 @const) | 可以   |
| @const        | 编译错误 |

```
p = Point(10, 20)
p.x = 100    # OK: 可变变量

@const
q = Point(10, 20)
q.x = 100    # 错误: @const 变量的字段不可变更
```

## 作为函数参数与返回值使用

```
fn distance(p: Point) -> float:
    return (p.x * p.x + p.y * p.y) as float

fn make_point(x: int, y: int) -> Point:
    return Point(x, y)
```

## 嵌套结构体

可以将结构体作为另一个结构体的字段使用。

```
record Point:
    x: int
    y: int

record Circle:
    center: Point
    radius: float

c = Circle(Point(0, 0), 1.0)
print(c.center.x)    # 0
```

## 比较 (== / !=)

记录类型自动支持 == 和 != 运算符。比较是逐字段进行的（结构性相等）。

```
record Point:
    x: int
    y: int
```

```
p1 = Point(10, 20)
p2 = Point(10, 20)
p3 = Point(30, 40)
```

```
print(p1 == p2) # true
print(p1 != p3) # true
```

- 所有字段按顺序比较。对于 ==，所有字段必须相等。对于 !=，至少一个字段必须不同。
- 嵌套记录会递归比较。
- 如果提供了用户定义的 operator== 或 operator!=，则优先于自动生成的版本。

## 记录子类型（继承）

记录支持使用 < 语法的单继承。子记录继承父记录的所有字段。

语法

```
record ChildName < ParentName:
    child_field: type
```

示例

```
record HttpError < Error:
    status: int
    url: str
```

字段继承

- 子记录在其布局的开头继承所有父字段。
- 构造函数先接受父字段，然后是子特有的字段。

```
err = HttpError("not found", 404, 404, "/api")
print(err.message) # "not found" (从 Error 继承)
print(err.status)  # 404 (自有字段)
```

子类型强制转换

子值可以传递给期望父类型的地方。子值会被自动切片以提取父前缀字段（值类型切片）。

```
fn handle(e: Error) -> str:
    return e.message
```

```
err = HttpError("fail", 500, 500, "/api")
handle(err) # OK — HttpError 强制转换为 Error
```

深层继承

记录可以形成继承链。每一层继承所有祖先字段。

```
record DetailedHttpError < HttpError:
    detail: str

# 构造函数: Error 字段 + HttpError 字段 + 自有字段
derr = DetailedHttpError("fail", 500, 500, "/x", "server crash")
handle(derr) # OK — 强制转换为 Error (祖父类型)
```

## 规则

| 规则                                       | 详细                                                                  |
|------------------------------------------|---------------------------------------------------------------------|
| 仅支持单继承                                   | <code>record A &lt; B:</code> — 只能有一个父类                             |
| 深层继承                                     | <code>record C &lt; B:</code> 其中 <code>record B &lt; A:</code> — 允许 |
| 名称冲突                                     | 子字段与父字段同名 → 编译错误                                                    |
| 自动 <code>==</code> / <code>to_str</code> | 包含所有继承字段                                                            |
| 不变量继承                                    | 构造或修改子记录时检查父的 <code>invariant</code> : 子句                           |
| 子类型强制转换                                  | 适用于: 函数参数、返回值、 <code>Err()</code> 、字段赋值、 <code>?</code> 运算符         |
| 泛型边界                                     | <code>&lt;T: RecordName&gt;</code> 将类型参数约束为记录的子类型                   |
| <code>@const</code>                      | 适用于所有字段 (包括继承的字段)                                                   |

## 约束与错误

| 约束                          | 详细                             |
|-----------------------------|--------------------------------|
| 相同字段名重复                     | 编译错误                           |
| <code>@const</code> 变量的字段赋值 | 编译错误                           |
| 直接将结构体传给 <code>print</code> | 编译错误 ( <code>print</code> 不支持) |

# 错误示例: 相同字段名重复

```
record Bad:
    x: int
    x: int # 错误
```

# 错误示例: 将结构体传给 `print`

```
p = Point(1, 2)
print(p) # 错误
```

## 枚举类型 (enum)

### 概述

枚举类型是具名常量的集合。默认以 i64 整数 (0, 1, 2, ...) 的连续编号表示。也可以指定显式的整数值。

### 定义语法

```
enum TypeName:
    VariantName
    VariantName
    ...
```

### 示例

```
enum Color:
    Red
    Green
    Blue
```

## 变体访问

使用 `::` 运算符访问变体。

```
c = Color::Red
print(c)    # Red
```

## 比较

enum 值为整数，因此可以直接使用 `==` / `!=` 进行比较。

```
print(Color::Red == Color::Red)    # true
print(Color::Red != Color::Green)  # true
```

## 在 if 语句中使用

```
c = Color::Green
if c == Color::Red:
    print("red")
elif c == Color::Green:
    print("green")
else:
    print("blue")
```

## 函数参数

类型名称使用 enum 名称。

```
fn is_red(c: Color) -> bool:
    return c == Color::Red

print(is_red(Color::Red))    # true
print(is_red(Color::Green))  # false
```

## print

使用 `print()` 会输出变体名称。

```
c = Color::Blue
print(c)    # Blue
```

## 显式值指定

simple enum 的变体可以指定显式的整数值。适用于 HTTP 状态码或位掩码模式等用途。

```
enum HttpStatus:
    Ok = 200
    NotFound = 404
    InternalError = 500

s = HttpStatus::NotFound
print(s)                    # NotFound
print(s == HttpStatus::NotFound)  # true

enum Permission:
    Read = 1
    Write = 2
    Execute = 4
```



规则：- 仅支持 simple enum（不含关联数据的 ADT 变体）。- 值必须为整数字面量（允许负值）。- 若任一变体有显式值，则所有变体都必须有（不可混用自动和手动）。- 重复值会产生编译错误。- `print()` 显示变体名称，而非整数值。

## ADT 变体的命名字段

ADT 变体字段可以选择性地包含名称以用于文档目的。命名字段使定义具有自描述性，而不改变构造或模式匹配的语义。

```
enum Shape:
  Circle(radius: float)
  Rect(width: float, height: float)
  Point
```

- 构造始终是位置性的：`Shape::Circle(3.14)`，而非 `Shape::Circle(radius: 3.14)`。
- 模式匹配绑定用户选择的变量名：`case Shape::Circle(r):`。
- 字段名必须为 `snake_case`。不允许在单个变体内混用命名和未命名字段。
- 未命名语法 (`Circle(float)`) 仍然有效。

## 约束与错误

| 约束                                       | 详细                                                   |
|------------------------------------------|------------------------------------------------------|
| 变体访问为 <code>EnumName::VariantName</code> | 必须使用 <code>::</code> 运算符                             |
| 变体值                                      | 默认为自动分配 (0, 1, 2, ...)，可使用 <code>= value</code> 显式指定 |
| 比较为整数比较                                  | 可使用 <code>==</code> 、 <code>!=</code>                |
| 命名字段名                                    | 必须为 <code>snake_case</code> ；同一变体内不可重复；不可混用命名/未命名    |

[English](#) | [日本語](#) | [繁體中文](#)

## 测试功能

Ry 内建 RSpec 风格的测试语法。使用 `ry test` 子命令执行测试文件。

### 运行测试

```
ry test          # 自动发现并运行项目内所有 *.test.ry 文件
ry test tests/spec # 递归运行指定目录下所有 *.test.ry 文件
ry test test_file.ry # 运行指定的测试文件
ry test -p       # 并行运行所有测试 (-p 或 --parallel)
ry test -p tests/ # 并行运行指定目录的测试
ry test -w       # 监视模式：文件变更时自动重新运行测试 (-w 或 --watch)
ry test -w -p    # 监视模式 + 并行运行
ry test -w tests/ # 监视指定目录
ry test --coverage # 所有测试 + 行覆盖率摘要
ry test --cov      # --coverage 的简写
```

退出码为 0 表示所有测试通过，1 表示有测试失败。

## 自动发现模式

不带参数运行 `ry test` 时：

1. 搜索 `package.toml` 以找到项目根目录
2. 在项目根目录下递归发现所有 `*.test.ry` 文件（`.git`、`build`、`node_modules` 会被跳过）
3. 逐一运行并汇总结果

---

## 语法

### `describe` / `it`

```
describe("description", fn():
  it("test case name", fn():
    # test body
    expect(actual_value).to_eq(expected_value)
  )
)
```

- `describe` 和 `it` 接受描述字符串和 `lambda` 参数 `fn()`：作为第二个参数
- `describe` 块内可以编写 `it` 块及其他语句（如变量声明等）
- 每个 `it` 块为独立的测试用例
- `describe` / `expect` 仅在 `ry test` 中可用（在普通的 `ry` 执行中会产生编译错误）

### 尾随块语法

任何函数调用（`describe`/`it`/`mock` 除外）都可以使用尾随块语法。在 `()` 后加上 `:` 会将缩进块作为无参数 `lambda` 传入最后的参数位置：

# 以下两者等价：

```
foo("arg"):
  bar()
```

```
foo("arg", fn():
  bar()
)
```

### `expect` / 匹配器

| 匹配器                                      | 说明                          | 支持类型                  |
|------------------------------------------|-----------------------------|-----------------------|
| <code>to_eq(expected)</code>             | 相等比较                        | int, float, bool, str |
| <code>to_not_eq(expected)</code>         | 不相等                         | int, float, bool, str |
| <code>to_be_true()</code>                | 为 <code>true</code>         | bool                  |
| <code>to_be_false()</code>               | 为 <code>false</code>        | bool                  |
| <code>to_be_none()</code>                | 为 <code>None</code>         | Option                |
| <code>to_be_some()</code>                | Option 为 <code>Some</code>  | Option                |
| <code>to_be_ok()</code>                  | Result 为 <code>Ok</code>    | Result                |
| <code>to_be_err()</code>                 | Result 为 <code>Err</code>   | Result                |
| <code>to_contain(val)</code>             | 容器包含值                       | List, Set, Map, str   |
| <code>to_not_contain(val)</code>         | 容器不包含值                      | List, Set, Map, str   |
| <code>to_be_greater_than(v)</code>       | <code>actual &gt; v</code>  | int, float            |
| <code>to_be_less_than(v)</code>          | <code>actual &lt; v</code>  | int, float            |
| <code>to_be_greater_than_or_eq(v)</code> | <code>actual &gt;= v</code> | int, float            |
| <code>to_be_less_than_or_eq(v)</code>    | <code>actual &lt;= v</code> | int, float            |
| <code>to_have_length(n)</code>           | 长度为 <code>n</code>          | List, Set, Map, str   |

| 匹配器                                | 说明             | 支持类型                |
|------------------------------------|----------------|---------------------|
| <code>to_be_empty()</code>         | 长度为 0          | List, Set, Map, str |
| <code>to_start_with(prefix)</code> | 字符串以 prefix 开头 | str                 |
| <code>to_end_with(suffix)</code>   | 字符串以 suffix 结尾 | str                 |

## fail

立即将当前测试标记为失败。

```
it("should not reach here", fn():
  fail("unexpected error")
)
```

- `fail()` — 使用通用消息标记测试失败
- `fail(msg)` — 使用自定义消息标记测试失败
- `fail()` 之后执行继续进行（不会中止测试）
- 仅在 `ry test` 模式下可用

## 输出格式

```
Calculator
+ adds numbers
+ subtracts
- fails test (red)
  line 10: expected 3, got 2
```

2 passed, 1 failed

- + 为成功（绿色），- 为失败（红色）
- 失败时显示行号与预期值/实际值

## 示例

```
describe("Arithmetic", fn():
  it("adds integers", fn():
    expect(1 + 2).to_eq(3)
  )
  it("compares strings", fn():
    expect("hello").to_eq("hello")
  )
  it("checks booleans", fn():
    expect(3 > 1).to_be_true()
  )
)
describe("Booleans", fn():
  it("false check", fn():
    expect(1 > 2).to_be_false()
  )
)
```

---

## 模拟 (Mock)

### `mock(fn_name, replacement)`

在当前 `it` 块中将函数替换为模拟实现。 `it` 块结束时模拟会自动清除。

```
fn fetch_data() -> str:
    return "real data"

describe("mocking", fn():
    it("replaces function", fn():
        mock(fetch_data, fn() => "fake")
        expect(fetch_data()).to_eq("fake")
    )
    it("auto-restores", fn():
        expect(fetch_data()).to_eq("real data")
    )
)
```

- 第一个参数为函数名称 (标识符, 非字符串)
- 第二个参数为替换用 `lambda`
- 替换函数必须与原始函数具有相同的参数类型和返回类型
- `it` 块结束时模拟会自动恢复

### `verify(fn_name)`

返回模拟函数被调用的次数 (`int`)。

```
describe("verify", fn():
    it("counts calls", fn():
        mock(fetch_data, fn() => "fake")
        fetch_data()
        fetch_data()
        expect(verify(fetch_data)).to_eq(2)
    )
)
```

## 限制

- 不支持重载函数的模拟
- 不支持使用捕获闭包进行模拟 (仅支持纯 `lambda`)
- 不支持 `@native fn` 的模拟

---

## 参数化测试 (@each)

`@each` 可以用多组参数运行同一个测试。将元组列表附加到 `it` 块:

```
@each([
    (1, 2, 3),
    (0, 0, 0),
    (-1, 1, 0)
])
it("adds {0} + {1} = {2}", fn(a: int, b: int, expected: int):
```

```
    expect(a + b).to_eq(expected)
)
```

- 列表必须包含与 lambda 参数数量相同的元组
  - 描述中的 {0}, {1}, ... 会被替换为参数值
  - 每个元组生成一个独立的测试用例
  - 支持的参数类型: int, float, bool, str
- 

## 基于属性的测试 (@property)

@property 生成随机输入并多次运行测试:

```
@property(count=100)
it("addition is commutative", fn(a: int, b: int):
    expect(a + b).to_eq(b + a)
)
```

- count=N 指定随机试验次数 (默认: 100)
  - 失败时会显示反例 (导致失败的输入值)
  - 在第一次失败时停止测试
  - 支持的参数类型: int ([-1000, 1000])、float ([-1000.0, 1000.0])、bool、str (随机 ASCII、0-20 字符)
- 

## 测试覆盖率

使用 --coverage (或 --cov) 标志来测量行覆盖率:

```
ry test --coverage          # 所有测试 + 覆盖率摘要
ry test --cov tests/spec/math.test.ry # 单个文件
ry test --coverage tests/spec/      # 目录
```

### 输出

```
Test Coverage Summary:
tests/spec/math.test.ry    100.0% (74/74 lines)
tests/spec/strings.test.ry 92.3%  (24/26 lines)
-----
Total                      95.1%  (98/100 lines)
```

- 仅报告用户代码; 标准库文件会被排除
  - --coverage 与 --parallel 同时指定时, 会退回为顺序执行
- 

## 限制

- 不支持 describe 的嵌套
- 不支持 before\_each / after\_each

[English](#) | [日本語](#) | [简体中文](#)

## 类型参考

### 类型一览

| 类型                         | 内部表示                                      | 字面值示例                              | 说明                                     |
|----------------------------|-------------------------------------------|------------------------------------|----------------------------------------|
| int                        | i64                                       | 42, -7, 0xFF, 0b1010               | 64 位有符号整数                              |
| u8                         | i8                                        | (无专用字面值)                           | 无符号 8 位整数 (0-255)。通过类型标注 b: u8 = 42 使用 |
| float                      | f64                                       | 3.14, 0.5                          | 64 位浮点数                                |
| bool                       | il                                        | true, false                        | 布尔值                                    |
| str                        | ptr                                       | "hello", "", "a\nb"                | 字符串（堆上的不可变字节序列）                        |
| Unit                       | void                                      | (无返回值)                             | 无返回值函数的返回类型。必须用-> Unit 显式指定            |
| Option<T><br>(T1, T2, ...) | { i1, T }<br>LLVM StructType<br>(literal) | Some(42), None<br>(1, 3.14)        | 可能存在值的类型<br>元组类型                       |
| List<T>                    | ptr (堆)                                   | [1, 2, 3]                          | 动态数组                                   |
| Map<K, V>                  | ptr (堆)                                   | {"a": 1}                           | 哈希映射                                   |
| Set<T>                     | ptr (堆)                                   | {1, 2, 3}                          | 不重复的集合                                 |
| fn(T1, T2) -> R            | ptr (函数指针)                                | fn(x: int) => x * 2                | 函数类型                                   |
| 用户定义类型                     | LLVM StructType<br>(named)                | record Point: ...                  | 以 record 关键字定义的结构体                     |
| enum                       | i64 / 标签联合                                | Color::Red,<br>Shape::Circle(3.14) | 以 enum 关键字定义的枚举类型（支持关联数据）              |
| Error                      | { ptr, i64 }                              | Error("msg"),<br>Error("msg", 404) | 内置错误类型                                 |
| any                        | { i64, [8 x i8] }                         | x: any = 42                        | 可持有任意基本值的标签联合                          |
| T1 \   T2                  | { i64, [N x i8] }                         | int \   str                        | union 类型（可持有多种类型之一）                    |
| Int 字面量                    | i64                                       | 42, 0 \   1                        | int 字面量类型（值约束）                         |
| String 字面量                 | ptr                                       | "N" \   "S"                        | str 字面量类型（值约束）                         |
| 范围                         | i64                                       | 1..12, -10..10                     | 范围类型（包含两端的整数范围约束）                      |
| i8                         | i8                                        | x: i8 = 42, x = 42i8               | 8 位有符号整数（低级，无隐式转换）                     |
| i16                        | i16                                       | x: i16 = 100, x = 100i16           | 16 位有符号整数（低级，无隐式转换）                    |
| i32                        | i32                                       | x: i32 = 42, x = 42i32             | 32 位有符号整数（低级，无隐式转换）                    |
| i64                        | i64                                       | x: i64 = 100, x = 100i64           | 64 位有符号整数（低级，无隐式转换）                    |
| u8                         | i8                                        | x: u8 = 200, x = 200u8             | 8 位无符号整数（低级，无隐式转换）                     |
| u16                        | i16                                       | x: u16 = 60000, x = 60000u16       | 16 位无符号整数（低级，无隐式转换）                    |
| u32                        | i32                                       | x: u32 = 3000000000, x = 100u32    | 32 位无符号整数（低级，无隐式转换）                    |
| u64                        | i64                                       | x: u64 = 100, x = 100u64           | 64 位无符号整数（低级，无隐式转换）                    |
| f32                        | float                                     | x: f32 = 3.14, x = 3.14f32         | 32 位浮点数（低级，无隐式转换）                      |
| [T; N]                     | [N x T]                                   | buf: [i32; 8]                      | 固定长度连续数组。低级类型 T 的 N 个元素（栈分配）           |

## 类型标注语法

声明变量时可以显式指定类型。当类型可推断时可以省略。

```
x: int = 42
b: u8 = 255
f: float = 3.14
s: str = "hello"
b: bool = true
opt: Option<int> = Some(10)
t: (int, float) = (1, 3.14)
xs: List<int> = [1, 2, 3]
m: Map<str, int> = {"a": 1}
s: Set<int> = {1, 2, 3}
fn_val: fn(int) -> int = fn(x: int) => x * 2
u: int | str = 42
a: any = 42
```

## 可用类型名称一览

| 类型名称             | 备注                                                                                                                                                  |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| int              | 内置标量类型                                                                                                                                              |
| u8               | 内置标量类型（无符号 0-255）                                                                                                                                   |
| float            | 内置标量类型                                                                                                                                              |
| bool             | 内置标量类型                                                                                                                                              |
| str              | 内置字符串类型                                                                                                                                             |
| Unit             | 无返回值函数的返回类型                                                                                                                                         |
| Option<T>        | 泛型类型（T 为任意类型）                                                                                                                                       |
| (T1, T2, ...)    | 元组类型（元素数量和类型组合任意）                                                                                                                                   |
| List<T>          | 泛型动态数组类型                                                                                                                                            |
| Map<K, V>        | 泛型哈希映射类型                                                                                                                                            |
| Set<T>           | 泛型集合类型                                                                                                                                              |
| fn(T1, ...) -> R | 函数类型                                                                                                                                                |
| Error            | 内置错误类型（message: str、code: int）                                                                                                                      |
| any              | 可持有任意基本值（int, float, bool, str）或 Unit 的内置类型。支持隐式转换：具体值赋值给 any 时自动包装，any 值赋值给具体类型时自动解包（带运行时类型检查）。支持 any(int) → float 的自动提升。详见 <a href="#">any 类型</a> |
| T1 \  T2 \  ...  | union 类型（以 \  分隔的多个类型之一）                                                                                                                            |
| i8               | 低级 8 位有符号整数（无隐式转换）                                                                                                                                  |
| i16              | 低级 16 位有符号整数（无隐式转换）                                                                                                                                 |
| i32              | 低级 32 位有符号整数（无隐式转换）                                                                                                                                 |
| i64              | 低级 64 位有符号整数（无隐式转换）                                                                                                                                 |
| u8               | 低级 8 位无符号整数（无隐式转换）                                                                                                                                  |
| u16              | 低级 16 位无符号整数（无隐式转换）                                                                                                                                 |
| u32              | 低级 32 位无符号整数（无隐式转换）                                                                                                                                 |
| u64              | 低级 64 位无符号整数（无隐式转换）                                                                                                                                 |
| f32              | 低级 32 位浮点数（无隐式转换）                                                                                                                                   |
| [T; N]           | 低级类型 T 的 N 个元素的固定长度数组。栈分配，连续内存。支持索引读写和 length()                                                                                                     |
| 用户定义类型名称         | 以 record 或 enum 关键字声明的类型                                                                                                                            |

## 类型别名

`type` 关键字为现有类型创建新的名称。别名与原始类型完全互换。

```
type Meters = float
type StringList = List<str>
```

```
d: Meters = 3.14
names: StringList = ["Alice", "Bob"]
```

命名约定：类型别名名称必须使用 PascalCase (如 `Meters`、`StringList`)。编译器会强制执行此约定。

类型别名也可以用于函数类型、字面量类型和范围类型：

```
type Callback = fn(int, int) -> int

add: Callback = fn(a: int, b: int) => a + b
print(add(3, 4))    # 7

type Month = 1..12
type Direction = "N" | "S" | "E" | "W"
type Digit = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

m: Month = 6
d: Direction = "N"
n: Digit = 5
```

---

## 字面量类型

字面量类型将变量的值限制为特定的常量值。对于常量值，在编译时进行约束检查；对于动态值，在运行时进行约束检查。

### int 字面量类型

```
x: 42 = 42           # 单一字面量类型
y: 0 | 1 = 0         # int 字面量的 union
z: 0 | 1 = 0
z = 1                # OK
# z = 2              # 编译错误（常量）或运行时错误（动态值）
```

### str 字面量类型

```
dir: "N" | "S" | "E" | "W" = "N"
# @const bad: "N" | "S" = "X"    # 编译错误
```

### 约束检查

- 编译时：当赋值为常量（`ConstantInt` 或字符串字面量）时，在编译时检查，违反时产生编译错误
  - 运行时：当值为动态（如函数返回值）时，在运行时检查，违反时程序以错误退出
- 

## 范围类型

范围类型将整数变量的值限制在连续的范围内（包含两端）。



```
month: 1..12 = 6          # OK
# @const bad: 1..12 = 0    # 编译错误：超出范围
# @const bad: 1..12 = 13   # 编译错误：超出范围

t: -10..10 = -5          # 支持负数范围
```

使用可变量重新赋值（运行时检查）

```
x: 1..12 = 6
x = 12          # OK
# x = dynamic_value() # 运行时检查：超出范围则错误退出
```

在函数参数中使用

```
fn set_month(m: 1..12) -> int:
    return m

set_month(6)          # OK
# set_month(13)       # 编译错误（常量参数）
```

---

## none 关键字与 Option 类型简写

none 关键字表示 Option 类型的值不存在，等同于 None。

T? 语法是 Option<T> 的简写。

```
x: int? = 42          # 等同于 Option<int>
y: int? = none        # 等同于 None

fn find(xs: List<int>, val: int) -> int?:
    for x in xs:
        if x == val:
            return Some(x)
    return none
```

---

## F-String（字符串插值）

使用 f"..." 语法进行字符串插值。{} 内的表达式会被求值并转换为字符串。

```
name = "world"
print(f"Hello {name}")    # Hello world

a = 1
b = 2
print(f"{a} + {b} = {a + b}") # 1 + 2 = 3
```

插值中支持的类型

{ } 内可使用求值结果为 int、float、bool 或 str 的任意表达式。

转义序列

| 序列             | 输出        |
|----------------|-----------|
| {{             | { (字面大括号) |
| }}             | } (字面大括号) |
| \n \r \t \\ \" | 与普通字符串相同  |

```
print(f"{{{braces}}}") # {braces}
```

### 类型转换 (as)

使用 `as` 关键字进行显式的类型转换。

```
x = 42 as float      # 42.0
y = 3.14 as int      # 3
z = 1 as bool        # true
s = 42 as str        # "42"
b = 255 as u8        # u8 值 255
```

### 支持的转换

| 来源                 | 目标    | 行为                  |
|--------------------|-------|---------------------|
| int                | float | SIToFP              |
| float              | int   | 截断 (FPToSI)         |
| int                | bool  | 0 → false、非零 → true |
| bool               | int   | false → 0、true → 1  |
| int / float / bool | str   | 字符串表示               |
| int                | u8    | 截断 (低 8 位)          |
| u8                 | int   | 零扩展                 |

```
int | i8 / i16 / i32 / i64 | 截断 (或 i64 时为恒等) |
i8 / i16 / i32 / i64 | int | 符号扩展 (SExt) |
int | u8 / u16 / u32 / u64 | 截断 (或 u64 时为恒等) |
u8 / u16 / u32 / u64 | int | 零扩展 (ZExt) |
有符号 | 有符号 (更宽) | 符号扩展 (SExt) |
有符号 | 有符号 (更窄) | 截断 |
无符号 | 无符号/有符号 (更宽) | 零扩展 (ZExt) |
无符号 | 无符号/有符号 (更窄) | 截断 |
有符号 / 无符号整数 | float | SIToFP / UIToFP 然后 f64 |
float | 有符号 / 无符号整数 | FPToSI / FPToUI |
float | f32 | FPTrunc |
f32 | float | FPExt |
有符号整数 | f32 | SIToFP |
无符号整数 | f32 | UIToFP |
f32 | 有符号 / 无符号整数 | FPToSI / FPToUI |
```

不支持的转换 (例如 `str as int`) 会产生编译错误。字符串转数值请使用 `to_int()` / `to_float()`。

### 带关联数据的 enum (ADT)

在变体名称后面加上括号并指定类型，`enum` 变体就可以携带关联数据。不带括号的变体仍然是单纯的标签。

```
enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point
```

## 命名字段

变体可以选择性地使用命名字段以提高文档清晰度。命名字段使变体定义具有自描述性，但不改变运行时行为——构造和模式匹配仍然是位置性的。

```
enum Shape:
    Circle(radius: float)
    Rectangle(width: float, height: float)
    Point
```

规则：- 字段名必须为 `snake_case`。- 在单个变体内，所有字段必须全部命名或全部未命名（不可混用）。- 同一变体内的重复字段名是编译错误。

## 构造函数

使用 `EnumName::Variant(value)` 语法构建带有数据的变体。参数始终是位置性的，即使字段有名称也是如此。

```
c = Shape::Circle(3.14)
r = Shape::Rectangle(4.0, 5.0)
p = Shape::Point
```

## 带绑定的模式匹配

使用 `case EnumName::Variant(binding):` 形式取出关联数据。绑定使用用户选择的变量名，而非字段名。

```
match c:
    case Shape::Circle(r):
        print(r)           # 3.14
    case Shape::Rectangle(w, h):
        print(w)
        print(h)
    case Shape::Point:
        print("point")
```

## 内部表示

ADT `enum` 以标签联合的形式存储：`{ i64 tag, [N x i8] data }`，`N` 的大小足以容纳最大变体的载荷。

---

## 泛型 `enum`

`enum` 可以使用角括号语法 `<T>` 带有类型参数，使相同的 `enum` 结构可以持有不同类型的载荷。

```
enum MyOption<T>:
    MySome(T)
    MyNone
```

## 使用方式

提供具体的类型参数来实例化。当编译器无法推断类型时，需要显式指定类型参数。

```
a = MyOption<int>::MySome(42)
b = MyOption<int>::MyNone
```

```
match a:
    case MyOption::MySome(v):
        print(v)           # 42
    case MyOption::MyNone:
        print("none")
```

---

## Error 类型

用于错误处理的内置类型。Error 具有两个字段：message (str) 和 code (int)。

```
e = Error("something went wrong")           # code 默认为 0
e2 = Error("not found", 404)                 # 显式指定 code

print(e.message)    # something went wrong
print(e2.code)      # 404
print(e2)           # Error: not found (code: 404)
```

## 使用 Result 进行错误处理

可能失败的函数返回 Result<V, E>：

```
fn divide(a: int, b: int) -> Result<int, Error>:
    if b == 0:
        return Err(Error("division by zero"))
    return Ok(a // b)

match divide(10, 2):
    case Ok(v):
        print(v)           # 5
    case Err(e):
        print(e.message)
```

当返回值没有意义时，使用 Result<Unit, Error>：

```
fn save(path: str, data: str) -> Result<Unit, Error>:
    return Ok(0 as u8)     # Unit 占位符

match save("/tmp/test.txt", "hello"):
    case Ok(_):
        print("saved")
    case Err(e):
        print(e.message)
```

## Result 类型

Result<V, E> 是一个内置的参数化类型，有两个构造函数：

- Ok(value) — 成功变体
- Err(error) — 错误变体

与 match 配合使用进行穷举的错误处理。Ok 和 Err 两种情况都必须覆盖（或使用 \_ 通配符）。

测试匹配器：- expect(x).to\_be\_ok() — 断言结果为 Ok - expect(x).to\_be\_err() — 断言结果为 Err

## 内部表示

Error 以 { ptr message, i64 code } 表示。Result<V, E> 以 { i1 isOk, V okValue, E errValue } 表示。

## union 类型

可以使用 | 声明可能持有多种类型的变量。

```
x: int | str = 42
x = "hello"      # 可重新赋值 (union 中的任一类型)
print(x)         # hello
```

在函数参数与返回值中的使用

```
fn show(x: int | str) -> int:
    print(x)
    return 0

fn get_val(flag: bool) -> int | str:
    if flag:
        return 42
    return "hello"
```

内部表示

union 类型以 { i64 tag, [N x i8] data } 表示。tag 表示各组成类型的索引（按字母顺序排序后），data 是最大组成类型大小的字节数组。

约束

- 赋值不属于 union 的类型会产生编译错误
- int | str 和 str | int 是相同的类型（会被规范化）
- 使用 print() 输出 union 值时，会根据运行时的 tag 以适当的类型显示

any 类型

any 类型是一种内置的动态类型，可以持有任意基本值。它采用类似 Python 的灵活类型方式——当不需要静态类型保证时，any 让您无需使用泛型或 union 类型即可处理多种类型。

支持的类型

any 可以持有以下类型：

| 类型    | 标签 | 说明                |
|-------|----|-------------------|
| int   | 0  | 64 位有符号整数         |
| float | 1  | 64 位浮点数           |
| bool  | 2  | 布尔值               |
| str   | 3  | 字符串               |
| Unit  | 4  | Unit 值（用于无返回值的函数） |

any 无法持有集合类型（List、Map、Set）、资源类型（TcpListener、TcpStream 等）、函数指针或用户定义类型（record、enum）。

内部表示

any 以标签联合实现：

```
{ i64 tag, [8 x i8] data }    // 共 16 字节
tag 字段标识存储的类型，data 字段持有值（最多 8 字节）。
```

## 包装与解包

具体类型的值在赋值给 `any` 时自动包装，`any` 的值在赋值给具体类型时自动解包。

```
# 包装：具体类型 → any
x: any = 42          # int 被包装成 any
x = "hello"         # 可以重新赋值为不同类型

# 解包：any → 具体类型
fn get_value() -> any:
    return 42
n: int = get_value() # any(int) 被解包为 int

# 解包时的 int → float 自动提升
f: float = get_value() # any(int) 被解包并提升为 float
```

如果运行时类型与目标类型不符（例如将 `any(str)` 解包至 `int` 变量），会产生运行时错误。

## 重新赋值

`any` 变量可以重新赋值为任何可持有类型的值：

```
x: any = 42
x = 3.14    # OK: 现在持有 float
x = "hello" # OK: 现在持有 str
x = true    # OK: 现在持有 bool
```

## 算术运算

当两个操作数都是 `any` 时，运算会在运行时根据实际类型进行分派：

| 运算   | 类型                    | 结果          |
|------|-----------------------|-------------|
| +    | int + int             | int         |
| +    | float + float         | float       |
| +    | int + float           | float       |
| +    | str + str             | str（拼接）     |
| -    | 数值                    | int 或 float |
| *    | 数值                    | int 或 float |
| *    | str * int / int * str | str（重复）     |
| /    | 数值                    | float（总是）   |
| //   | int // int            | int         |
| //   | 含 float               | float       |
| %    | 数值                    | int 或 float |
| **   | 数值                    | float（总是）   |
| 一元 - | int                   | int         |
| 一元 - | float                 | float       |

当一个操作数是 `any`、另一个是具体类型时，具体值会在运算前自动包装。

```
x: any = 10
y: any = x + 20    # 20 被自动包装；结果是 any(int) = 30
```

不兼容的类型组合（例如 `str - int`）会导致运行时错误。

## 比较运算

| 运算              | 行为                         |
|-----------------|----------------------------|
| == 、 !=         | 相同类型之间有效；int/float 混合比较可行  |
| < 、 <= 、 > 、 >= | 数值（int/float 混合可）和字符串（字典序） |

```
x: any = 3
y: any = 3.0
print(x == y)    # true (int/float 比较)
```

比较时类型不符（例如 int < str）会导致运行时错误。

### 字符串转换

any 值支持 print() 和 f-string 插值：

```
x: any = 42
print(x)           # 42
print(f"value: {x}") # value: 42
```

转换规则：int → 十进制字符串、float → %g 格式、bool → "true"/"false"、str → 原样、Unit → "Unit"。

### 将 any 传递给有类型的函数

any 值可以传递给具有具体参数类型的函数。值会通过运行时类型检查自动解包：

```
fn add_one(x: int) -> int:
    return x + 1

v: any = 42
result = add_one(v)    # any(int) 被解包为 int；结果是 43
```

## 类型规则（运算时的类型转换）

| 运算              | 左操作数        | 右操作数                     | 结果类型        | 备注                                            |
|-----------------|-------------|--------------------------|-------------|-----------------------------------------------|
| + - *           | int         | int                      | int         | 低级类型：原生宽度的无符号运算，无隐式提升                         |
| + - *           | u8          | u8                       | u8          |                                               |
| + - *           | float 或 int | float 或 int（其中一方为 float） | float       | 隐式 float 提升                                   |
| /               | 任意数值        | 任意数值                     | float       | 始终为 float                                     |
| //              | 任意数值        | 任意数值                     | int 或 float | 向下取整除法（向-∞）；int 操作数结果为 int，含 float 则结果为 float |
| **              | 任意数值        | 任意数值                     | float       | 使用 libm pow                                   |
| %               | int         | int                      | int         |                                               |
| %               | float 或 int | float 或 int（其中一方为 float） | float       |                                               |
| +               | str         | str                      | str         | 字符串拼接                                         |
| == != < <= > >= | str         | str                      | bool        | 字典序比较                                         |
| == != < <= > >= | 数值或 bool    | 数值或 bool                 | bool        |                                               |
| in              | 任意          | Set                      | bool        | 元素是否包含在集合中                                    |

| 运算              | 左操作数    | 右操作数    | 结果类型 | 备注                 |
|-----------------|---------|---------|------|--------------------|
| & \   ^ ~ << >> | int     | int     | int  | 对 float 会产生错误      |
| + - *           | i32     | i32     | i32  | 低级类型：无隐式转换，需要相同类型  |
| / //            | i32     | i32     | i32  | 有符号整数除法 (SDiv)     |
| / //            | u32     | u32     | u32  | 无符号整数除法 (UDiv)     |
| %               | i32     | i32     | i32  | 有符号取余 (SRem)       |
| %               | u32     | u32     | u32  | 无符号取余 (URem)       |
| + - * /         | f32     | f32     | f32  |                    |
| == !=           | i32/u32 | i32/u32 | bool | 符号无关的相等比较          |
| < <= > >=       | i32     | i32     | bool | 有符号比较 (ICMP_SLT 等) |
| < <= > >=       | u32     | u32     | bool | 无符号比较 (ICMP_ULT 等) |
| >>              | i32     | i32     | i32  | 算术右移（保留符号位）        |
| >>              | u32     | u32     | u32  | 逻辑右移（零填充）          |
| **              | 低级      | 任意      | 错误   | 低级类型不支持幂运算符        |
| 混合              | 低级      | 不同      | 错误   | 混合低级和高级类型是编译错误     |

转义序列（str 字面值内）

| 序列 | 含义  |
|----|-----|
| \n | 换行  |
| \r | 回车  |
| \t | 制表符 |
| \\ | 反斜杠 |
| \" | 双引号 |
| \0 | 空字符 |

## 类型安全约束

- 隐式拓宽转换 — 函数调用中支持安全的拓宽转换： $u8 \rightarrow int$ 、 $u8 \rightarrow float$ 、 $int \rightarrow float$ 。对于二元运算符，混合 `int` 和 `float` 会触发 `float` 提升。`u8` 是低级类型，使用原生宽度的无符号运算；在二元运算符中混合 `u8` 与 `int` 是编译错误。窄化转换（例如  $float \rightarrow int$ ）不允许隐式进行。从 `int` 字面量到 `u8` 的窄化转换仅在带有类型标注 `b: u8 = 42` 时允许。
- 变量类型在声明时固定 — 一旦以 `int` 声明的变量，就无法重新赋值为 `float`。
- 位运算仅限 `int` — 对 `float` 或 `bool` 使用位运算会产生编译错误。
- 非 `bool` 类型也可用于条件式 — `if` 的条件式可使用 `int`（`0 = false`、非 `0 = true`）等 `bool` 以外的类型。
- 数值字面量后缀 — 低级类型可通过字面量后缀指定：`42i32`、`255u8`、`3.14f32`、`0xFFu8`、`0b1010u8`。带有 `float` 后缀的整数字面量（`42f32`）会产生浮点值。带有整数后缀的浮点字面量（`3.14i32`）是编译错误。超出范围的值（例如 `256u8`、`129i8`）也是编译错误。
- 低级数值类型（`i8`、`i16`、`i32`、`i64`、`u8`、`u16`、`u32`、`u64`、`f32`）无隐式转换—混合低级类型之间或低级与高级类型（`int`、`float`）之间会产生编译错误。请使用显式 `as` 转换。低级整数的 `/` 运算符执行整数除法（类似 Rust），而非浮点除法。有符号类型使用 `SDiv/SRem`，无符号类型使用 `UDiv/URem`。



- 有符号与无符号 — 有符号类型 (`i8`、`i16`、`i32`、`i64`) 使用有符号比较 (`ICMP_SLT` 等) 和算术右移 (`AShr`)。无符号类型 (`u8`、`u16`、`u32`、`u64`) 使用无符号比较 (`ICMP_ULT` 等) 和逻辑右移 (`LShr`)。`>>>` 运算符无论符号性如何, 始终执行逻辑移位。
- 低级整数溢出会回绕 — 低级整数类型的算术运算在溢出时使用补码回绕 (有符号) 或模运算 (无符号)。例如, `i32` 最大值 `2147483647 + 1` 会回绕为 `-2147483648`。这与 C 的行为一致。如果溢出是问题所在, 请使用高级 `int` 类型 (64 位)。如需显式溢出控制, 请使用 `checked_add/sub/mul` (返回 `Result<T, Error>`)、`saturating_add/sub/mul` (钳制到类型边界) 或 `wrapping_add/sub/mul` (自文档化的回绕行为)。参见[函数参考](#)。